

QuestHub

Comparador de ofertas de videojuegos

Proyecto fin de Ciclo Formativo

Kerin Aguilar, Mohamed Bada, Freddy De Andrade



Índice

1. Introducción	5
2. Definición del proyecto	6
2.1. Problema que resuelve.....	6
2.2. Objetivos del proyecto.....	6
2.3. Justificación	7
2.4. Alcance y limitaciones	7
2.5. Metodología de trabajo.....	8
3. Estudio previo y análisis	9
3.1. Análisis de plataformas similares	9
3.2. Necesidades del sistema y del usuario	9
3.3. Requisitos funcionales	10
3.4. Requisitos no funcionales.....	11
4. Tecnologías utilizadas.....	12
4.1. Frontend	12
4.2. Backend	12
4.3. Base de datos.....	13
4.4. APIs externas	14
4.5. Autenticación.....	14
4.6. Testing.....	15
4.7. Herramientas de desarrollo	15
4.8. Despliegue	16
5. Arquitectura del proyecto	17
5.1. Arquitectura general.....	17
5.2. Comunicación frontend-backend	18
5.3. Flujo de datos	18
5.4. Sistema de autenticación.....	19
5.5. Arquitectura de sincronización.....	20
6. Base de datos.....	22

6.1.	Diseño inicial y evolución del modelo	22
6.2.	Modelo entidad-relación final	23
6.3.	Tablas	24
6.4.	Relaciones de las tablas	30
6.5.	Triggers y automatización	30
7.	Backend	32
7.1.	Configuración Inicial	32
7.2.	Arquitectura	34
7.3.	Capa /config (Configuraciones)	36
7.4.	Capa /domain (Entity's y Repos)	40
7.5.	Capa /api (Controladores)	42
7.6.	Capa /service (Servicios)	44
7.7.	Capa /external (APIs externas)	49
7.8.	Capa /exceptions (Excepciones)	51
7.9.	Capa /infrastructure (Infraestructura)	51
7.10.	Capa /util (Entity's y Repos)	51
7.11.	Capa /validation (Validación)	52
8.	Frontend	53
8.1.	Configuración Inicial	53
8.2.	Arquitectura del frontend	54
8.3.	Componentes	56
8.4.	Paginas/Vistas	59
8.5.	Comunicación con backend	68
8.6.	Estados globales (Context)	69
8.7.	Rutas de navegación	70
8.8.	Modelos y constantes	72
8.9.	Utilidades (utils)	73
9.	Testing	75
9.1.	Frontend	75
9.2.	Backend	76
10.	Despliegue	77

10.1.	Integracion con GitHub (CI/CD)	77
10.2.	Despliegue en Neon (Base de datos).....	77
10.3.	Despliegue en Railway (Backend)	78
10.4.	Despliegue en Vercel (Frontend)	79
10.5.	Despliegue de Supabase (Authentication)	80
10.6.	Despliegue de Resend (Correos).....	82
11.	Problemas y errores	83
11.1.	Errores técnicos	83
11.2.	Errores no técnicos	86
12.	Conclusiones.....	88
12.1.	Conclusiones generales	88
12.2.	Aprendizaje y conocimientos adquiridos	88
12.3.	Objetivos alcanzados	89
12.4.	Mejoras a futuro	90
12.5.	Últimos pensamientos.....	91
13.	Bibliografia	92

1.Introducción

QuestHub es una aplicación web orientada a la comparación de ofertas de videojuegos para PC entre distintas tiendas digitales. La plataforma permite a los usuarios consultar ofertas actualizadas, visualizar información relacionada con videojuegos/bundles, y localizar rápidamente la oferta más económica disponible entre múltiples distribuidores.

El proyecto nace con el objetivo de centralizar información que normalmente se encuentra dispersa en diferentes plataformas, facilitando al usuario una experiencia más cómoda, rápida y visualmente clara. Actualmente, existen numerosas *tiendas digitales* de videojuegos, cada una con sus propias ofertas, promociones y catálogos, lo que obliga a los usuarios a consultar múltiples páginas para encontrar las mejores ofertas disponible.

La aplicación integra información procedente de *APIs externas*, principalmente **Steam** y **CheapShark**, cuyos datos son procesados y almacenados en nuestra base de datos, para crear nuestra API propia. De esta manera se permite desacoplar el *frontend* de las *APIs externas*, controlar mejor la información recibida y optimizar el acceso a los datos.

Se ha desarrollado siguiendo una arquitectura *full-stack* desacoplada, utilizando **React** y **TypeScript** para el *frontend*, **Spring Boot** y **Java** para el *backend* y **PostgreSQL** como sistema gestor de base de datos. Además, el proyecto incorpora sistemas de autenticación, sincronización automática de ofertas, cache y despliegue en entornos cloud.

Actualmente, la aplicación se encuentra en una fase *beta funcional*, incluyendo distintas funcionalidades como autenticación de usuarios, wishlist personalizada, alertas mediante correo electrónico, entre otras varias, aunque todavía quedan aspectos por pulir para poder considerarla terminada.

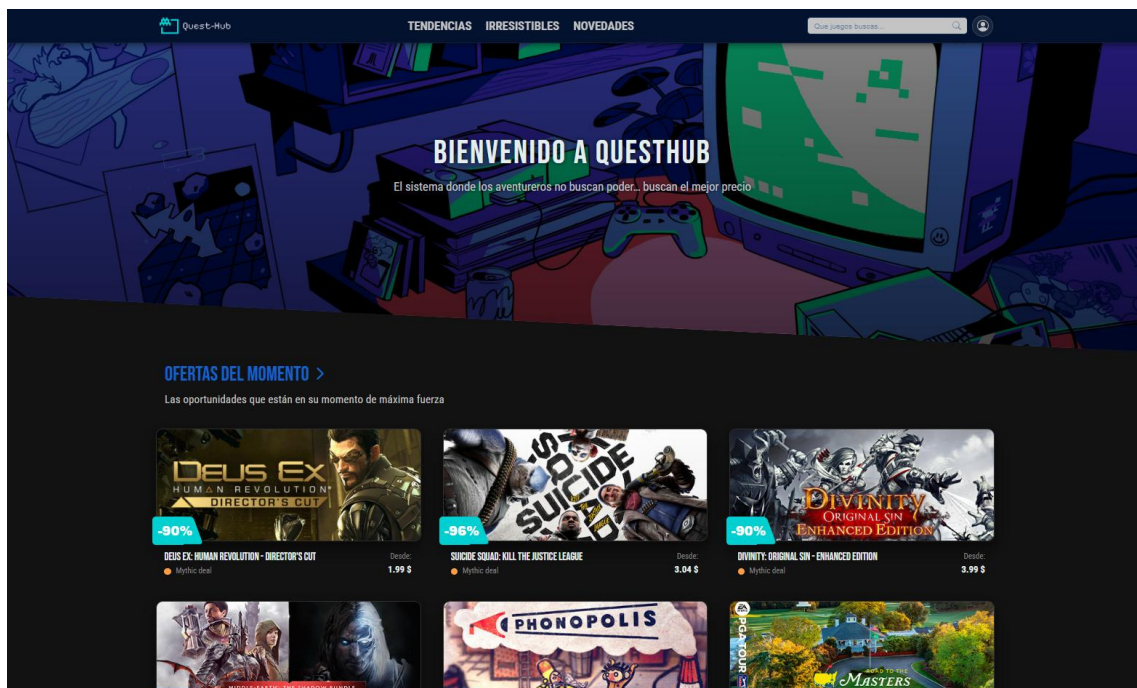


Figura 1. Página principal de QuestHub.

2. Definición del proyecto

2.1. Problema que resuelve

El mercado digital de videojuegos ha crecido notablemente en los últimos años gracias a plataformas como **Steam**, **Epic Games Store**, **GOG** o **Humble Bundle**, que ofrecen amplios catálogos y frecuentes ofertas. Sin embargo, esta diversidad de tiendas provoca una fragmentación de la información, lo que dificulta al usuario encontrar de forma rápida y eficiente la mejor oferta disponible para un videojuego/bundle, ya que debe consultar múltiples sitios de manera manual.

Además, algunos comparadores existentes presentan distintos niveles de complejidad en sus interfaces o en la organización de la información, lo que puede afectar en mayor o menor medida a la experiencia de usuario y el tiempo necesario para localizar una oferta en concreto.

En este contexto surge **QuestHub**, una plataforma minimalista y funcional que centraliza la información de precios de videojuegos, destacando siempre la oferta más económica y priorizando una experiencia de usuario clara, rápida y accesible.

2.2. Objetivos del proyecto

El objetivo principal del **TFG** es desarrollar una plataforma web funcional capaz de centralizar y comparar ofertas de videojuegos y bundles, procedentes de diferentes tiendas digitales, permitiendo al usuario consultar esta información de manera sencilla.

Además de este objetivo principal, el proyecto persigue una serie de objetivos técnicos y formativos relacionados con el desarrollo web moderno y la aplicación práctica de los conocimientos adquiridos durante el ciclo formativo de Desarrollo de Aplicaciones Web.

Entre los principales objetivos del proyecto se encuentran:

-  Diseñar una arquitectura desacoplada basada en *frontend*, backend y base de datos.
-  Desarrollar una API REST utilizando **Spring Boot**.
-  Integrar y procesar datos procedentes de distintas *APIs externas*.
-  Gestionar información persistente mediante **PostgreSQL**.
-  Desarrollar una interfaz moderna y *responsive* utilizando **React** y **TypeScript**.
-  Implementar un sistema de autenticación y gestión de usuarios mediante **Supabase**.
-  Permitir la gestión de wishlists por usuario.
-  Implementar tareas automáticas de sincronización de ofertas.
-  Optimizar el rendimiento mediante sistemas de *cache*.
-  Desplegar la aplicación en entornos reales.
-  Aplicar pruebas (*testing*) en frontend y backend.

Además, el proyecto busca acercarse a un entorno de desarrollo profesional real, utilizando herramientas, arquitecturas y metodologías ampliamente empleadas en el sector del desarrollo web.

2.3. Justificación

La elección de este proyecto se basa tanto en una motivación personal como en un interés técnico y formativo de mejorar lo que hemos aprendido. Como usuarios habituales de plataformas de videojuegos y comparadores de precios, surgió la idea de desarrollar una herramienta propia tomando como referencia aplicaciones existentes, con el objetivo de replicar sus funcionalidades principales e integrarlas en un diseño propio y personalizado, con un enfoque más *minimalista*.

Desde el punto de vista técnico, **QuestHub** destaca por la diversidad de áreas que abarca, al integrar tanto *frontend*, *backend*, *base de datos*, *integración de servicios* de terceros y *despliegue*.

Durante su desarrollo ha sido necesario trabajar con arquitecturas desacopladas, integración de APIs externas, bases de datos relacionales, autenticación de usuarios, despliegues, *testing*, sincronización automática de datos, optimización mediante sistemas de *caché* y mucho más.

Uno de los aspectos más relevantes ha sido el sistema de sincronización automática de ofertas. Debido a las limitaciones de las APIs externas y a la necesidad de mantener la disponibilidad del sistema, se implementó una estrategia basada en tareas programadas, tablas *staging* y mecanismos de *caché* que permiten actualizar la información sin afectar al usuario.

En conjunto, **QuestHub** permite aplicar y ampliar los conocimientos adquiridos durante el ciclo formativo, abordando problemas reales del desarrollo de aplicaciones web modernas, aumentar nuestros conocimientos y abordar nuevas tecnologías, formas de optimización y más.

2.4. Alcance y limitaciones

La versión actual del proyecto se encuentra en una fase *beta funcional* e incluye las principales funcionalidades necesarias para el uso básico de la plataforma.

Entre las funcionalidades implementadas y funcionales se encuentran algunas como:

-  autenticación e inicio/registro de usuarios
-  Una *wishlist* por usuario
-  consulta de videojuegos/bundles
-  visualización de ofertas
-  sincronización automática de datos
-  alertas mediante correo electrónico
-  despliegue funcional en entornos cloud

En cuanto al alcance, la plataforma se centra específicamente en la consulta y comparación de precios en oferta de videojuegos/bundle para PC, obteniendo información de distintos distribuidores digitales. Actualmente no se contempla el seguimiento del precio base de los videojuegos fuera de promociones, lo cual se contempla implementar en un futuro.

Sin embargo, debido a distintas limitaciones y el alcance académico del proyecto, algunas funcionalidades no han sido implementadas en esta versión final. Entre ellas se encuentran:

- historial de precios
- opción de cambiar o recuperar contraseña
- recomendaciones personalizadas
- integración de nuevas APIs (**RAWG**)
- gráficos estadísticos
- soporte para mercados de claves de terceros
- mejoras avanzadas de interfaz de usuario
- seguimiento de precios base en distintas tiendas

Otra limitación importante del sistema es la dependencia de APIs externas como **CheapShark** y **Steam**, ya que cualquier cambio en sus políticas, estructura de datos o límites de peticiones puede afectar al funcionamiento de la aplicación.

Asimismo, aunque el sistema ha sido diseñado con una arquitectura escalable y mantenible, existen aspectos susceptibles de mejora en futuras versiones, especialmente en términos de rendimiento, automatización de pruebas y optimización de procesos internos.

2.5. Metodología de trabajo

El desarrollo de **QuestHub** se realizó siguiendo un enfoque progresivo e incremental, comenzando por el diseño general de la aplicación y desarrollando posteriormente cada uno de los componentes del sistema de forma independiente.

Inicialmente se definieron los requisitos principales de la aplicación, así como las tecnologías que formarían parte del proyecto. Posteriormente se diseñó una arquitectura desacoplada basada en *frontend*, *backend* y *base de datos* independientes, permitiendo trabajar cada componente de forma separada y facilitando la escalabilidad del sistema y el progreso constante durante el desarrollo.

El desarrollo del *frontend* se realizó utilizando **React** y **TypeScript**, priorizando la reutilización de componentes, el diseño *responsive* y una interfaz clara y minimalista. Paralelamente, el backend fue desarrollado con **Spring Boot** y **Java**, mediante una arquitectura en capas basada en controladores, servicios, modelos, repositorios y otras.

A medida que se avanzaba, el proyecto fue incrementando su complejidad técnica. Aspectos como el despliegue en la nube, la sincronización automática de ofertas, el *testing* o la optimización mediante *caché* requirieron la investigación de nuevas herramientas y la adaptación de estas al proyecto, lo cual muchas veces modificaba los tiempos pensados para realizar distintas tareas.

El control de versiones se gestionó mediante **GitHub**, utilizando un flujo de trabajo basado en ramas (*branching model*) y *Pull Requests*, lo que permitió una mejor organización del desarrollo y revisión del código y errores antes de su integración.

Finalmente, el proyecto fue desplegado utilizando servicios cloud independientes para cada componente del sistema, acercando el entorno de desarrollo a un escenario de producción real.

3. Estudio previo y análisis

3.1. Análisis de plataformas similares

Antes del desarrollo de **QuestHub** se realizó un análisis de diferentes plataformas relacionadas con la comparación y/o venta de videojuegos/bundles. Entre las principales referencias utilizadas destacan **CheapShark**, **GG.deals**, **IsThereAnyDeal**, **Eneba** y **DekuDeals**.

Durante este análisis se estudiaron distintas características funcionales y de diseño presentes en este tipo de plataformas, especialmente relacionadas con la organización de la información, los sistemas de búsqueda, la comparación de precios y el seguimiento de ofertas.

Del análisis realizado se observó la existencia de distintos enfoques en cuanto a la presentación y organización de la información que se le daba a los usuarios. Esta variabilidad de maneras de mostrar mucha o poca información sirvió como base para definir un diseño centrado en la claridad, la estructura de los datos y un enfoque minimalista en la interfaz de usuario.

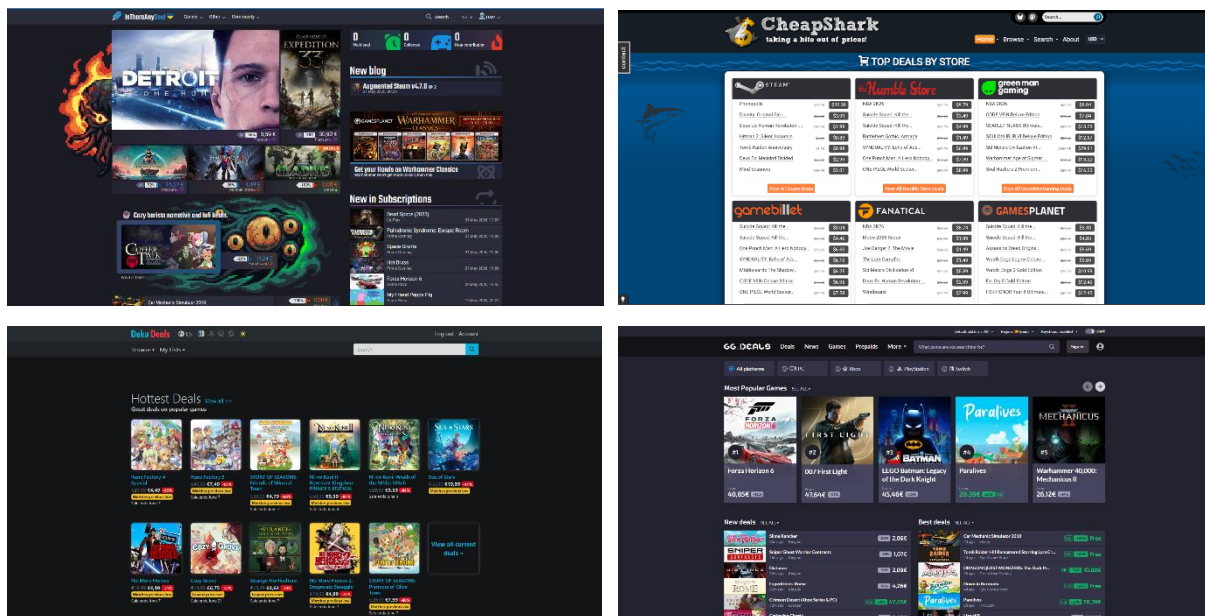


Figura 2. Comparativa visual de distintas plataformas de comparación de precios de videojuegos

3.2. Necesidades del sistema y del usuario

Tras el análisis, se identificaron distintas necesidades comunes entre los usuarios interesados en plataformas de comparación de precios de videojuegos/bundles.

La principal necesidad detectada fue la posibilidad de consultar ofertas de distintas tiendas desde un único lugar, evitando la necesidad de acceder manualmente a múltiples páginas web. También se consideró importante facilitar el acceso rápido a la información relevante, reduciendo elementos visuales innecesarios y mejorando la navegación general del sistema. A partir de esto, **QuestHub** fue diseñado priorizando:

-  una interfaz clara y minimalista

-  rapidez de acceso a la información
-  navegación intuitiva
-  centralización de ofertas
-  experiencia visual moderna
-  una paleta de colores oscuros
-  poder ser usada en distintos dispositivos

3.3. Requisitos funcionales

Requisito	Descripción
Registro e inicio de sesión	Autenticación mediante Supabase utilizando correo electrónico.
Sincronización de usuario	Sincronización del usuario autenticado con el backend mediante JWT.
Gestión de cuenta	Configuración de preferencias como notificaciones.
Eliminación de cuenta	Permite al usuario eliminar permanentemente su cuenta.
Consulta de ofertas	Visualización de videojuegos y bundles disponibles.
Mejor oferta	Identificación de la opción más económica disponible.
Detalle de producto	Acceso a información completa de cada videojuego o bundle.
Búsqueda dinámica	Búsqueda de ofertas en tiempo real.
Filtrado avanzado	Filtrado por título, tienda, precio, ahorro y valoración.
Filtrado en tiempo real	Actualización inmediata de resultados.
Acceso rápido	Navegación directa desde el header.
Gestión de wishlist	Añadir y eliminar videojuegos de la lista de deseos.
Visualización wishlist	Página para visualizar lista de deseos.
Organización	Ordenación según disponibilidad de ofertas, ahorro, precio, título, etc...
Envío de emails	Notificaciones automáticas por correo electrónico cuando cambia algo en la wishlist.
Control de notificaciones	Activación o desactivación de notificaciones.
Videos	Reproducción de videos.
Capturas	Visualización de imágenes asociadas al producto.
Actualización ofertas	Actualización mediante tareas programadas.
Integración externa	Uso de APIs como Steam y CheapShark.

Procesamiento asíncrono	Actualización sin bloquear el uso de la aplicación.
Página de inicio	Ofertas destacadas y acceso a búsqueda.
Detalle de producto	Vista individual de videojuegos o bundles.
Búsqueda avanzada	Página con filtros completos.
Wishlist	Gestión de favoritos del usuario.
Autenticación	Login y registro de usuarios.
Error 404	Página para rutas no encontradas.

3.4. Requisitos no funcionales

Requisito	Descripción
Rendimiento	El sistema debe mantener tiempos de respuesta reducidos incluso durante sincronizaciones masivas de datos. Para ello se implementaron mecanismos de <i>caché</i> mediante Spring Cache y Caffeine, además de procesamiento asíncrono para tareas pesadas.
Escalabilidad	La arquitectura se diseñó de forma desacoplada, separando frontend, backend y base de datos. Esto facilita la ampliación futura del sistema y la incorporación de nuevas funcionalidades sin afectar la estructura principal.
Disponibilidad	La aplicación debe permanecer operativa durante las actualizaciones de datos. Para ello se utiliza un sistema basado en tablas <i>staging</i> , evitando bloqueos sobre las tablas principales durante las sincronizaciones.
Usabilidad	La interfaz sigue un enfoque claro, intuitivo y minimalista, facilitando la navegación y reduciendo la sobrecarga visual habitual en plataformas similares.
Responsive Design	La aplicación se adapta correctamente a distintos dispositivos, incluyendo ordenadores, tablets y móviles, garantizando una experiencia consistente.
Mantenibilidad	El sistema utiliza una arquitectura en capas basada en separación de responsabilidades, facilitando el mantenimiento, la organización del código y su evolución futura.
Seguridad	La autenticación y gestión de sesiones se realiza mediante Supabase y JWT, evitando almacenar credenciales directamente en el sistema y utilizando configuraciones seguras tanto en backend como en despliegue.

4. Tecnologías utilizadas

El desarrollo se ha realizado utilizando distintas tecnologías y herramientas orientadas al desarrollo de aplicaciones web. La selección de cada una de ellas se llevó a cabo teniendo en cuenta aspectos como el rendimiento, la escalabilidad, la mantenibilidad y la facilidad de integración entre los diferentes componentes del sistema y nuestros conocimientos de estas.

4.1. Frontend

El frontend de la aplicación ha sido desarrollado utilizando **React** junto con **Vite**, proporcionando un entorno de desarrollo rápido y optimizado.

Para la implementación de la lógica de la interfaz se utilizó **TypeScript**, permitiendo incorporar tipado estático y mejorar la mantenibilidad del código.

La interfaz visual fue desarrollada utilizando únicamente **CSS personalizado**, sin emplear librerías externas de componentes UI, lo que permitió un mayor control sobre el diseño visual y la experiencia de usuario de la aplicación.

Entre las principales librerías y dependencias utilizadas en frontend destacan:

-  **React Router DOM**, para la gestión de rutas y navegación entre páginas.
-  **Axios**, utilizado para la comunicación con la API REST del backend.
-  **Framer Motion**, empleado para animaciones y transiciones de interfaz.
-  **React Toastify**, utilizado para mostrar notificaciones visuales al usuario.
-  **Supabase JS**, utilizado para la gestión de autenticación desde el cliente.
-  **HLS.js**, empleado para la reproducción de determinados vídeos y contenidos multimedia.

La gestión del estado de autenticación y usuario se realizó mediante **React Context** junto con **Supabase Auth**, permitiendo compartir la información de sesión y autenticación entre los distintos componentes de la aplicación.

La estructura del frontend se organizó en distintos directorios según su responsabilidad, separando componentes reutilizables, páginas, servicios, contextos, modelos y constantes, facilitando así la organización y mantenibilidad del proyecto.

4.2. Backend

El backend fue desarrollado utilizando **Spring Boot** y **Java**, siguiendo una arquitectura en capas basada en controladores, servicios, DTOs, mappers, modelos y repositorios.

La API REST desarrollada actúa como intermediaria entre el frontend, las APIs externas y la base de datos, permitiendo centralizar la lógica de negocio y desacoplar completamente la interfaz de usuario de las fuentes externas de datos.

Entre las principales tecnologías y dependencias utilizadas destacan:

-  **Spring Web**, para el desarrollo de endpoints REST.
-  **Spring Data JPA**, para la persistencia y gestión de entidades.
-  **Spring Cache** junto con **Caffeine**, para optimización del rendimiento mediante mecanismos de *cache*.
-  **Spring Scheduling**, utilizado para la ejecución automática de tareas programadas.
-  **Spring Async**, empleado para procesamiento asíncrono.
-  **Spring Mail**, utilizado para el envío de notificaciones mediante correo electrónico.
-  **Java JWT**, empleado para el manejo de tokens JWT.
-  **RestClient**, utilizado para la integración con APIs externas.

La arquitectura implementada separa la lógica del sistema en distintas capas mediante controladores, servicios, repositorios, DTOs, mappers y módulos específicos para APIs externas.

Esta organización facilita la mantenibilidad del código, la reutilización de lógica y la escalabilidad futura del sistema.

4.3. Base de datos

La persistencia de datos se realizó utilizando **PostgreSQL** como sistema gestor de base de datos relacional.

El acceso y gestión de los datos se implementó mediante **Spring Data JPA**, permitiendo trabajar con clases Java asociadas directamente a las tablas de PostgreSQL y simplificando las operaciones de persistencia y consulta.

La base de datos almacena información relacionada con:

-  videojuegos,
-  bundles,
-  ofertas,
-  tiendas,
-  usuarios,
-  wishlists,
-  géneros,
-  capturas,
-  videos.

El modelo relacional incorpora relaciones **OneToMany**, **ManyToMany** y tablas intermedias para gestionar asociaciones entre entidades.

Además, se implementó una tabla *staging* utilizada durante el proceso de sincronización automática de ofertas. Los datos obtenidos desde APIs externas se almacenan temporalmente en esta tabla antes de actualizar la tabla principal de ofertas, permitiendo mantener la disponibilidad del sistema durante las actualizaciones.

También se implementó una **vista materializada** utilizada para optimizar la consulta y visualización de ofertas dentro del catálogo principal. Gracias a esta vista, la aplicación muestra únicamente la oferta más económica disponible para cada videojuego o bundle dentro de las tarjetas del catálogo, evitando duplicados y reduciendo la cantidad de datos enviados al frontend. Posteriormente, al acceder a la página de detalles del producto, el sistema recupera y muestra el resto de ofertas disponibles asociadas a dicho videojuego o bundle.

4.4. APIs externas

La aplicación integra información procedente de APIs externas relacionadas con videojuegos y ofertas digitales.

Las principales APIs utilizadas fueron:

-  **CheapShark**, utilizada para obtener información relacionada con ofertas, precios y tiendas.
-  **Steam API**, utilizada para recuperar información detallada de videojuegos y bundles, como lo son contenido multimedia, descripciones, géneros, etc.

La integración con estas APIs se realizó mediante **RestClient** de Spring Boot, centralizando las configuraciones de acceso y facilitando la reutilización de conexiones externas.

Los datos obtenidos son procesados y almacenados posteriormente dentro de la base de datos propia del sistema, reduciendo la dependencia directa del frontend respecto a servicios externos.

4.5. Autenticación

El sistema de autenticación fue implementado mediante **Supabase Auth**.

La aplicación permite autenticación mediante correo electrónico y contraseña, así como autenticación OAuth utilizando proveedores externos como:

-  Google,
-  GitHub,
-  Discord.

Una vez autenticado el usuario, el frontend obtiene el token JWT generado por Supabase y sincroniza automáticamente la información del usuario con el backend.

El sistema almacena internamente el identificador UUID asociado al usuario autenticado, permitiendo relacionar funcionalidades como la wishlist, preferencias de usuario y sistema de notificaciones.

La validación y gestión de autenticación se delega completamente a Supabase, evitando almacenar contraseñas directamente dentro del sistema.

4.6. Testing

Para garantizar el correcto funcionamiento de la aplicación se han implementado pruebas tanto en el frontend como en el backend.

En el frontend se han utilizado herramientas como **Vitest**, **Testing Library** y **JSDOM**, permitiendo validar componentes, lógica de interfaz e interacciones del usuario mediante pruebas unitarias.

En el backend se han empleado herramientas del ecosistema de testing de **Spring Boot**, principalmente **JUnit** y **Mockito**, utilizadas para validar servicios, lógica de negocio e integración entre los distintos componentes del sistema.

Estas pruebas permiten asegurar el correcto comportamiento de la aplicación y la estabilidad del sistema durante su desarrollo y despliegue.

4.7. Herramientas de desarrollo

Durante el desarrollo del proyecto se han utilizado diversas herramientas que han facilitado la programación, depuración, control de versiones y gestión de la base de datos.

4.7.1. Entornos de desarrollo (IDEs)

-  **Visual Studio Code**: utilizado principalmente para el desarrollo del frontend en React y TypeScript, así como para la gestión general del código.
-  **IntelliJ IDEA**: utilizado para el desarrollo del backend con Spring Boot, facilitando la ejecución del servidor, gestión de dependencias y depuración.
-  **Eclipse / Spring Tool Suite (STS)**: utilizado como entorno adicional para el desarrollo y configuración del backend.

4.7.2. Control de versiones y repositorio

-  **Git**: sistema de control de versiones utilizado para el seguimiento del desarrollo del proyecto.
-  **GitHub**: plataforma utilizada para el almacenamiento del repositorio y la gestión de ramas y cambios mediante Pull Requests.

4.7.3. Base de datos y gestión

-  **PostgreSQL**: sistema gestor de base de datos utilizado en el proyecto.
-  **pgAdmin**: herramienta utilizada para la administración, consulta y visualización de la base de datos.

4.7.4. Herramientas de apoyo

-  **Inteligencia artificial (IA):** utilizada como herramienta de apoyo para resolución de dudas técnicas, optimización de código y asistencia en la redacción de documentación, siempre bajo supervisión del desarrollador.

4.8. Despliegue

El proyecto fue desplegado utilizando distintos servicios cloud independientes para cada componente del sistema. Las principales plataformas utilizadas fueron:

-  **Vercel**, para el despliegue del frontend.
-  **Railway**, para el despliegue del backend.
-  **Neon**, como proveedor de base de datos PostgreSQL.

Los despliegues se realizan de forma automática tras la actualización del repositorio remoto, gracias a una integración continua configurada entre GitHub y los distintos servicios de despliegue. En concreto, los cambios realizados en la rama destinada a despliegue (*Despliegue*) son detectados automáticamente, provocando la actualización y redeploy de la aplicación en los entornos correspondientes. De este modo, se facilita la publicación de nuevas versiones del sistema, centralizando las actualizaciones únicamente en dicha rama y simplificando el proceso de mantenimiento y despliegue.

5.Arquitectura del proyecto

5.1. Arquitectura general

La aplicación fue desarrollada siguiendo una arquitectura *full-stack desacoplada*, separando frontend, backend y base de datos en componentes independientes que se comunican mediante peticiones HTTP REST.

El proyecto se organiza dentro de un único repositorio (*monorepo*) git y el frontend y backend se encuentran separados en directorios independientes, permitiendo desarrollar y desplegar cada componente/capa de forma independiente.

La arquitectura general del sistema se compone de:

-  **Frontend** desarrollado con React y TypeScript.
-  **Backend** desarrollado con Spring Boot.
-  **Base de datos PostgreSQL** desplegada en Neon.
-  **APIs externas** utilizadas para obtener información y ofertas de videojuegos.
-  **Servicios cloud** independientes para el despliegue.

El frontend actúa únicamente como cliente visual, encargándose de mostrar la información y gestionar la interacción del usuario. Toda la lógica de negocio, procesamiento de datos, autenticación interna y acceso a la base de datos se realiza desde el backend.

Por su parte, el backend centraliza la comunicación con las APIs externas, procesa la información recibida y expone endpoints REST consumidos posteriormente por el frontend.

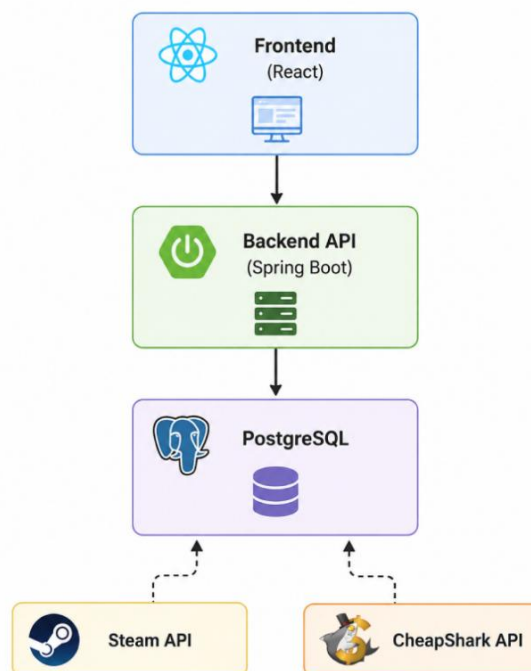


Figura 3. Arquitectura general del sistema.

5.2. Comunicación frontend-backend

La comunicación entre frontend y backend se realiza mediante una API REST propia desarrollada con Spring Boot.

El frontend utiliza **Axios** para realizar peticiones HTTP a los distintos endpoints disponibles en el backend, permitiendo consultar ofertas, videojuegos, bundles, wishlist, datos de usuario y más.

La información intercambiada entre ambos sistemas se transmite principalmente en formato JSON.

Además, se implementó un sistema de *interceptors* en Axios para detectar errores de conexión o caídas del backend, permitiendo mostrar notificaciones visuales al usuario mediante React Toastify.

Debido a que frontend y backend se encuentran desplegados en servicios y dominios independientes, fue necesario configurar políticas **CORS (Cross-Origin Resource Sharing)** en el backend para permitir la comunicación segura entre ambos componentes y controlar los orígenes autorizados para acceder a la API.

La arquitectura desacoplada utilizada permite mantener una separación clara entre la capa visual y la lógica de negocio, facilitando el mantenimiento y futuras ampliaciones del sistema.

5.3. Flujo de datos

El flujo de datos de la aplicación sigue una arquitectura centralizada donde el backend actúa como intermediario entre el frontend, la base de datos y las APIs externas.

Cuando un usuario realiza una búsqueda o consulta información desde la interfaz, el frontend envía una petición HTTP al backend mediante la API REST.

Para consultas generales, como el catálogo principal de ofertas, búsquedas rápidas o filtros, el sistema sigue un flujo estándar:

-  El frontend realiza una petición al backend.
-  El backend comprueba si la información solicitada se encuentra almacenada en *cache*.
-  Si los datos están cacheados, se devuelven directamente al frontend.
-  En caso contrario, el backend consulta la base de datos PostgreSQL.
-  Finalmente, los datos procesados son enviados nuevamente al frontend.

Este mecanismo permite reducir consultas repetitivas y mejorar considerablemente los tiempos de respuesta de la aplicación.

Sin embargo, determinados datos detallados de videojuegos y bundles utilizan una estrategia de *lazy loading*. En este caso, la información completa de determinados productos no se almacena previamente en la base de datos, sino únicamente cuando un usuario accede por primera vez a dicho contenido.

-  El flujo utilizado en estos casos es el siguiente:
-  El frontend solicita la información detallada de un videojuego o bundle.

- 📁 El backend verifica si dichos datos existen en la base de datos.
- 📁 Si la información ya existe, se devuelve directamente al frontend.
- 📁 En caso contrario, el backend consulta las APIs externas correspondientes.
- 📁 Con la información obtenida se construye una respuesta temporal que es enviada inmediatamente al frontend.
- 📁 Posteriormente, el almacenamiento completo de la información se realiza de manera asíncrona en segundo plano.
- 📁 Finalmente, los datos quedan persistidos en la base de datos y almacenados en *caché* para futuras consultas.

Gracias a este enfoque, el usuario puede visualizar la información rápidamente sin necesidad de esperar a que finalice todo el proceso de almacenamiento interno.

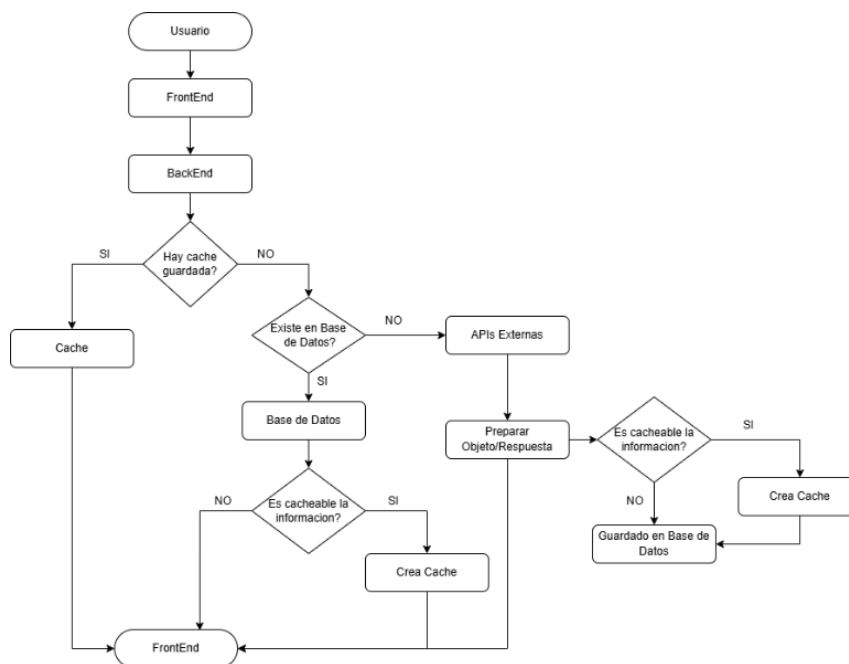


Figura 4. Diagrama de Flujo de manejo de datos

5.4. Sistema de autenticación

La autenticación de usuarios se implementó mediante Supabase Auth.

El proceso de autenticación comienza en el frontend, donde el usuario puede iniciar sesión mediante correo electrónico y contraseña o utilizando proveedores OAuth externos como Google, GitHub o Discord.

Una vez autenticado, Supabase genera un identificador único (*UUID*) asociado al usuario. Este identificador se sincroniza posteriormente con el backend, donde se crea un registro propio del usuario dentro de la base de datos si todavía no existe.

A partir de ese momento, el sistema utiliza dicho *UUID* para gestionar funcionalidades internas como:

-  wishlist personalizada,
-  preferencias de notificaciones,
-  y datos asociados a cada usuario.

De esta forma, la autenticación queda delegada completamente en Supabase, evitando almacenar contraseñas directamente dentro del sistema desarrollado.

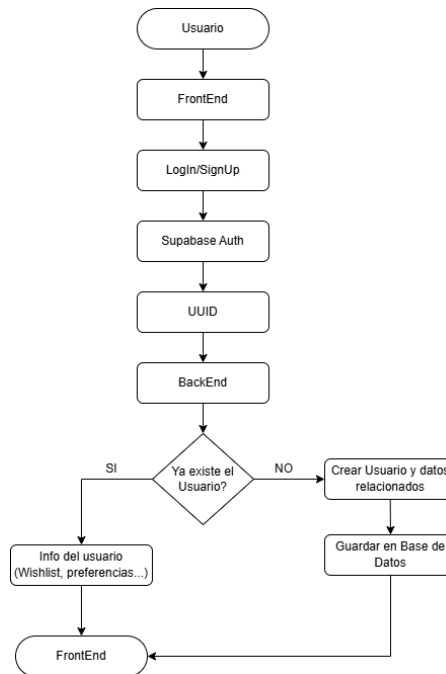


Figura 5. Diagrama de Flujo de autenticación/usuarios

5.5. Arquitectura de sincronización

La actualización de ofertas y tiendas se realiza mediante un sistema de sincronización automática implementado en el backend.

Dado que las ofertas de videojuegos y tiendas varían de forma constante y dependen de información procedente de APIs externas, se ha diseñado un mecanismo que permite mantener los datos actualizados sin comprometer la disponibilidad ni el acceso a la información por parte de los usuarios.

Este sistema se implementa mediante tareas programadas en Spring Boot utilizando la anotación **@Scheduled**, que ejecuta procesos de sincronización de forma periódica a partir de expresiones **CRON** definidas en el sistema. El flujo de sincronización de ofertas sigue los siguientes pasos:

-  Se ejecuta la tarea programada de sincronización.
-  El backend realiza peticiones a una de las API externa (CheapShark).
-  Se incorporan pequeños retardos (*delays*) entre peticiones para evitar superar límites de uso de esta.
-  Los datos obtenidos se procesan y validan antes de su almacenamiento.
-  La información se guarda temporalmente en la tabla *staging* (ofertas-staging).

- 📁 Una vez completada la sincronización, los datos de la tabla principal se reemplazan con la información nueva.
- 📁 La tabla *staging* se limpia para futuras sincronizaciones.
- 📁 Se actualiza la vista materializada utilizada para el catálogo de ofertas.
- 📁 Se limpia toda la cache guardada hasta el momento

Mientras que el flujo de sincronización de tiendas es más sencillo debido a la manera de que se obtienen los datos de la API y sigue los siguientes pasos:

- 📁 Se ejecuta la tarea programada de sincronización.
- 📁 El backend realiza la petición a una de las APIs externa (CheapShark).
- 📁 Los datos obtenidos se procesan y validan antes de editar la Base de datos.
- 📁 Se crean o eliminan las tiendas dependiendo de los datos procesados antes.
- 📁 Se actualiza la Base de datos.

Este enfoque permite evitar inconsistencias en los datos y garantiza que la aplicación siga funcionando correctamente durante todo el proceso de actualización.

Gracias a esta arquitectura, el sistema puede mantener la información actualizada de forma periódica sin interrumpir la experiencia del usuario ni bloquear el acceso a la aplicación.

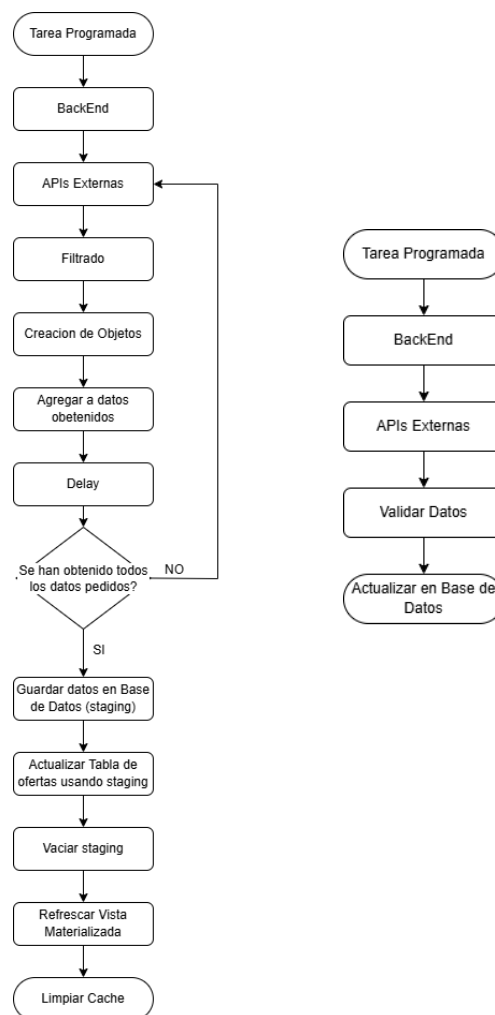


Figura 6. Diagramas de Flujo de tareas programadas

6. Base de datos

6.1. Diseño inicial y evolución del modelo

Antes de comenzar el desarrollo de la base de datos se realizó un diseño inicial del modelo entidad-relación con el objetivo de definir las principales entidades del sistema y sus relaciones, una vez definido esto se comenzó con el desarrollo utilizando **PostgreSQL**.

Durante este el diseño fue evolucionando progresivamente debido a nuevas necesidades funcionales, problemas detectados durante las pruebas y mejoras relacionadas con el rendimiento y la organización de los datos.

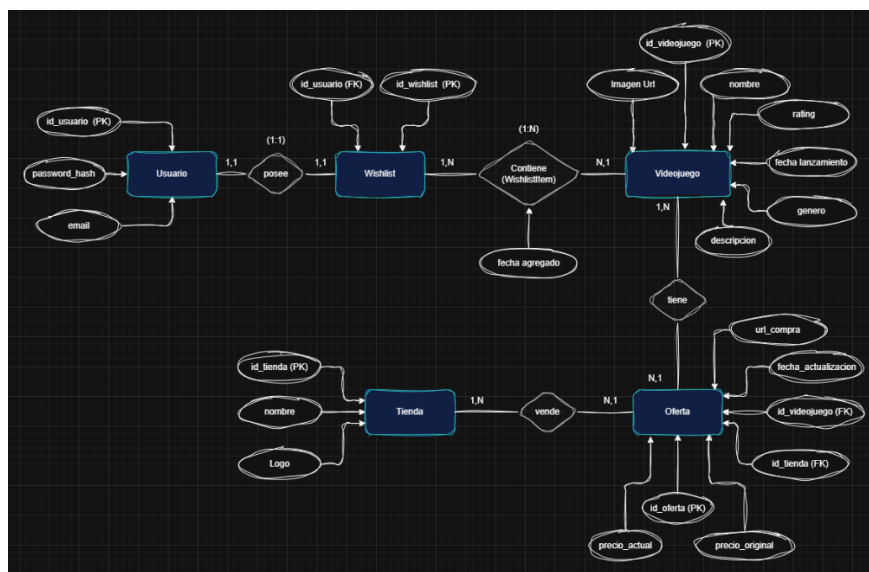


Figura 7. Diagrama E-R inicial

En las primeras versiones del sistema la estructura era considerablemente más simple. Sin embargo, conforme avanzó el desarrollo se detectaron distintos problemas que obligaron a replantear parte del modelo de datos.

Uno de los cambios más importantes fue la introducción de nuevas estructuras como **bundles**, que inicialmente no estaban contempladas en el diseño original. Esto provocó una reestructuración general del modelo para poder soportar correctamente distintos tipos de contenido procedentes de APIs externas.

Esto llevó a otro cambio importante, la separación entre videojuegos y bundles. Aunque ambos comparten parte de la información relacionada con ofertas y wishlist, se decidió mantener entidades independientes debido a las diferencias existentes entre ambos tipos de contenido y para simplificar el tratamiento de datos recibidos desde APIs externas.

Durante esta evolución también se añadieron nuevas tablas auxiliares para almacenar capturas, vídeos, géneros y relaciones intermedias necesarias para mejorar la organización de los datos.

En las primeras fases del proyecto cada integrante trabajaba con una base de datos local independiente. Posteriormente se decidió migrar a una base de datos centralizada desplegada en Neon para facilitar el trabajo colaborativo y asegurar que todos los integrantes trabajasen sobre los mismos datos y estructura.

6.2. Modelo entidad-relación final

The diagram illustrates a database schema for a video game store. It includes tables for genres, games, users, sellers, bundles, offers, and offer imaging. Relationships are defined between these tables using lines and cardinalities (1, N). The schema is organized into several groups, with tables belonging to different namespaces (public, private) indicated by color-coded icons.

El sistema se estructura alrededor de un conjunto de entidades principales que forman el núcleo del modelo de datos:

- Página 23 de 92

-  **oferta:** almacena la información de precios, descuentos y disponibilidad
-  **usuario:** gestiona la información básica de los usuarios del sistema
-  **wishlist:** representa la lista de favoritos asociada a cada usuario

Estas entidades se complementan con varias tablas auxiliares que permiten organizar la información de forma más estructurada y evitar redundancias:

-  géneros de videojuegos
-  productos asociados a bundles y su información complementaria
-  capturas de pantalla de videojuegos y productos bundle
-  videos de videojuegos y productos bundle
-  tablas intermedias para relaciones de tipo Many-to-Many

El modelo define distintas relaciones entre entidades en función de su naturaleza:

-  **One-to-Many:** relaciones jerárquicas como videojuego > capturas/videos o también videojuego > ofertas
-  **Many-to-One:** relaciones de pertenencia como ofertas > tienda
-  **Many-to-Many:** relaciones múltiples como wishlist y videojuegos o wishlist y bundles
-  **One-to-One:** en casos concretos donde existe una relación directa entre entidades como usuario y wishlist

El resultado final es un modelo de datos estructurado que permite representar de forma coherente la información del sistema, facilitando su consulta y mantenimiento dentro de la aplicación en estos momentos de desarrollo.

6.3. Tablas

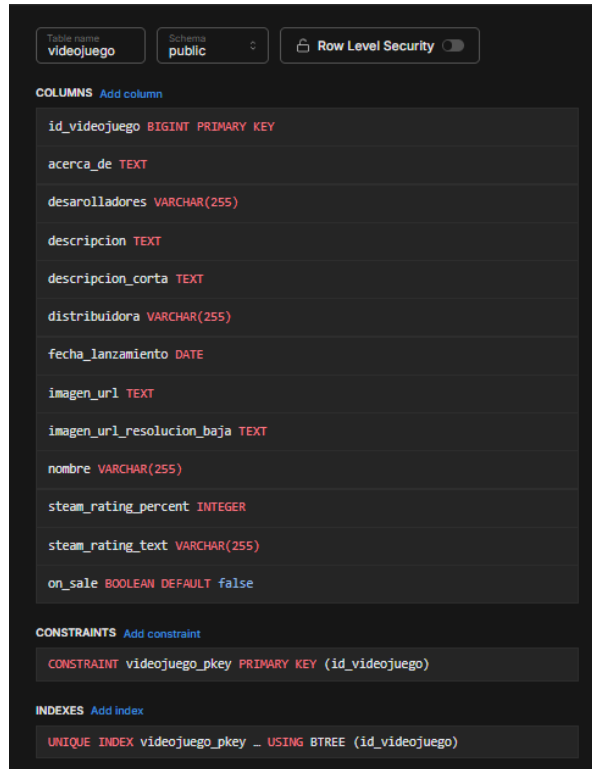
Las siguientes tablas conforman la estructura principal de la base de datos y almacenan la información utilizada por el sistema.

6.3.1. Tabla videojuego

La tabla **videojuego** constituye el núcleo principal del sistema, ya que almacena la información de los videojuegos obtenidos desde la API de Steam. Incluye datos como:

-  ID
-  nombre del videojuego
-  imágenes principales y de baja resolución
-  descripción corta, detallada y acerca de
-  fecha de lanzamiento
-  desarrolladores y distribuidoras
-  valoraciones de Steam
-  estado OnSale

Además, esta entidad actúa como punto central de relación con otras tablas como ofertas, géneros, capturas, videos y wishlist.



The screenshot shows a database schema editor interface. At the top, there are tabs for 'Table name: videojuego', 'Schema: public', and a 'Row Level Security' toggle. Below this, the 'COLUMNS' section lists the following fields:

- id_videojuego BIGINT PRIMARY KEY
- acerca_de TEXT
- desarrolladores VARCHAR(255)
- descripcion TEXT
- descripcion_corta TEXT
- distribuidora VARCHAR(255)
- fecha_lanzamiento DATE
- imagen_url TEXT
- imagen_url_resolucion_baja TEXT
- nombre VARCHAR(255)
- steam_rating_percent INTEGER
- steam_rating_text VARCHAR(255)
- on_sale BOOLEAN DEFAULT false

Below the columns, the 'CONSTRAINTS' section shows a primary key constraint:

- CONSTRAINT videojuego_pkey PRIMARY KEY (id_videojuego)

The 'INDEXES' section shows a unique index:

- UNIQUE INDEX videojuego_pkey _ USING BTREE (id_videojuego)

Figura 9. Tabla Videojuego

6.3.2. Tabla bundle

La tabla **bundle** representa paquetes de productos que agrupan varios videojuegos u otros elementos relacionados. Cada bundle almacena información básica como:

-  ID
-  nombre
-  imagen principal
-  estado OnSale

A diferencia de los videojuegos, los bundles funcionan como una entidad independiente debido a la naturaleza distinta de los datos que reciben desde fuentes externas y su estructura interna.

6.3.3. Tabla bundle_productos

La tabla **bundle_productos** almacena los productos individuales pertenecientes a un bundle.

Cada uno de estos contiene información básica del producto, como:

-  identificador del juego (steamAppld)
-  nombre
-  descripción
-  imagen

Esta tabla actúa como nivel intermedio entre el bundle y los elementos multimedia asociados a cada producto.

6.3.4. Tabla captura y tablas bundle_productos_capturas

La tabla **captura** y **bundle_productos_capturas** almacena las capturas de pantalla asociadas a los videojuegos o productos. Cada registro contiene:

-  imagen en tamaño completo
-  miniatura

Esta entidad permite gestionar de forma independiente el contenido visual de cada elemento.

6.3.5. Tabla movie y tabla bundle_productos_movies

La tabla **movie** almacena los vídeos asociados a los videojuegos o productos. Incluye información como:

-  título del vídeo
-  miniatura
-  enlace al vídeo

Se utiliza para representar trailers y contenido multimedia adicional relacionado con cada elemento.

6.3.6. Tabla genero

La tabla **genero** almacena los distintos géneros de videojuegos disponibles en el sistema. Cada género contiene:

-  ID
-  descripción

Esta entidad permite clasificar los videojuegos en diferentes categorías.

6.3.7. Tabla genero_videojuego

La tabla **genero_videojuego** es una tabla intermedia que gestiona la relación entre videojuegos y géneros. Permite que:

-  un videojuego pertenezca a múltiples géneros
-  un género contenga múltiples videojuegos

6.3.8. Tabla tienda

La tabla **tienda** almacena la información de las tiendas digitales donde se encuentran disponibles las ofertas. Incluye datos como:

-  ID
-  nombre de la tienda
-  logo
-  banner
-  icono

Esta entidad permite identificar la procedencia de cada oferta dentro del sistema.

6.3.9. Tabla usuario

La tabla **usuario** almacena la información básica de los usuarios registrados en el sistema. Los datos que almacena son:

-  ID (UUID)
-  recibir notificaciones
-  email del usuario

6.3.10. Tabla wishlist

La tabla **wishlist** representa la lista de favoritos asociada a cada usuario. Cada usuario dispone de una única wishlist, que permite almacenar:

-  videojuegos favoritos
-  bundles favoritos

6.3.11. Tabla oferta

La tabla **oferta** es una de las entidades más importantes del sistema, ya que almacena la información de precios y descuentos obtenidos desde APIs externas. Incluye datos como:

-  ID
-  precio original y precio rebajado
-  título de la oferta
-  URL de la oferta
-  Imagen de la oferta
-  porcentaje de ahorro
-  valoración de la oferta
-  tienda asociada
-  identificador del videojuego o bundle
-  fecha de inicio de la oferta

Esta tabla conecta directamente los videojuegos y bundles con las tiendas donde se encuentran disponibles.

Table name	oferta	Schema	public	Row Level Security	☐
COLUMNS Add column					
id_oferta	VARCHAR(255)	PRIMARY KEY			
ahorro	DOUBLE PRECISION				
cambiar_img	BOOLEAN	DEFAULT false			
inicio_oferta	TIMESTAMP				
oferta_rating	DOUBLE PRECISION				
precio_oferta	DOUBLE PRECISION	NOT NULL			
precio_original	DOUBLE PRECISION	NOT NULL			
steam_appid	BIGINT	NOT NULL			
steam_rating	INTEGER				
thumb	TEXT				
titulo	VARCHAR(255)				
url_compra	TEXT				
tienda	BIGINT				
videojuego	BIGINT				
bundle	BIGINT				
url_imagen	TEXT				
CONSTRAINTS Add constraint					
CONSTRAINT fk_oferta_bundle FOREIGN KEY (bundle) REFERENCES public.bundle (id_bundle) ON DELETE SET NULL					
CONSTRAINT fk_oferta_tienda FOREIGN KEY (tienda) REFERENCES public.tienda (id_tienda) ON DELETE SET NULL					
CONSTRAINT fk_oferta_videojuego FOREIGN KEY (videojuego) REFERENCES public.videojuego (id_videojuego) ON DELETE CASCADE					
CONSTRAINT oferta_pkey PRIMARY KEY (id_oferta)					
INDEXES Add index					
UNIQUE INDEX oferta_pkey _ USING BTREE (id_oferta)					

Figura 10. Tabla oferta

6.3.12. Tabla oferta_staging

La tabla **oferta_staging** actúa como una tabla temporal de oferta (los mismos campos) utilizada durante los procesos de sincronización de ofertas. Su objetivo es:

-  almacenar temporalmente los datos obtenidos desde APIs externas
-  evitar errores durante actualizaciones masivas
-  permitir validaciones antes de insertar los datos definitivos

Una vez finalizada la sincronización, los datos válidos se trasladan a la tabla principal de ofertas.

6.3.13. Tablas wishlist_videojuegos y wishlist_bundles

Estas tablas intermedias permiten gestionar las relaciones entre la wishlist y los elementos del sistema:

-  **wishlist_videojuegos**: relaciona wishlist con videojuegos
-  **wishlist_bundles**: relaciona wishlist con bundles

6.3.14. Vista materializada mv_ofertas_unicas

La vista materializada **mv_ofertas_unicas** almacena una representación agregada de las ofertas disponibles en el sistema, agrupadas por videojuego. Su estructura permite mantener:

-  la información principal de la oferta
-  la oferta asociada con mejor precio (más barata)
-  datos adicionales de la oferta como valoración, ahorro y tienda

Esta vista se genera a partir de la tabla de ofertas y se actualiza periódicamente tras los procesos de sincronización de datos.

Su objetivo es mantener una versión consolidada de las ofertas sin necesidad de recalcular agregaciones complejas en tiempo de consulta.

```

View name: mv_ofertas_unicas  Schema: public

WITH
  cheapest AS (
    SELECT DISTINCT
      ON (o.steam_appid) o.steam_appid,
      o.titulo,
      o.precio_oferta,
      o.ahorro,
      o.oferta_rating,
      o.inicio_oferta AS recent,
      o.steam_rating AS reviews,
      o.thumb AS imagen
    FROM
      oferta o
    ORDER BY
      o.steam_appid,
      o.precio_oferta,
      o.oferta_rating DESC NULLS LAST
  ),
  tiendas AS (
    SELECT
      oferta.steam_appid,
      (
        ', '::text || string_agg(DISTINCT oferta.tienda::text, ', '::text)
      ) || ', '::text AS tienda_ids
    FROM
      oferta
    WHERE
      oferta.tienda IS NOT NULL
    GROUP BY
      oferta.steam_appid
  )
SELECT
  c.steam_appid,
  c.titulo,
  c.precio_oferta,
  c.ahorro,
  c.oferta_rating,
  c.recent,
  c.reviews,
  c.imagen,
  t.tienda_ids
FROM
  cheapest c
LEFT JOIN tiendas t USING (steam_appid)

```

steam_appid bigint	titulo varchar(255)	precio_oferta double precision	ahorro double precision	oferta_rating double precision	recent timestamp	reviews integer	imagen text	tienda_ids text
1911360	Trinity Fusion	3.99	80.04002	7.7	2026-05-18 10:32:03	86	https://shared.akamai...	,30,
1928690	Bionic Bay	4.55	77.230619	9.3	2026-05-22 17:28:04	85	https://shared.akamai...	,15,23,27,
1933180	Varney Lake	0.99	90.09009	8.3	2026-05-18 11:18:08	100	https://shared.akamai...	,30,7,
1933190	Bahnsen Knights	1.49	85.085085	8.1	2026-05-18 11:18:02	97	https://shared.akamai...	,30,7,
1933370	Mark of the Deep	11.06	63.12104	7.5	2026-05-25 23:22:49	74	https://shared.akamai...	,23,
1937780	Total War: PHAROAH	7.16	82.095524	8	2026-05-22 19:18:01	64	https://shared.akamai...	,21,23,27,3,

Figura 11. Vista Materializada y resultados

6.4. Relaciones de las tablas

Este apartado describe las relaciones existentes entre las principales entidades del sistema y cómo se conectan entre sí dentro del modelo de datos.

Entidad origen	Entidad destino	Tipo de relación	Descripción
Usuario	Wishlist	One-to-One	Cada usuario tiene una única lista de favoritos
Wishlist	Videojuego	Many-to-Many	Un videojuego puede estar en múltiples wishlists
Wishlist	Bundle	Many-to-Many	Los bundles también pueden guardarse como favoritos
Videojuego	Oferta	One-to-Many	Un videojuego puede tener varias ofertas asociadas
Bundle	Oferta	One-to-Many	Un bundle puede agrupar varias ofertas
Tienda	Oferta	One-to-Many	Cada tienda puede contener múltiples ofertas
Videojuego	Género	Many-to-Many	Un videojuego puede pertenecer a varios géneros
Videojuego	Captura	One-to-Many	Un videojuego contiene varias capturas
Videojuego	Movie	One-to-Many	Un videojuego contiene varios vídeos
Bundle	BundleProductos	One-to-Many	Un bundle agrupa múltiples productos
BundleProductos	bundle_productos_capturas	One-to-Many	Un producto puede tener múltiples capturas
BundleProductos	bundle_productos_movies	One-to-Many	Un producto puede tener múltiples vídeos

6.5. Triggers y automatización

La base de datos incorpora triggers con el objetivo de automatizar la actualización del estado de disponibilidad (**onSale**) tanto en videojuegos como en bundles. Este campo indica si un elemento del sistema dispone actualmente de ofertas activas asociadas.

Estos triggers se ejecutan de forma automática cada vez que se produce una inserción o eliminación en la tabla **oferta**, evitando así tener que gestionar esta lógica desde la aplicación y garantizando que los datos se mantengan siempre coherentes.

```
✓ CREATE OR REPLACE FUNCTION actualizar_on_sale()
  RETURNS trigger AS $$
  BEGIN
    UPDATE public.videojuego v
    SET on_sale = EXISTS (
      SELECT 1 FROM public.oferta o
      WHERE o.videojuego = v.id_videojuego
    )
    WHERE v.id_videojuego = NEW.videojuego;

    RETURN NEW;
  END;
  $$ LANGUAGE plpgsql;
```

```
✓ CREATE OR REPLACE FUNCTION actualizar_on_sale_bundle()
  RETURNS trigger AS $$
  BEGIN
    UPDATE public.bundle b
    SET on_sale = EXISTS (
      SELECT 1
      FROM public.oferta o
      WHERE o.bundle = b.id_bundle
    )
    WHERE b.id_bundle = NEW.bundle;

    RETURN NEW;
  END;
  $$ LANGUAGE plpgsql;
```

Figura 12. Funciones de actualización de estado OnSale

Estas funciones recalculan el estado **onSale** de un videojuego y/o bundle concreto comprobando si existe al menos una oferta asociada a su identificador. En caso afirmativo, el campo se marca como true en caso contrario como false.

```
✓ CREATE TRIGGER oferta_cambio
  AFTER INSERT OR DELETE ON public.oferta
  FOR EACH ROW
  EXECUTE FUNCTION actualizar_on_sale();
```

```
✓ CREATE TRIGGER oferta_cambio_bundle
  AFTER INSERT OR DELETE ON public.oferta
  FOR EACH ROW
  EXECUTE FUNCTION actualizar_on_sale_bundle();
```

Figura 13. Triggers de la BBDD

Los triggers se ejecutan automáticamente tras cada operación de inserción o eliminación sobre la tabla **oferta**. En el momento en que se detecta un cambio, se invocan las funciones anteriores para recalculan el estado del videojuego o bundle afectado.

De esta forma, cualquier modificación en las ofertas se refleja inmediatamente en los datos principales del sistema, garantizando que el campo **onSale** se mantenga siempre sincronizado con la realidad de la base de datos.

7.Backend

7.1. Configuración Inicial

El backend ha sido desarrollado utilizando **Spring Boot 4.0.2** y **Java 21**, lo que permite aprovechar las mejoras del lenguaje en cuanto a rendimiento, concurrencia y expresividad del código y utilizando **Gradle** como gestor de dependencias.

La elección de Spring Boot se fundamenta en su capacidad para simplificar el desarrollo de aplicaciones web, proporcionando una amplia integración con tecnologías como JPA, sistemas de *caché*, clientes HTTP y servicios de mensajería, lo que lo convierte en la mejor opción además de ser lo que hemos usado a lo largo del curso.

7.1.1. Dependencias principales

La gestión de dependencias se realiza mediante Gradle, integrando los siguientes módulos principales:

Spring Boot Starters

-  spring-boot-starter-web: creación de API REST con servidor embebido Tomcat
-  spring-boot-starter-data-jpa: acceso a datos mediante JPA/Hibernate
-  spring-boot-starter-cache: sistema de *caché* en memoria
-  spring-boot-starter-mail: envío de correos electrónicos
-  spring-boot-starter-restclient: cliente moderno para consumo de APIs externas

Dependencias externas

-  PostgreSQL Driver (v42.7.10): conexión con base de datos relacional
-  Java JWT (v4.4.0): gestión de tokens de autenticación
-  Caffeine (v3.2.0): sistema de *caché* de alto rendimiento en memoria
-  Apache Commons Text (v1.11.0): utilidades para manipulación de texto

7.1.2. Configuración del sistema

La configuración realizada para desarrollar la aplicación fue definida en el **application.properties**, donde se definen los parámetros principales del sistema.

```
2 spring.datasource.url=jdbc:postgresql://ep-muddy-thunder-abb7j08e.eu-west-2.aws.neon.tech/GameStore?sslmode=require&channel_binding=require
3 spring.datasource.username=neondb_owner
4 spring.datasource.password=npg_6cJ5FBP1HXnD
5 spring.datasource.driver-class-name=org.postgresql.Driver
6 spring.jpa.show-sql=false
7 spring.jpa.hibernate.ddl-auto=none
8 spring.sql.init.mode=never
```

Figura 14. Conexión a Base de Datos

La aplicación se conecta a la base de datos PostgreSQL alojada en Neon, utilizando el driver oficial de PostgreSQL. La URL de conexión incluye parámetros de seguridad como **sslmode=require**, garantizando una comunicación cifrada entre el backend y la base de datos.

Al principio se utilizó una configuración más flexible de **spring.jpa.hibernate.ddl-auto**, inicialmente en modo más permisivo, lo que permitía la creación y modificación directa del esquema de base de datos desde el backend. Más adelante, se pasó a **none** para evitar cualquier modificación automática del esquema y asegurar un mayor control sobre la estructura de datos.

Y durante el desarrollo, todas las credenciales se mantienen directamente en el archivo de configuración para facilitar el trabajo y las pruebas en local. Sin embargo, en el entorno de despliegue (**la rama Despliegue**) se sustituyen por **variables de entorno**, con el objetivo de mejorar la seguridad y evitar la exposición de información sensible. Este aspecto se detalla más adelante en el documento.

```
spring.jpa.properties.hibernate.jdbc.batch_size=50
spring.jpa.properties.hibernate.order_inserts=true
```

Figura 15. Optimización de hibernate

Estas propiedades optimizan el rendimiento de las operaciones de escritura en base de datos, especialmente en escenarios donde se realizan múltiples inserciones, lo cual es nuestro caso.

La configuración de **batch_size=50** permite agrupar las sentencias INSERT en lotes de hasta 50 registros, reduciendo el número de viajes entre la aplicación y la base de datos. Por otro lado, **order_inserts=true** reorganiza las inserciones para agrupar entidades similares, lo que mejora la eficiencia interna de Hibernate al momento de persistir los datos.

```
#Configuración para Resend (SMTP)
spring.mail.host=smtp.resend.com
spring.mail.port=465
spring.mail.username=resend

#¡IMPORTANTE! Cambia esto para que Spring lo reconozca:
spring.mail.password=re_bbZhDneQ_B6rVMRbmsFpd39VVQ7pZr4aD

#Configuración de seguridad para puerto 465 (SSL)
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.ssl.enable=true
spring.mail.properties.mail.smtp.socketFactory.port=465
spring.mail.properties.mail.smtp.socketFactory.class=javax.net.ssl.SSLSocketFactory

#Esto es opcional pero ayuda a debuggear errores de conexión
spring.mail.properties.mail.debug=true
|
app.mail.from=onboarding@resend.dev
```

Figura 16. Configuración de correo electrónico (SMTP)

La aplicación utiliza **Resend** como proveedor de correo electrónico mediante SMTP. Esta configuración permite el envío de correos desde el backend utilizando un servidor externo seguro.

- ✏ *host* y *port* definen el servidor SMTP y el puerto utilizado para la conexión.
- ✏ *username* y *password* se emplean para autenticar la aplicación frente al servicio de correo.
- ✏ **mail.smtp.ssl.enable=true** activa la comunicación cifrada mediante SSL, garantizando la seguridad en el envío de correos.
- ✏ **mail.debug=true** habilita logs detallados de la conexión SMTP, lo que facilita la detección de errores durante el desarrollo.
- ✏ *app.mail.from* define la dirección de correo remitente que aparecerá en los mensajes enviados por la aplicación.

```
#SUPABASE
supabase.url=https://qtjymactaeykkxpwtihn.supabase.co
supabase.service_role_key=La calve aqui
```

Figura 17. Integración con Supabase

La autenticación y gestión de usuarios se apoya en **Supabase**, que actúa como proveedor externo de identidad, como llevamos diciendo. Desde el backend se utiliza una **service role key**, que permite realizar operaciones administrativas seguras como la sincronización, borrado y gestión de usuarios.

Las credenciales de acceso a Supabase no se ha incluido directamente en el código ya que se gestiona mediante **variables de entorno** en el despliegue y haciendo pruebas simplemente la colocamos, se prueba y borra, para evitar la exposición de información sensible y aumentar la seguridad.

7.2. Arquitectura

Esta se ha desarrollado siguiendo una arquitectura en capas, con el objetivo de separar claramente las responsabilidades y facilitar su mantenimiento, escalabilidad y testeo.

Este enfoque organiza el proyecto de forma jerárquica, donde cada capa tiene un rol específico y depende de la inferior según sea necesario. De este modo, los controladores delegan la lógica en los servicios, y estos a su vez interactúan con los repositorios para acceder a los datos, evitando que la lógica de negocio se mezcle con la capa de acceso a datos o con la de presentación.

7.2.1. Estructura del proyecto

La organización del backend sigue una estructura modular basada en capas, donde cada paquete agrupa componentes con una responsabilidad específica. Esto facilita el mantenimiento del sistema, mejora la legibilidad del código y permite ampliar funcionalidades de forma más ordenada.

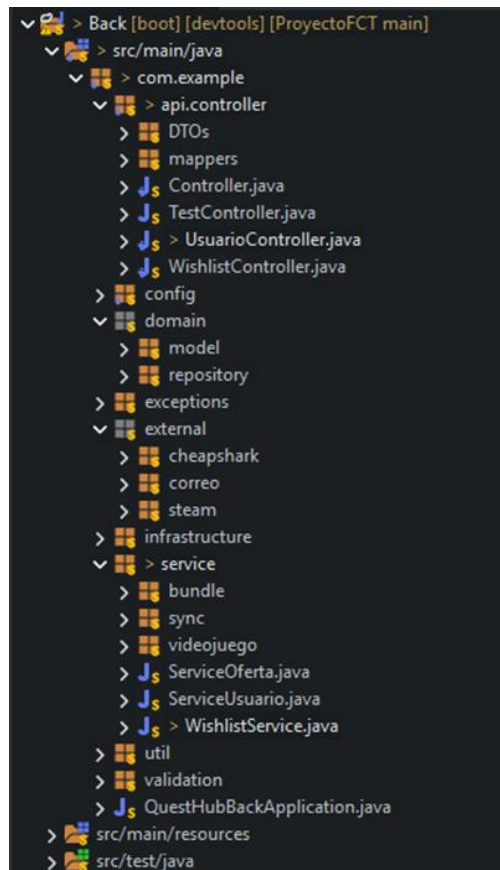


Figura 17. Estructura backend

-  **/api** corresponde a la capa de presentación del sistema. En ella se encuentran los controladores REST encargados de recibir las peticiones HTTP, validar parámetros básicos y devolver respuestas al cliente. También incluye **DTOs** (Data Transfer Objects), utilizados para transferir únicamente la información necesaria entre backend y frontend. Los **mappers** centralizan la transformación entre entidades y DTOs.
-  **/service** contiene toda la lógica de negocio de la aplicación. Los servicios actúan como intermediarios entre los controladores y los repositorios, coordinando operaciones, aplicando validaciones y gestionando procesos complejos como sincronizaciones, búsquedas o tareas asíncronas.
-  **/domain** se agrupan las entidades JPA (**/model**) y los repositorios (**/repository**). Las entidades representan las tablas de la base de datos, mientras que los repositorios permiten acceder a los datos.
-  **/external** engloba los componentes encargados de la comunicación con servicios externos. En este proyecto se incluyen clientes REST para las APIs de CheapShark y Steam, los cuales por dentro también poseen **DTOs** y **Wrappers** y el servicio encargado del envío de correos electrónicos mediante SMTP.
-  **/infrastructure** contiene componentes técnicos que soportan el funcionamiento interno del sistema sin formar parte de la lógica de negocio. Incluye la gestión de eventos, la actualización de vistas materializadas y la limpieza de *caché*.

-  **/config** almacena las configuraciones globales de la aplicación, como configuraciones de *caché*, CORS, clientes HTTP, manejo de excepciones, tareas programadas y/o tareas asíncronas. Esto permite centralizar ajustes comunes y mantener separada la configuración de la lógica de negocio.
-  **/exceptions** contiene las excepciones personalizadas.
-  **/validation** agrupa validaciones de datos utilizadas en distintas partes del sistema, de momento solo están las validaciones de los paginables.
-  **/util** incluye clases auxiliares y métodos de apoyo.
-  **/resources** contiene los recursos necesarios para la configuración y ejecución de la aplicación. Entre ellos destaca el archivo *application.properties*, donde se definen las propiedades de configuración del sistema.
-  **/test** está destinada a las pruebas del sistema. En ella se desarrollan pruebas unitarias e integradas para validar el funcionamiento de la aplicación.

7.3. Capa /config (Configuraciones)

Esta capa se encarga de las distintas configuraciones encargadas de gestionar aspectos concretos de la aplicación, como la ejecución de tareas asíncronas, el sistema de *caché*, la comunicación con APIs externas y más

Estas configuraciones se agrupan dentro del paquete/capa **/config**, y utilizan todas, la anotación de **@Configuration** para que Spring sepa que son como su nombre dice configuraciones.

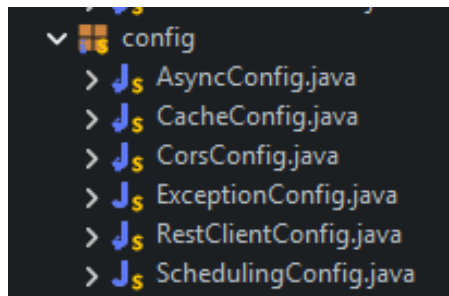


Figura 18. Capa config

7.3.1. Configuración de Tareas Asíncronas (AsyncConfig)

Para evitar bloqueos durante operaciones pesadas o procesos que pueden tardar varios segundos, el sistema utiliza procesamiento asíncrono mediante *ThreadPoolTaskExecutor*. Esto permite ejecutar determinadas tareas en segundo plano sin afectar el tiempo de respuesta de las peticiones HTTP realizadas por el usuario.

La configuración se centraliza en la clase **AsyncConfig**, donde se definen distintos pools de hilos especializados según el tipo de operación que se ejecuta. Se utilizan principalmente dos ejecutores:

-  **cheapSharkExecutor**: encargado de tareas relacionadas con sincronizaciones y peticiones hacia la API de CheapShark.
-  **gameExecutor**: utilizado para la creación y almacenamiento asíncrono de videojuegos y bundles obtenidos desde Steam.

La separación de ejecutores permite distribuir mejor la carga del sistema y evitar que procesos pesados, como sincronizaciones masivas, interfieran con operaciones más frecuentes relacionadas con videojuegos individuales.

Cada *ThreadPoolTaskExecutor* define distintos parámetros para controlar el comportamiento de los hilos:

-  **Core Pool Size**: número mínimo de hilos activos que el sistema mantiene disponibles.
-  **Max Pool Size**: límite máximo de hilos simultáneos permitidos.
-  **Queue Capacity**: cantidad de tareas que pueden permanecer en cola antes de crear nuevos hilos.
-  **Thread Name Prefix**: prefijo personalizado para identificar fácilmente los hilos durante tareas de debugging y monitorización.

Gracias a esta configuración, el backend puede ejecutar procesos concurrentes de forma controlada, mejorando el rendimiento general de la aplicación y evitando bloqueos innecesarios.

```
1 package com.example.config;
2
3 import java.util.concurrent.Executor;
4
5 @Configuration
6 public class AsyncConfig {
7
8     @Bean(name = "cheapSharkExecutor")
9     public Executor cheapSharkExecutor() {
10         ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
11         executor.setCorePoolSize(2); // hilos minimos
12         executor.setMaxPoolSize(3); // hilos maximos
13         executor.setQueueCapacity(10); // tareas en cola
14         executor.setThreadNamePrefix("CheapShark-");
15         executor.initialize();
16         return executor;
17     }
18
19     @Bean(name = "gameExecutor")
20     public Executor gameExecutor() {
21         ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
22         executor.setCorePoolSize(10);
23         executor.setMaxPoolSize(20);
24         executor.setQueueCapacity(500);
25         executor.setThreadNamePrefix("GameCreator-");
26         executor.initialize();
27         return executor;
28     }
29 }
30
31 }
```

Figura 19. Código configuración Asíncrona

Además de definir los ejecutores personalizados, fue necesario habilitar el soporte de tareas asíncronas en la aplicación mediante la anotación **@EnableAsync** colocándola en la app principal.

Esto permite que los métodos anotados con **@Async** se ejecuten en hilos independientes gestionados por Spring.

7.3.2. Configuración de Cache (CacheConfig)

Con el objetivo de mejorar el rendimiento general de la aplicación y reducir accesos innecesarios a la base de datos y APIs externas, utilizamos **Caffeine** como solución de *caché* en memoria, ya que nos permitía poner un TTL a diferencia del cache base de Spring.

La configuración se centraliza en la clase **CacheConfig**, utilizando *CacheManager* para definir múltiples *cachés* especializadas según el tipo de información almacenada. Cada una de ellas está pensada para responder a necesidades concretas de acceso y frecuencia de uso dentro de la aplicación. Entre las principales *cachés* implementadas destacan:

-  **videojuegos**: almacena información detallada de videojuegos obtenida desde Steam.
-  **bundles**: utilizada para cachear información relacionada con bundles.
-  **search-ofertas**: guarda resultados de búsquedas frecuentes realizadas por los usuarios.
-  **tiendas**: almacena información de tiendas, la cual cambia con poca frecuencia.
-  **wishlist_ids**: utilizada para consultas rápidas de favoritos asociados a usuarios.

Cada *caché* define configuraciones independientes adaptadas a sus necesidades específicas:

-  **Maximum Size**: limita la cantidad máxima de elementos almacenados en memoria.
-  **Expire After Write**: establece el tiempo de vida de los datos antes de invalidarse automáticamente.
-  **Estrategia de almacenamiento**: ajustada según el volumen y frecuencia de acceso de cada recurso.

Gracias a esto, la app puede optimizar el uso de memoria y tener tiempos de respuesta más rápidos en operaciones frecuentes, evitando consultas.

Y se habilita usando la anotación de **@EnableCaching**, la cual a diferencia de la de **@EnableAsync** se puede colocar en la misma configuración.

```
package com.example.config;

import com.github.benmanes.caffeine.cache.Caffeine;

@Configuration
@EnableCaching
public class CacheConfig {

    @Bean
    public CacheManager cacheManager() {
        SimpleCacheManager manager = new SimpleCacheManager();

        List<CaffeineCache> caches = List.of(
            new CaffeineCache("wishlist_ids",
                Caffeine.newBuilder().maximumSize(10_000).expireAfterWrite(Duration.ofMinutes(15)).build()),
            new CaffeineCache("tiendas",
                Caffeine.newBuilder().maximumSize(100).expireAfterWrite(Duration.ofDays(1)).build()),
            new CaffeineCache("max-precio",
                Caffeine.newBuilder().maximumSize(10).expireAfterWrite(Duration.ofHours(8)).build()),
            new CaffeineCache("videojuegos",
                Caffeine.newBuilder().maximumSize(8_000).expireAfterWrite(Duration.ofHours(8)).build()),
            new CaffeineCache("videojuego-entity",
                Caffeine.newBuilder().maximumSize(2_000).expireAfterWrite(Duration.ofHours(8)).build()),
            new CaffeineCache("bundles",
                Caffeine.newBuilder().maximumSize(5_000).expireAfterWrite(Duration.ofHours(8)).build()),
            new CaffeineCache("bundle-entity",
                Caffeine.newBuilder().maximumSize(1_000).expireAfterWrite(Duration.ofHours(8)).build()),
            new CaffeineCache("search-ofertas",
                Caffeine.newBuilder().maximumSize(5_000).expireAfterWrite(Duration.ofHours(8)).build());
        manager.setCaches(caches);

        return manager;
    }
}
```

Figura 20. Código configuración cache

7.3.3. Configuración de Clientes REST (RestClientConfig)

La comunicación con APIs externas se centraliza mediante la configuración de distintos clientes REST utilizando RestClient, el cliente HTTP moderno incorporado en Spring.

La configuración se encuentra en **RestClientConfig**, donde se define un *RestClient.Builder* re-utilizable para la creación de estos RestClients y varios clientes especializados según el servicio externo consumido. Los clientes configurados son:

-  **CheapShark API**: utilizada para obtener ofertas, precios y tiendas.
-  **Steam API**: utilizada para recuperar información detallada de videojuegos y bundles.
-  **RAWG API**: preparada para futuras funcionalidades.

Cada cliente define su propia *baseUrl*, permitiendo simplificar posteriormente las llamadas HTTP realizadas desde la capa de servicios.

En el caso de **CheapShark**, además, se configura un encabezado **User-Agent** para evitar bloqueos en esta API.

7.3.4. Configuración de manejo de Excepciones (ExceptionConfig)

Se desarrollo un manejo global de excepciones mediante la clase **ExceptionConfig**, utilizando la anotación **@RestControllerAdvice**. Esta anotacion permite interceptar errores producidos en cualquier parte de la aplicación y devolver respuestas HTTP consistentes al cliente.

Para cada tipo de error se utilizan métodos anotados con **@ExceptionHandler**, asociando además un código HTTP mediante **@ResponseStatus**, el cual mandara como respuesta un estado http. Entre las excepciones controladas tenemos:

-  `BadRequestException`
-  `IllegalArgumentException`
-  `RestClientException`
-  `HttpClientErrorException.TooManyRequests`
-  `Exception`

Además, los errores registran/muestran el *stack trace* mediante `printStackTrace()`, facilitando las tareas de debugging durante el desarrollo.

7.3.5. Configuración de Tareas Programadas (SchedulingConfig)

Para poder tener las tareas programadas y así automatizar distintos procesos periódicos, especialmente aquellos relacionados con la sincronización de ofertas y tiendas, ya que estas cambian considerablemente al paso del tiempo se implementó el siguiente mecanismo.

Este se encuentra en la clase **SchedulingConfig**, donde se habilita el soporte de planificación de tareas mediante la anotación **@EnableScheduling**. Además de esto, se configura un *ThreadPoolTaskScheduler* encargado de gestionar la ejecución controlada de estas tareas en segundo plano.

Las sincronizaciones automáticas utilizan posteriormente la anotación **@Scheduled**, permitiendo definir intervalos de ejecución mediante expresiones CRON, eligiendo cada cuanto tiempo, o cada cuanto después de la última ejecución.

7.3.6. Configuración de CORS (CorsConfig)

Para permitir la comunicación entre el frontend y el backend, se hace la configuración del CORS (Cross-Origin Resource Sharing), se encuentra en la clase **CorsConfig**, donde se define qué dominios pueden realizar peticiones a la API, así como los métodos HTTP y cabeceras permitidas.

```
package com.example.config;

import org.springframework.context.annotation.Bean;

@Configuration
public class CorsConfig {
    //probar config a ver si funciona
    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                registry.addMapping("/**")
                    .allowedOrigins("http://localhost:5173/",
                                   "https://quest-hub-ruddy.vercel.app/")
                    .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE")
                    .allowedHeaders("*");
            }
        };
    }
}
```

Figura 21. Código configuración CORS

-  **allowedOrigins:** define los dominios autorizados (local y producción).
-  **allowedMethods:** especifica los métodos HTTP permitidos.
-  **allowedHeaders:** permite el uso de cabeceras como Authorization.

Esta configuración evita bloqueos por políticas del navegador y permite la comunicación entre frontend y backend de forma controlada.

7.4. Capa /domain (Entity's y Repos)

La capa de dominio es la que se encarga de representar la estructura de datos principal del proyecto y gestionar su persistencia en la base de datos. En esta capa se definen las entidades JPA y los repositorios, que constituyen la base del modelo de información.

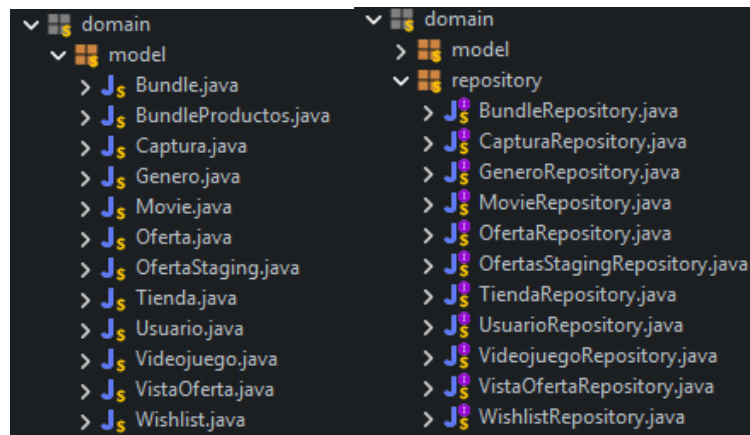


Figura 22. Capa domain

Por un lado, las **entidades (models)** representan las tablas de la base de datos mediante anotaciones de JPA. En ellas se definen los atributos, relaciones entre entidades (como uno a muchos o muchos a muchos) y las restricciones necesarias para mantener la integridad de los datos. Estas clases constituyen el modelo de datos central de la aplicación.

```

11 @Entity
12 @Table(name = "wishlist")
13 public class Wishlist {
14
15     @Id
16     @GeneratedValue(strategy = GenerationType.IDENTITY)
17     @Column(name = "id_wishlist")
18     private Long id;
19
20     @OneToOne
21     @JoinColumn(
22         name = "user_id",
23         referencedColumnName = "id_usuario",
24         unique = true,
25         nullable = false
26     )
27     @JsonIgnore
28     private Usuario usuario;
29
30     @ManyToMany
31     @JoinTable(
32         name = "wishlist_videojuegos",
33         joinColumns = @JoinColumn(name = "id_wishlist"),
34         inverseJoinColumns = @JoinColumn(name = "id_videojuego")
35     )
36     private Set<Videojuego> videojuegos = new HashSet<>();
37
38     @ManyToMany
39     @JoinTable(
40         name = "wishlist_bundles",
41         joinColumns = @JoinColumn(name = "id_wishlist"),
42         inverseJoinColumns = @JoinColumn(name = "id_bundle")
43     )
44     private Set<Bundle> bundles = new HashSet<>();
45
46     @Column(name = "fecha_agregado")
47     private LocalDateTime fechaAgregado = LocalDateTime.now();
48

```

Figura 23. Entity WishList

Por otro lado, los **repositorios** se encargan del acceso a la base de datos. A través de Spring Data JPA, permiten realizar operaciones CRUD y consultas personalizadas de forma automática, sin necesidad de implementar SQL manualmente. Esto simplifica el acceso a datos y mejora la mantenibilidad del código.

Esta capa se caracteriza por su enfoque exclusivamente en la persistencia, sin incluir lógica de negocio, la cual se delega a la capa de servicios.

7.5. Capa /api (Controladores)

La capa de api (**controllers**) actúa como punto de entrada de la nuestra API REST. Su función principal es recibir las peticiones HTTP del frontend, validar parámetros básicos y delegar toda la lógica de la aplicación en los servicios correspondientes. Además, esta capa trabaja junto a los DTOs y mappers de esta capa para adaptar correctamente la información enviada y recibida por la aplicación.

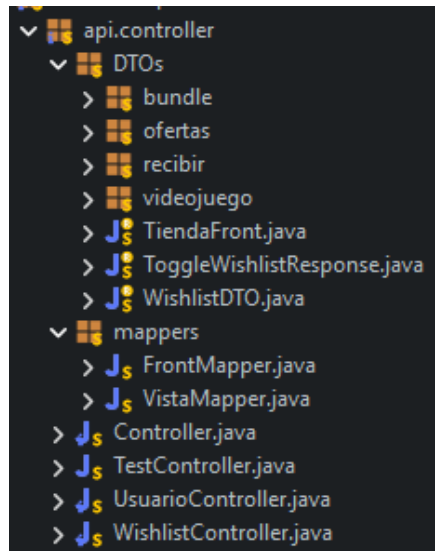


Figura 24. Capa util

Todos los endpoints de la aplicación siguen una convención común de versión, iniciando con el prefijo **/api/v1.0**. Esto permite mantener versionado el sistema y facilitar futuras evoluciones sin romper compatibilidad con clientes existentes.

Controller se encarga de los endpoints públicos relacionados con ofertas, videojuegos, bundles y tiendas. Desde aquí se gestionan búsquedas de ofertas, paginación, filtros dinámicos y recuperación de información detallada de juegos o bundles. También se realizan validaciones de paginación mediante **PageableValidator**, perteneciente a la capa de validación (**validation**) antes de delegar el procesamiento al servicio de oferta, los servicios de videojuegos y/o bundles. Endpoints principales:

-  **GET /oferta/{id}**: recibe un id (long) y devuelve un VideojuegoFront o BundleFront según exista el recurso.
-  **GET /ofertas/mayorPrecio**: no recibe parámetros y devuelve un Double con el precio máximo de ofertas.
-  **GET /tiendas**: no recibe parámetros y devuelve una lista de TiendaFront.
-  **GET /ofertas**: recibe Pageable y FiltrosOfertas, y devuelve una página de ViewOfertaFront filtrada y paginada.
-  **GET /ofertas/search**: recibe título (String) y devuelve un BuscadorResponseDTO con resultados limitados de búsqueda.

```

1 package com.example.api.controller;
2
3 import com.example.api.controller.DTOs.TiendaFront;
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22 @RestController
23 @RequestMapping("/api/v1.0")
24 public class Controller {
25
26     private final ServiceBundle serviceBundle;
27     private final ServicioVideojuego servicioVideojuego;
28     private final ServiceOferta serviceOferta;
29
30     public Controller(ServicioVideojuego servicioVideojuego,
31                     ServiceOferta serviceOferta, ServiceBundle serviceBundle) {
32         this.serviceBundle = serviceBundle;
33         this.servicioVideojuego = servicioVideojuego;
34         this.serviceOferta = serviceOferta;
35     }
36
37     @GetMapping("/oferta/{id}")
38     public ResponseEntity<?> getJuego(@PathVariable(name = "id") long id) {
39         VideojuegoFront dato = servicioVideojuego.buscarPorId(id);
40         if (dato != null) {
41             Map<String, VideojuegoFront> respuesta = Map.of("Juego", dato);
42             return ResponseEntity.ok(respuesta);
43         }
44
45         BundleFront data = serviceBundle.buscarPorId(id);
46         if (data != null) {
47             Map<String, BundleFront> respuesta = Map.of("Bundle", data);
48             return ResponseEntity.ok(respuesta);
49         }
50
51         return ResponseEntity.notFound().build();
52     }
53
54     @GetMapping("/ofertas/mayorPrecio")
55     public ResponseEntity<?> getMaxPrecio() {
56         Double max = serviceOferta.obtenerMaxPrecio();
57         if (max == null) { throw new BadRequestException("No hay precios disponibles"); }
58         return ResponseEntity.ok(max);
59     }
60

```

Figura 25. Parte del código de Controller

UsuarioController gestiona toda la parte de usuarios y su integración con Supabase. Su responsabilidad no es solo CRUD básico, sino mantener sincronizada la identidad del usuario entre el proveedor externo de autenticación y la base de datos interna. Además, controla preferencias de usuario como notificaciones y permite la eliminación completa de la cuenta, incluyendo la parte externa de autenticación. Endpoints de usuario:

-  **POST /usuarios/sincronizar:** recibe UsuarioSync (id, email) y crea el usuario si no existe en base de datos.
-  **DELETE /usuarios/eliminar:** recibe un JWT en el header Authorization, extrae el UUID del usuario y elimina tanto la cuenta en Supabase como en la base de datos local.
-  **GET /usuarios/preferencias:** recibe id (UUID) y devuelve un Boolean indicando si el usuario tiene activadas las notificaciones.
-  **PATCH /usuarios/preferencias:** recibe UsuarioNotificaciones (id, preferencia) y actualiza la configuración del usuario.

WishlistController se encarga de la gestión de favoritos del usuario, actuando como una capa de interacción directa entre el usuario autenticado y su lista personalizada de videojuegos y bundles. Incluye operaciones de alta/baja dinámica, consulta paginada y recuperación optimizada de IDs para uso en frontend (por ejemplo, para marcar elementos ya guardados en la UI). Extrae el usuario directamente desde el JWT para evitar dependencias adicionales en la petición. Endpoints de wishlist:

-  **POST /wishlist/toggle:** recibe JWT en Authorization y body con id (Long). Añade o elimina el elemento según exista en la lista y devuelve un ToggleWishlistResponse.
-  **DELETE /wishlist/eliminar/{itemId}:** recibe JWT y itemId por path, eliminando el elemento específico de la wishlist.

- 📄 **GET /wishlist/mis-favoritos:** recibe JWT, un título opcional y Pageable, y devuelve una página de WishlistDTO con filtrado en memoria y ordenación por relevancia (ofertas primero).
- 📄 **GET /wishlist/ids:** recibe JWT y devuelve una lista de IDs (List<Long>) presentes en la wishlist del usuario.

TestController agrupa un conjunto de endpoints utilizados exclusivamente para durante la fase de desarrollo. Su responsabilidad no es formar parte del flujo funcional de la aplicación, sino permitir la validación rápida de procesos críticos como sincronizaciones, inserciones masivas o comprobaciones de integración con servicios externos.

Esta capa se apoya directamente en sus DTOs, para enviar y recibir información estructurada desde el frontend sin exponer directamente las entidades internas de la base de datos. De esta forma se desacopla la representación de datos de la lógica de persistencia.

Junto a los DTOs y los mappers (**FrontMapper, VistaMapper, etc.**) se encargan de transformar entidades del dominio en objetos preparados para el frontend y viceversa. Esto permite adaptar estructuras complejas a respuestas más ligeras, optimizadas y adecuadas para el consumo desde la interfaz de usuario.

En conjunto, la capa de controladores funciona como intermediaria entre el frontend y la lógica del backend, manteniendo una separación clara de responsabilidades.

7.6. Capa /service (Servicios)

La capa de servicios contiene la lógica de la aplicación. Actúa como intermediaria entre la capa de Controladores (api.controller) y la capa de acceso a datos (domain), coordinando las operaciones necesarias para cumplir los requisitos funcionales.

En esta capa se encapsulan procesos complejos como búsquedas, sincronización de datos, gestión de usuarios, manejo de *caché*, tareas asíncronas, etc. Su objetivo principal es mantener los controladores lo más ligeros posible, delegando toda la lógica a los servicios concretos.

Además, los servicios pueden combinar múltiples repositorios, clientes externos y utilidades internas, garantizando una separación de responsabilidades y una arquitectura escalable.

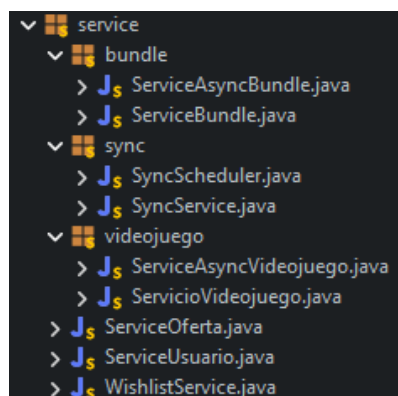


Figura 26. Capa service

7.6.1. Servicios bundle

Los servicios de bundles se dividen en dos clases: **ServiceBundle** y **ServiceAsyncBundle**. Ambos trabajan de forma conjunta para gestionar la obtención, construcción y almacenamiento de bundles dentro del sistema.

ServiceBundle se encarga de atender las peticiones del usuario y devolver una respuesta lo más rápida posible, priorizando la información ya disponible en base de datos o en *caché*. En caso de que el bundle no exista, se apoya en la capa external para obtener los datos desde Steam y activa el proceso de creación en segundo plano.

Por otro lado, **ServiceAsyncBundle** se encarga de la creación completa del bundle. Este proceso incluye la obtención de la información desde Steam, su transformación a entidades del sistema, el enriquecimiento de los productos con datos adicionales como imágenes, vídeos o capturas, y su posterior almacenamiento en base de datos. Además, gestiona la asociación de ofertas relacionadas y controla la concurrencia para evitar duplicados durante la creación.

La relación entre ambos servicios permite separar claramente la lectura rápida de datos y la construcción completa de los mismos. De esta forma, el usuario recibe una respuesta inmediata mientras el sistema va completando y optimizando la información en segundo plano.

Este módulo se apoya directamente en varias capas del sistema. Desde la capa de dominio se gestionan las entidades y repositorios, desde la capa external se obtienen los datos de Steam, desde config se habilitan mecanismos como la asincronía y el *caché*, y desde la capa API se realiza la transformación de entidades a DTOs para su consumo en el frontend.

7.6.2. Servicios de videojuego

Los servicios de videojuegos siguen la misma estructura que los de bundles, organizándose en dos componentes principales: **ServicioVideojuego** y **ServiceAsyncVideojuego**.

ServicioVideojuego se encarga de responder a las peticiones del usuario de forma rápida, utilizando base de datos o *caché* cuando es posible, y recurriendo a Steam en caso de que el videojuego no exista, activando su creación en segundo plano.

ServiceAsyncVideojuego gestiona la construcción completa del videojuego, incluyendo la obtención de datos desde Steam, su transformación a entidades, el enriquecimiento con información adicional y su almacenamiento en base de datos, además de la asociación de ofertas y la gestión de concurrencia.

Este flujo mantiene el mismo enfoque que en bundles, separando la consulta rápida de la creación en segundo plano para mejorar el rendimiento del sistema.

Al igual que en bundles, este módulo se apoya en las capas de dominio, external, config y API, que permiten la gestión de datos, comunicación con servicios externos, ejecución asíncrona y transformación de entidades.

7.6.3. Servicios de sync

Los servicios de sincronización se encargan de mantener actualizados los datos de ofertas y tiendas mediante procesos automáticos ejecutados en segundo plano. Este módulo se compone principalmente de **SyncScheduler** y **SyncService**, los cuales trabajan conjuntamente para coordinar la actualización periódica de la información del sistema.

El **SyncScheduler** define la planificación automática de las tareas mediante anotaciones **@Scheduled**, utilizando expresiones *CRON* para establecer distintos intervalos de ejecución. De esta forma, las ofertas se sincronizan cada cierta hora, mientras que las tiendas se actualizan de forma periódica en intervalos más amplios.

Por otro lado, **SyncService** centraliza la lógica de sincronización y controla la ejecución de los procesos para evitar sincronizaciones simultáneas. Desde este servicio se coordinan las llamadas a la capa externa encargada de obtener la información desde CheapShark, así como el posterior procesamiento y almacenamiento de los datos mediante el servicio de ofertas.

Este sistema permite automatizar completamente la actualización de datos sin intervención manual, manteniendo la información sincronizada de forma continua y reduciendo el impacto sobre el rendimiento general de la aplicación.

El módulo se apoya principalmente en la capa externa para la obtención de datos, en domain para la persistencia y en config para la ejecución de tareas programadas y asíncronas.

7.6.4. Servicio de ofertas

El servicio de ofertas se encarga de gran parte de la lógica principal del backend, ya que se encarga tanto de las búsquedas y filtrados avanzados como de la sincronización y actualización masiva de ofertas provenientes de CheapShark.

Una de sus funciones principales es la gestión de consultas dinámicas sobre las ofertas almacenadas. Para ello utiliza paginación mediante **Pageable** y especificaciones JPA (**Specification**), permitiendo construir filtros complejos según distintos parámetros enviados por el usuario, como precio, ahorro, valoración, tiendas y/o título. Este sistema facilita realizar búsquedas flexibles sin necesidad de crear múltiples consultas específicas.

El servicio también incorpora validaciones propias antes de ejecutar determinadas operaciones. Estas comprobaciones se realizan mediante excepciones personalizadas (**BadRequestException**), utilizadas para controlar datos inválidos como límites incorrectos, tamaños excesivos de filtros o valores fuera de rango.

Para optimizar el rendimiento, varios métodos utilizan la anotación **@Cacheable**, almacenando temporalmente resultados frecuentes como búsquedas rápidas, listado de tiendas o valores máximos de precio. Esto reduce considerablemente la carga sobre base de datos y mejora los tiempos de respuesta.

Otro de los componentes más importantes del servicio es el sistema de sincronización masiva de ofertas. Durante este proceso, el sistema obtiene miles de ofertas desde CheapShark y las procesa antes de almacenarlas en la base de datos.

Para evitar inconsistencias y bloqueos sobre la tabla principal de ofertas, se utiliza una estrategia basada en tablas staging. En lugar de insertar directamente los datos sincronizados en la tabla definitiva, todas las ofertas se almacenan primero en una tabla temporal (**oferta_staging**) mediante operaciones **upsert**, permitiendo insertar o actualizar registros existentes de forma eficiente.

Una vez finalizada completamente la carga de datos, el sistema ejecuta un proceso transaccional utilizando **@Transactional**, garantizando que todos los cambios sobre la base de datos se ejecuten como una única transacción. En este proceso se vacía la tabla principal de ofertas, se copian todos los registros desde la tabla staging hacia la tabla definitiva y posteriormente se limpia nuevamente la tabla temporal. Gracias a este enfoque, la actualización completa de ofertas se realiza de forma mucho más controlada y segura.

Tras finalizar la sincronización, el servicio invoca un evento (**SwapFinishedEvent**) mediante **ApplicationEventPublisher**. Este evento es capturado posteriormente desde la capa infrastructure, donde se refrescan automáticamente las materialized views y se limpian las *cachés* del sistema para garantizar que los nuevos datos estén disponibles inmediatamente.

Además, durante la sincronización también se gestionan automáticamente relaciones con videojuegos, bundles y tiendas existentes, se detectan nuevas tiendas provenientes de la API externa y se actualizan imágenes, videos o metadatos cuando es necesario.

Este módulo se apoya directamente en varias capas del sistema. Desde domain utiliza entidades y repositorios, desde external obtiene y transforma datos de CheapShark, desde infrastructure coordina eventos y refresco de vistas materializadas, desde exceptions controla validaciones y errores personalizados, y desde config utiliza funcionalidades como *caché*, tareas programadas y gestión transaccional.

7.6.5. Servicio de usuarios

El servicio de usuario se encarga de gestionar las operaciones relacionadas con la cuenta del usuario y su integración con el sistema de autenticación externo basado en Supabase.

Una de sus principales responsabilidades es la eliminación completa de cuentas. Para ello, el servicio primero elimina el usuario del sistema de autenticación de Supabase mediante peticiones HTTP autenticadas utilizando la **service_role_key**. Posteriormente, elimina también la información asociada dentro de la base de datos del sistema. Todo este proceso se ejecuta utilizando **@Transactional**, garantizando consistencia durante la operación.

El servicio también incluye funcionalidades relacionadas con la gestión de preferencias del usuario, como la activación o desactivación de notificaciones. Estas operaciones se realizan directamente sobre las entidades persistidas mediante los repositorios de la capa domain.

Otro aspecto importante es la extracción y validación del identificador de usuario desde los tokens JWT proporcionados por Supabase. Para ello, el servicio decodifica el payload del token utilizando **Base64** y procesa la información mediante **ObjectMapper**, obteniendo el UUID del usuario autenticado. En caso de errores durante el procesamiento del token, se lanzan excepciones controladas para evitar accesos inválidos.

Además, este módulo utiliza propiedades externas configuradas mediante **@Value**, permitiendo acceder de forma segura a variables sensibles como URLs o claves privadas almacenadas en el archivo de configuración.

Al igual que el resto de servicios, este componente se apoya en varias capas del sistema. Desde domain utiliza entidades y repositorios, desde external interactúa con Supabase mediante peticiones HTTP, desde exceptions controla errores personalizados y desde config aprovecha la gestión transaccional y configuración global de la aplicación.

7.6.6. Servicio de wishlist

El servicio de wishlist se encarga de gestionar la lista de favoritos de cada usuario, permitiendo añadir, eliminar y consultar videojuegos o bundles almacenados dentro de su colección personal.

La operación principal del servicio es **toggleWishlist**, encargada de añadir o eliminar automáticamente un elemento según si ya existe previamente en la wishlist. Esto permite centralizar toda la lógica de favoritos dentro del backend y simplificar considerablemente el funcionamiento del frontend.

Además, el servicio incluye funcionalidades para eliminar elementos individuales, vaciar completamente la wishlist y recuperar listados paginados de favoritos. Durante estas consultas también se aplican filtros por título y ordenación según si los elementos poseen ofertas activas, mejorando así la experiencia de navegación del usuario.

Para optimizar el rendimiento, se utiliza *caché* mediante **@Cacheable** y **@CacheEvict**, almacenando temporalmente los identificadores de elementos favoritos y limpiando automáticamente la *caché* cuando la wishlist sufre modificaciones.

Las operaciones de escritura utilizan **@Transactional**, garantizando consistencia durante la actualización de relaciones entre usuarios, videojuegos y bundles dentro de la base de datos.

El servicio también se integra directamente con los módulos de videojuegos y bundles. Si un usuario intenta añadir un elemento que todavía no existe en la base de datos, el sistema puede recuperarlo o crearlo automáticamente reutilizando la lógica ya implementada en **ServicioVideojuego** y **ServiceBundle**.

Este módulo se apoya principalmente en la capa domain mediante entidades y repositorios, en exceptions para el control de errores personalizados y en config para funcionalidades como *caché* y gestión transaccional.

7.7. Capa /external (APIs externas)

La capa *external* se encarga de gestionar toda la comunicación con servicios externos utilizados por la aplicación. Su objetivo principal es desacoplar el consumo de APIs externas del resto del sistema, centralizando tanto las peticiones HTTP como la transformación de datos externos hacia las entidades internas utilizadas por la aplicación.

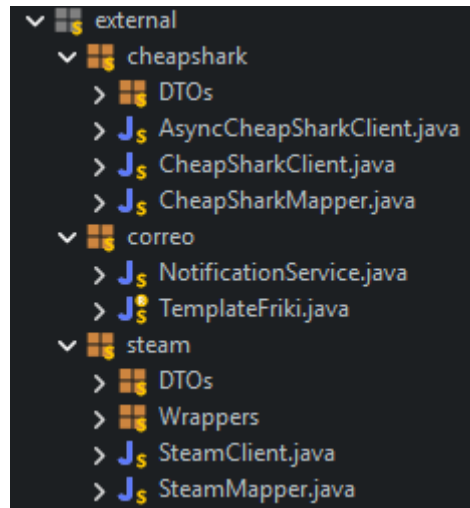


Figura 27. Capa external

Esta capa se encuentra organizada en distintos módulos según el proveedor externo utilizado, destacando principalmente las integraciones con *Steam*, *CheapShark* y el sistema de notificaciones mediante *Resend*. Cada integración dispone de sus propios *DTOs*, wrappers, clientes HTTP y mappers, permitiendo mantener separada la estructura de datos externa del modelo interno del sistema.

La integración con *Steam* se utiliza para obtener información detallada de videojuegos y bundles. Desde *SteamClient* se consumen endpoints como *appdetails* y *packagedetails* utilizando *RestClient*. Debido a que Steam devuelve respuestas dinámicas utilizando IDs como claves JSON, fue necesario implementar wrappers específicos para poder deserializar correctamente las respuestas recibidas.

Posteriormente, *SteamMapper* transforma los *DTOs* externos en entidades internas como *Videojuego*, *Bundle*, *Genero*, *Movie* o *Captura*. Durante este proceso también se realizan conversiones auxiliares mediante clases de la capa *util*, como *DateConversion* para el tratamiento de fechas o *SteamDecoderDescription* para limpiar y adaptar descripciones HTML procedentes de Steam.

Por otro lado, la integración con *CheapShark* representa una de las partes más importantes de toda la arquitectura del proyecto, ya que constituye el núcleo del sistema de sincronización automática de ofertas. La aplicación depende directamente de CheapShark como fuente principal de datos relacionados con precios, descuentos, tiendas y ofertas activas de videojuegos y bundles.

Desde *CheapSharkClient* se realiza el consumo masivo de la API externa mediante peticiones paginadas. El proceso de sincronización comienza descargando la primera página de ofertas para obtener el número total de páginas disponibles a través de las cabeceras HTTP devueltas por CheapShark.

A partir de este punto, el sistema inicia un proceso progresivo de descarga del resto de páginas utilizando procesamiento asíncrono mediante *CompletableFuture* y *@Async* desde *Async-CheapSharkClient*.

Uno de los principales problemas encontrados durante esta implementación fueron las limitaciones de *rate limiting* impuestas por CheapShark. Durante las primeras pruebas, realizar demasiadas peticiones consecutivas provocaba bloqueos temporales de la API y errores *429 Too Many Requests*. Para solucionar este problema fue necesario implementar distintos mecanismos de control, incluyendo:

-  pausas automáticas entre batches de peticiones,
-  tiempos aleatorios de espera entre sincronizaciones,
-  limitación del número de páginas descargadas simultáneamente,
-  control de errores automáticos cuando la API bloqueaba temporalmente las solicitudes.

Además, durante la sincronización también se filtran automáticamente aquellas ofertas que no pertenecen a Steam, ya que la aplicación trabaja principalmente sobre videojuegos asociados a esta plataforma.

Una vez descargadas todas las ofertas, el sistema ejecuta distintos procesos internos desde la capa de servicios, incluyendo:

-  validación de tiendas existentes,
-  guardado masivo de ofertas,
-  actualización de registros antiguos,
-  reconstrucción de vistas materializadas utilizadas posteriormente por el frontend.

Todo este proceso permite mantener sincronizada la base de datos local con el estado actual de CheapShark, permitiendo que el frontend pueda trabajar sobre datos optimizados y ya procesados sin depender directamente de llamadas constantes a APIs externas.

Además, *CheapSharkMapper* transforma los DTOs externos en entidades internas como *Oferta* y *Tienda*, adaptando imágenes, fechas, puntuaciones y URLs de compra. Incluso se implementó un sistema auxiliar capaz de comprobar automáticamente si Steam dispone de imágenes de mejor calidad para determinados productos mediante peticiones HTTP tipo *HEAD*.

Finalmente, esta capa también incluye el sistema de notificaciones por correo mediante *Resend*. Desde *NotificationService* se utilizan tareas programadas con *@Scheduled* para comprobar periódicamente si existen nuevas ofertas relacionadas con videojuegos o bundles presentes en la wishlist de los usuarios. Cuando se detectan cambios relevantes, el sistema genera automáticamente correos HTML personalizados utilizando plantillas dinámicas y realiza posteriormente el envío mediante *JavaMailSender*.

En conjunto, la capa *external* funciona como el punto central de integración con servicios externos del proyecto, concentrando tanto el consumo de APIs como el sistema de sincronización automática y los servicios auxiliares utilizados por la aplicación.

7.8. Capa /exceptions (Excepciones)

La capa de excepciones se encarga de gestionar los errores de la aplicación de forma controlada y consistente. En este caso, se utiliza principalmente una excepción personalizada para manejar situaciones en las que los datos enviados por el cliente no son válidos.

Se ha definido una excepción propia (**BadRequestException**) que se utiliza cuando se detectan errores en la validación de parámetros o en el formato de los datos recibidos. Esto permite diferenciar los errores de negocio de los errores internos del sistema.

Además, esta excepción se integra con el controlador global de excepciones mediante **@RestControllerAdvice**, lo que permite devolver respuestas HTTP estandarizadas al cliente sin necesidad de repetir lógica de manejo de errores en cada controlador.

Aunque actualmente la capa es sencilla, su estructura permite ampliarse fácilmente en el futuro con nuevas excepciones específicas según crezcan las necesidades del sistema.

7.9. Capa /infrastructure (Infraestructura)

La capa de infraestructura agrupa componentes técnicos que soportan el funcionamiento interno del sistema sin formar parte de la lógica de negocio. Su objetivo es coordinar tareas en específico como la gestión de eventos internos, la actualización de vistas materializadas y la limpieza de *cache* tras operaciones críticas.

Esta capa permite desacoplar procesos técnicos del resto de la aplicación, garantizando una mayor coherencia de datos y facilitando la ejecución de tareas que deben ocurrir automáticamente tras determinados cambios en el sistema.

7.10. Capa /util (Entity's y Repos)

La capa util reúne distintas clases auxiliares utilizadas en múltiples partes del sistema. Su objetivo es centralizar lógica reutilizable relacionada con conversiones, tipos compartidos, enumeraciones y procesamiento de datos externos, evitando duplicación de código entre servicios y facilitando el mantenimiento de la aplicación.

DateConversion se encarga de transformar fechas procedentes de APIs externas al formato utilizado internamente por Java. Por un lado, convierte timestamps Unix recibidos desde CheapShark a objetos **LocalDateTime**, y por otro procesa fechas de lanzamiento procedentes de Steam, soportando tanto formatos en inglés como en español. Además, controla casos especiales como juegos marcados como "Coming Soon".

Enums agrupa diferentes enumeraciones utilizadas en filtros y clasificaciones del sistema. Entre ellas destacan los niveles de ofertas (**OfferTier**) y las categorías de reviews de Steam (**Reviews**), utilizadas posteriormente en búsquedas avanzadas y filtrado dinámico de ofertas.

SteamDecoderDescription actúa como utilidad de saneamiento de texto, permitiendo decodificar correctamente descripciones procedentes de Steam que contienen caracteres escapados en formato JSON.

TypeRefs centraliza distintos tipos reutilizados en llamadas HTTP y lógica de negocio. Contiene referencias genéricas (**ParameterizedTypeReference**) utilizadas por **RestClient** para deserializar respuestas complejas de CheapShark y Steam. Además, incluye constantes globales del sistema, como los campos válidos de ordenación de ofertas, la conversión de puntuaciones de Steam a texto legible y la configuración de tiers de ofertas mediante estructuras auxiliares (**TierInfo**).

En conjunto, esta capa funciona como soporte común para el resto de módulos del sistema, especialmente los servicios, clientes externos y procesos de sincronización.

7.11. Capa /validation (Validación)

La capa de **validation** se encarga de centralizar las reglas de validación de los datos de entrada antes de que estos lleguen a la capa de servicios. Su objetivo principal es evitar la duplicación de lógica en los controladores y garantizar que las peticiones cumplen unos requisitos mínimos de consistencia y seguridad. Actualmente está centrada principalmente en la validación de la paginación y en la construcción de filtros dinámicos para las consultas de ofertas.

La clase **PageableValidator** se encarga de validar y asegurar la correcta construcción de los parámetros de paginación utilizados en los endpoints de ofertas. En ella se realizan validaciones como la comprobación de que la página no sea negativa o que el tamaño máximo no supere un límite definido, además de validar que el campo de ordenación sea correcto y que la dirección de ordenación sea válida. Estas validaciones se aplican sobre el objeto **Pageable** recibido desde el frontend, garantizando que no se puedan enviar parámetros inválidos o manipulados. Además de la validación, esta clase construye un **Pageable** seguro aplicando una ordenación secundaria por **steamAppId**, con el objetivo de garantizar consistencia en los resultados. También se incluye la validación del rango de páginas, comprobando que la página solicitada no exceda el total disponible en la respuesta.

La clase **VistaOfertaFiltros** implementa un sistema de filtros dinámicos utilizando **Spring Data JPA Specifications**, permitiendo construir consultas complejas de forma modular y reutilizable. Estos filtros se aplican directamente sobre la entidad de ofertas y permiten combinar diferentes criterios de búsqueda de forma flexible. Entre los filtros principales se incluyen la búsqueda por título, filtros de precio mínimo y máximo, filtrado por porcentaje de ahorro y la aplicación de filtros por categorías de oferta como los *tiers*, definidos en **TypeRefs**. También se incluyen filtros basados en valoraciones de Steam (**Reviews**) y otros criterios como la fecha de inicio de la oferta o la lista de tiendas asociadas, todos ellos contruidos de forma dinámica mediante Specifications.

Aunque actualmente la capa de **validation** está centrada principalmente en la paginación y los filtros de ofertas, su objetivo es convertirse en una capa centralizada de validación del sistema, trasladando progresivamente aquí el resto de validaciones que actualmente se encuentran distribuidas en controladores y servicios, con el objetivo de mejorar la cohesión del código, reducir la duplicación de lógica y facilitar el mantenimiento general de la aplicación a medio y largo plazo.

8. Frontend

8.1. Configuración Inicial

El desarrollo del frontend se realizó utilizando **React 19** como biblioteca principal para la construcción de la interfaz de usuario. Para la creación y configuración del proyecto se empleó **Vite**, una herramienta moderna de desarrollo que proporciona tiempos de compilación reducidos y una experiencia de desarrollo más fluida gracias a su sistema de recarga rápida de módulos.

La aplicación fue desarrollada utilizando **TypeScript**, un superset de JavaScript que incorpora tipado estático al lenguaje. Esta elección permitió aumentar la robustez del código, mejorar la detección temprana de errores y facilitar el mantenimiento del proyecto a medida que aumentaba su complejidad.

El frontend utilizado en este proyecto tuvo como punto de partida una aplicación desarrollada previamente durante las actividades realizadas en el ciclo formativo. Sin embargo, dicha aplicación fue sometida a una importante refactorización para adaptarla a los requisitos específicos del TFG. La estructura general del proyecto fue reorganizada, se añadieron nuevas funcionalidades y se modificó gran parte de los componentes existentes para ajustarlos a las necesidades de la plataforma.

Uno de los cambios más significativos fue la migración progresiva de la base de código desde **JavaScript** hacia **TypeScript**. Esta transición permitió definir interfaces y modelos para los datos intercambiados con el backend, reduciendo errores relacionados con el tipado dinámico y mejorando la legibilidad y mantenibilidad del código fuente.

La aplicación se desarrolló siguiendo el modelo de **Single Page Application (SPA)**, permitiendo que la navegación entre las distintas secciones se realizara sin necesidad de recargar completamente la página. Esta aproximación mejora la experiencia del usuario y reduce el tiempo necesario para acceder a la información. Para complementar el desarrollo se incorporaron diversas bibliotecas externas:

-  **React Router DOM**, para la gestión de rutas y navegación entre páginas.
-  **Axios**, para la comunicación con la API REST del backend.
-  **Supabase JavaScript SDK**, para la autenticación y gestión de usuarios.
-  **React Toastify**, para la visualización de notificaciones.
-  **Framer Motion**, para la creación de animaciones e interacciones visuales.
-  **HLS.js**, para la reproducción de contenido multimedia en formato HLS.

Asimismo, se configuró un sistema de variables de entorno mediante el archivo **.env**, utilizado para almacenar parámetros de configuración que pueden variar entre los distintos entornos de ejecución, aunque de momento solo se utiliza para la configuración de supabase.

Durante el desarrollo también se incorporaron herramientas orientadas a mejorar la calidad del software, como **ESLint** para el análisis estático del código y **Vitest** junto con **Testing Library** para la ejecución de pruebas automatizadas.

8.2. Arquitectura del frontend

El frontend se ha desarrollado siguiendo una arquitectura modular basada en la separación de responsabilidades, con el objetivo de facilitar el mantenimiento, la escalabilidad y la reutilización del código. Esta organización permite dividir la aplicación en distintos módulos especializados, donde cada uno cumple una función concreta dentro del sistema.

A diferencia del backend, donde se utiliza una arquitectura clásica en capas, en el frontend se ha optado por una estructura orientada a componentes y funcionalidades. De esta forma, las vistas se encargan de la presentación de la información, los componentes encapsulan elementos reutilizables de la interfaz, los servicios gestionan la comunicación con el backend y los contextos permiten compartir información entre diferentes partes de la aplicación.

Esta separación reduce el acoplamiento entre módulos y facilita la incorporación de nuevas funcionalidades sin afectar significativamente al resto del sistema. Además, la utilización de **TypeScript** aporta una capa adicional de seguridad mediante el tipado estático de los datos intercambiados entre componentes y servicios.

8.2.1. Estructura del proyecto

La organización del proyecto sigue una estructura modular donde cada directorio agrupa elementos relacionados con una responsabilidad específica. Esta distribución permite mantener una base de código más ordenada y facilita la localización de funcionalidades durante el desarrollo y mantenimiento de la aplicación.

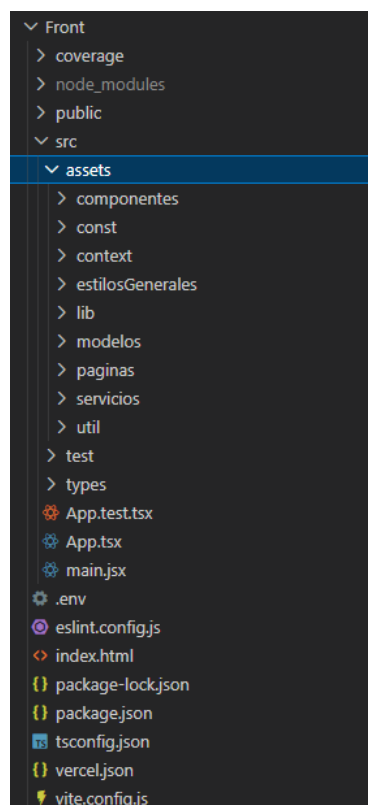


Figura 28. Estructura frontend

-  **/coverage** contiene los informes generados durante la ejecución de las pruebas automatizadas. En esta carpeta se almacenan las métricas de cobertura del código, permitiendo analizar qué porcentaje de la aplicación está siendo validado mediante pruebas y detectar posibles áreas que requieran una mayor cobertura.
-  **/node_modules** almacena todas las dependencias instaladas mediante npm. Este directorio se genera automáticamente a partir del archivo **package.json** y contiene las librerías externas necesarias para el funcionamiento del proyecto. No se incluye en el control de versiones debido a su gran tamaño y a que puede regenerarse fácilmente mediante la instalación de dependencias.
-  **/public** almacena recursos estáticos accesibles directamente desde la aplicación sin necesidad de ser procesados por Vite durante la compilación. Entre estos recursos se incluyen imágenes, iconos, fuentes tipográficas personalizadas y otros elementos utilizados por la interfaz de usuario.
-  **/src** contiene el código fuente principal de la aplicación y agrupa todos los módulos necesarios para su funcionamiento.
-  **/componentes** contiene los componentes reutilizables de la interfaz de usuario. Estos elementos encapsulan funcionalidades específicas que pueden ser utilizadas desde distintas páginas de la aplicación, evitando duplicación de código y favoreciendo la consistencia visual.
-  **/paginas** agrupa las diferentes vistas principales de la aplicación. Cada página representa una funcionalidad concreta accesible mediante el sistema de navegación y se construye utilizando componentes reutilizables.
-  **/servicios** centraliza la comunicación con el backend mediante peticiones HTTP. Esta capa abstrae las llamadas a la API REST y evita que la lógica de acceso a datos se encuentre dispersa por los componentes de la interfaz.
-  **/context** contiene los contextos utilizados para la gestión del estado global de la aplicación. Gracias al uso de Context API es posible compartir información entre diferentes componentes sin necesidad de propagar propiedades de forma excesiva.
-  **/modelos** almacena las clases y estructuras de datos utilizadas internamente por la aplicación para representar la información recibida desde el backend.
-  **/types** contiene interfaces y definiciones de tipos de TypeScript utilizadas para garantizar el tipado estático y mejorar la seguridad durante el desarrollo.
-  **/const** agrupa constantes globales utilizadas en diferentes partes de la aplicación, evitando la repetición de valores y facilitando su mantenimiento.
-  **/util** incluye funciones auxiliares y utilidades reutilizables que apoyan distintas funcionalidades del sistema.
-  **/lib** almacena configuraciones e integraciones con librerías externas utilizadas por la aplicación, como la configuración de Supabase o herramientas de terceros.
-  **/estilosGenerales** contiene los estilos globales compartidos por toda la aplicación, permitiendo mantener una apariencia visual homogénea entre todas las vistas.
-  **/assets** agrupa recursos y módulos relacionados con la interfaz y la lógica del frontend, actuando como directorio principal para la organización interna de la aplicación.

- 📁 **/test** está destinado a las pruebas automatizadas del frontend. En él se desarrollan pruebas unitarias e integradas para validar el correcto funcionamiento de los componentes y funcionalidades de la aplicación.
- 📁 **App.tsx** actúa como componente raíz de la aplicación, definiendo la estructura principal y la configuración general de navegación.

8.3. Componentes

El frontend se ha diseñado bajo un enfoque basado en **componentes reutilizables**, con el objetivo de modularizar la interfaz, reducir duplicación de código y facilitar el mantenimiento y escalabilidad de la aplicación.

Para ello, se ha seguido una estructura similar a la utilizada en *frameworks* como Angular, donde cada componente se encuentra encapsulado en su propia carpeta. Dentro de cada una se agrupan los elementos necesarios para su funcionamiento, principalmente:

- 📁 El archivo principal del componente (**.tsx o .ts**), donde se define la lógica y la estructura visual.
- 📁 El archivo de estilos (**.css**), encargado de la presentación.
- 📁 El archivo para realizar el testing (**test.tsx**).
- 📁 En algunos casos, archivos auxiliares o utilidades específicas del componente.

Esta organización permite que cada componente sea independiente, reutilizable y fácilmente sustituible sin afectar al resto del sistema.

8.3.1. Tipología de componentes

Dentro del proyecto se distinguen principalmente dos tipos de componentes:

- 📁 **Componentes simples:** encargados de una única responsabilidad, como botones, loaders y/o elementos visuales.
- 📁 **Componentes compuestos:** contruidos a partir de otros componentes, integrando lógica más compleja y representación de datos más compleja.

Un ejemplo claro de componente compuesto es la tarjeta de oferta, ya que integra múltiples elementos y depende de otros componentes internos.

8.3.2. Listado de componentes principales

Componente	Tipo	Descripción
Header	Compuesto	Componente de navegación superior. Incluye enlaces a secciones principales y un buscador dinámico con debounce y conexión al backend. Gestiona el estado de autenticación y muestra opciones como wishlist, cuenta o logout.
Footer	Simple	Pie de página estático con enlaces informativos como privacidad, contacto y acerca del proyecto. Se muestra de forma global en la mayoría de vistas.
OfertaTarjeta	Compuesto	Muestra información de una oferta de videojuego (título, imagen, precio y descuento). Permite navegación a detalle e integración con wishlist, con animaciones y estados de carga.
OfertasLista	Compuesto	Contenedor que renderiza listas de ofertas en formato grid. Permite alternar entre tarjetas normales y de wishlist según el contexto.
PanelFiltros	Compuesto	Sistema de filtrado avanzado para ofertas. Permite filtrar por precio, título, descuento, tiendas, tiers y valoraciones con controles interactivos.
ModalMedia	Compuesto	Modal para visualizar imágenes y vídeos de videojuegos. Soporta navegación entre elementos e incluye reproducción de vídeo mediante HLS.
WishListBoton	Simple	Botón para añadir o eliminar elementos de la wishlist. Usa estado optimista y sincroniza cambios con el backend.
WishListTarjeta	Compuesto	Variante de tarjeta para wishlist. Mantiene la estructura de oferta, pero añade lógica de eliminación y actualización en tiempo real.
Paginator	Simple	Componente de paginación dinámico. Gestiona navegación entre páginas con cálculo automático de rangos y scroll al cambiar.
Contacto	Simple	Componente configurable que permite contacto por email y acceso a redes sociales mediante props.
ModalBorrado	Simple	Modal de confirmación para eliminación de cuenta. Requiere confirmación explícita y ejecuta acciones mediante callbacks.

8.3.3. Análisis de componente

El componente **OfertaTarjeta** representa una de las piezas principales dentro de la interfaz de la aplicación, ya que se encarga de la visualización individual de cada oferta de videojuego dentro de los listados.

Este componente se integra dentro de **OfertasLista**, que actúa como contenedor encargado de renderizar múltiples instancias de tarjetas en formato grid, permitiendo su reutilización en distintas vistas del sistema.

El componente recibe la información mediante *props*, lo que garantiza su independencia respecto al estado global y facilita su reutilización. A partir de estos datos, construye la interfaz mostrando elementos como título, imagen, precio, descuento y valoración de la oferta.

Además, incorpora navegación mediante *react-router-dom*, permitiendo el acceso directo a la vista de detalle del videojuego desde cada tarjeta. También utiliza animaciones con *framer-motion*, mejorando la fluidez visual durante el renderizado en listados dinámicos.

Incluye el componente *WishListBoton*, que permite gestionar la lista de deseos directamente desde la tarjeta, integrando así funcionalidad interactiva dentro de la propia presentación.

Finalmente, implementa renderizado condicional en función del estado de los datos, mostrando estados de carga (skeleton o placeholders) o adaptando la interfaz en caso de datos incompletos o no disponibles, garantizando una visualización consistente.

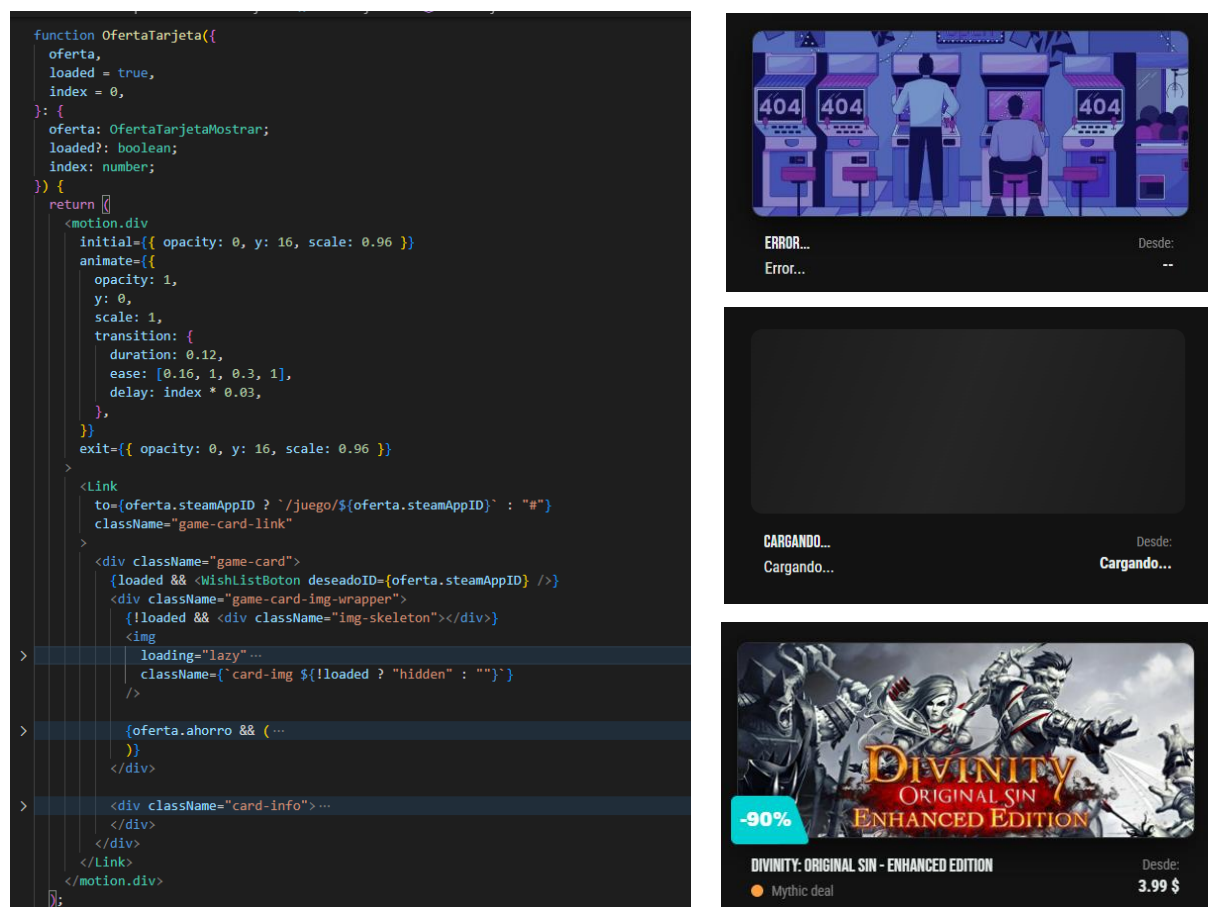


Figura 29. Componente OfertaTarjeta

8.4. Páginas/Vistas

Las páginas de la aplicación constituyen las distintas vistas principales del frontend, encargadas de organizar y presentar la información al usuario final. Cada una de estas vistas actúa como un contenedor de lógica de presentación, integrando componentes reutilizables, servicios y estados necesarios para construir la experiencia completa de la plataforma.

Su diseño sigue un enfoque modular, donde cada página representa una funcionalidad concreta del sistema, como la visualización de ofertas, la gestión de la cuenta del usuario o la consulta de información detallada de un videojuego. De este modo, se consigue una navegación clara y coherente entre las distintas secciones de la aplicación.

8.4.1. Página de inicio

La página de inicio actúa como el punto de entrada principal de la aplicación y tiene como objetivo ofrecer una visión rápida, visual y organizada de las ofertas más relevantes disponibles en la plataforma. Su diseño está orientado a destacar contenido dinámico y facilitar el acceso a las secciones más importantes del sistema.

Una de sus principales características es la presentación de **tres bloques diferenciados de ofertas destacadas**, cada uno de ellos construido a partir de las **6 mejores ofertas disponibles** según distintos criterios de clasificación:

-  **Ofertas más recientes**, que muestran los últimos productos añadidos al sistema.
-  **Ofertas con mayor ahorro**, centradas en las oportunidades con mayor porcentaje de descuento.
-  **Ofertas en tendencia**, basadas en la relevancia y popularidad dentro de la plataforma.

De esta forma, el usuario puede obtener en un solo vistazo una visión global del estado del catálogo, sin necesidad de aplicar filtros manualmente.

Además, la página incorpora elementos visuales de bienvenida y presentación del proyecto, reforzando la identidad de la aplicación y guiando al usuario hacia el resto de secciones, como el listado completo de ofertas o el sistema de filtrado avanzado.

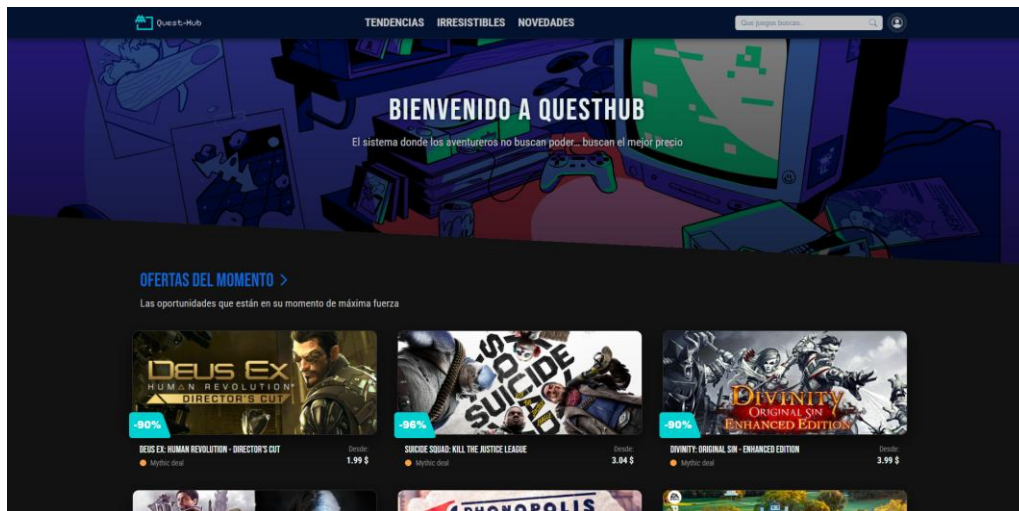


Figura 30. Página de inicio

8.4.2. Página de ofertas

La página de ofertas constituye una de las vistas principales del sistema, ya que permite consultar de forma completa el catálogo de ofertas disponibles en la plataforma. Está diseñada como un espacio de exploración avanzada, donde el usuario puede buscar, ordenar y filtrar grandes volúmenes de información de manera dinámica.

Una de sus funcionalidades clave es la **ordenación de resultados**, que permite reorganizar las ofertas según diferentes criterios, entre los que destacan:

-  Ofertas más recientes, mostrando los últimos registros añadidos al sistema.
-  Por precio, tanto ascendente como descendente.
-  Por porcentaje de ahorro, priorizando las mejores rebajas.
-  Por tendencia o relevancia, destacando las ofertas más populares o significativas.

Gracias a este sistema de ordenación, el usuario puede adaptar la visualización del catálogo según sus necesidades o intereses en cada momento.

Además, la página incorpora un **panel de filtros avanzado**, que permite refinar los resultados de búsqueda de forma precisa. Este panel actúa directamente sobre el listado de ofertas, aplicando restricciones en tiempo real según distintos parámetros, como:

-  Tienda de origen.
-  Rango de precios (mínimo y máximo).
-  Título de la oferta.
-  Nivel de descuento o ahorro.
-  Otros criterios adicionales de clasificación.

Este sistema de filtrado permite reducir significativamente el volumen de resultados, facilitando la localización de ofertas específicas dentro del catálogo.

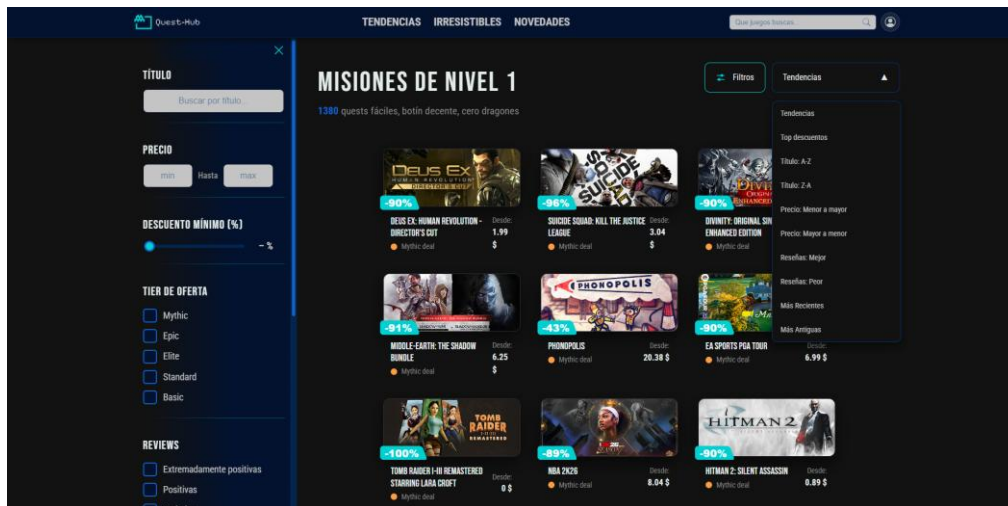


Figura 31. Página de ofertas con todo abierto

8.4.3. Página de detalle de videojuego/bundle

La página de detalle de videojuego es la vista encargada de mostrar toda la información específica de un título seleccionado, ya sea un videojuego individual o un bundle. Su objetivo principal es ofrecer una visión completa y centralizada del contenido, integrando datos descriptivos, multimedia y ofertas disponibles en un único entorno.

En el caso de los **videojuegos individuales**, esta página muestra información detallada como la descripción del juego, desarrolladores, distribuidores, fecha de lanzamiento, géneros, valoración y reseñas. Además, incluye una sección específica donde se listan las distintas ofertas disponibles en diferentes tiendas, permitiendo al usuario comparar precios directamente desde la propia vista de detalle.

Cuando se trata de un **bundle**, la página adapta su contenido para mostrar la estructura del conjunto, presentando todos los productos incluidos dentro del mismo. Cada uno de estos elementos se muestra con su respectiva información básica, como nombre, imagen y descripción, permitiendo al usuario comprender fácilmente el contenido completo del paquete.

Uno de los elementos más relevantes de esta vista es la integración de un **modal de contenido multimedia**, encargado de mostrar imágenes y vídeos asociados al videojuego. Este modal permite visualizar capturas y trailers de forma ampliada, ofreciendo una experiencia más inmersiva y facilitando la exploración del contenido visual sin salir de la página.

Asimismo, la página incorpora un **botón de wishlist**, que permite al usuario añadir o eliminar el videojuego o bundle de su lista de deseos. Esta funcionalidad está integrada directamente en la vista de detalle, lo que facilita la gestión rápida de juegos de interés sin necesidad de navegar a otras secciones de la aplicación.

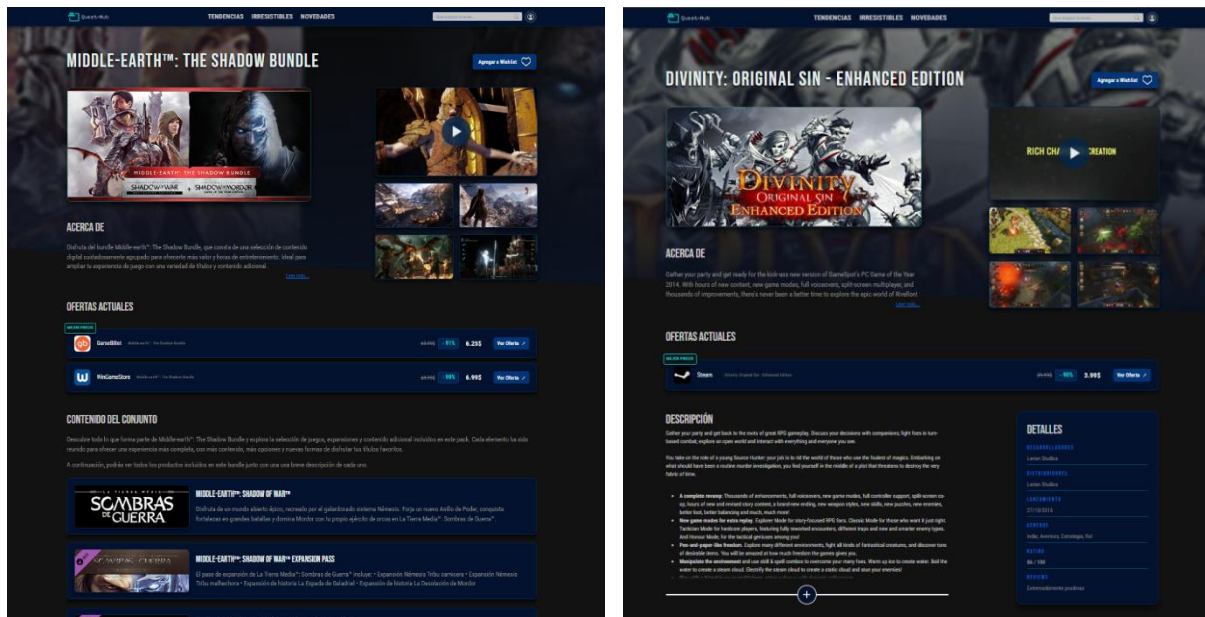


Figura 32. Página de detalles

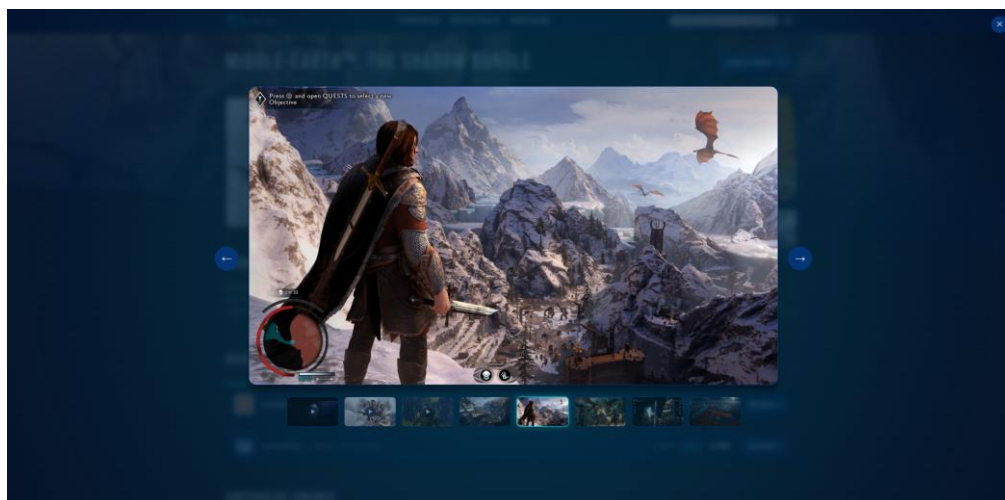


Figura 33. Modal Multimedia

8.4.4. Página de cuenta de usuario

La página de cuenta de usuario está diseñada como un espacio de gestión personal dentro de la aplicación, donde el usuario puede consultar y modificar información básica relacionada con su perfil y preferencias de uso.

En primer lugar, esta vista muestra los **datos principales del usuario**, destacando su correo electrónico y el nombre de usuario asociado a la cuenta. Esta información se presenta de forma sencilla y directa, ya que actúa únicamente como panel de consulta.

Una de las funcionalidades más importantes de esta sección es la posibilidad de **gestionar las notificaciones por correo electrónico**. El usuario puede activar o desactivar el envío de avisos relacionados con ofertas y actualizaciones de los videojuegos incluidos en su wishlist. Este control permite personalizar la experiencia y ajustar el nivel de comunicación recibido desde la plataforma.

Por último, la página incluye una opción para **eliminar la cuenta de usuario**, una acción permanente que elimina los datos asociados al perfil dentro del sistema. Esta funcionalidad está implementada como una medida crítica y se presenta de forma clara para que el usuario sea consciente de sus implicaciones antes de ejecutarla.

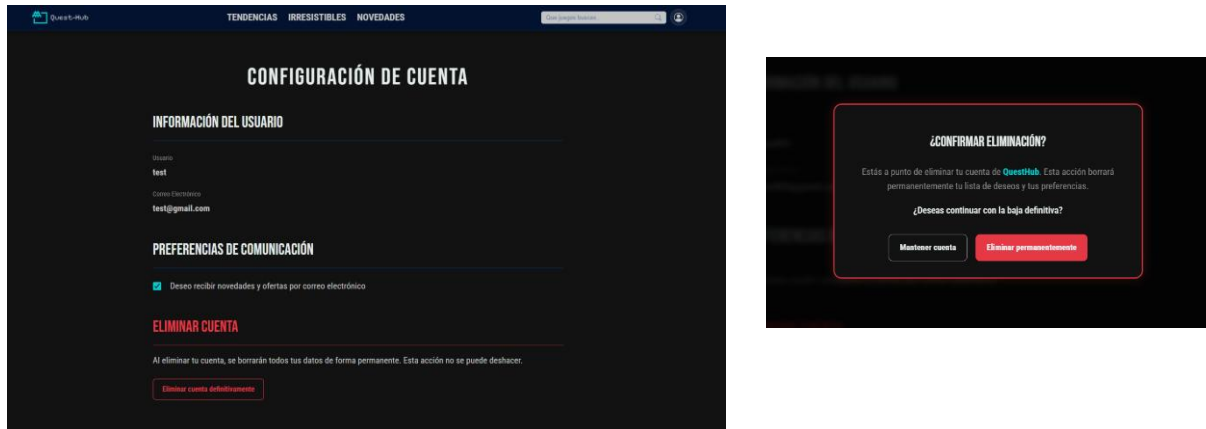


Figura 34. Pagina cuenta y modal de borrado

8.4.5. Página de wishlist

La página de Wishlist se encarga de mostrar los videojuegos que el usuario ha añadido previamente a su lista de deseos, actuando como un espacio personal donde puede consultar rápidamente los títulos que le interesan.

Esta vista presenta los elementos de la wishlist en forma de tarjetas, reutilizando el mismo formato visual que el resto de la aplicación para mantener coherencia en la interfaz. Cada elemento incluye información básica del videojuego, como su nombre, imagen y estado de oferta, permitiendo al usuario identificar de forma rápida cada entrada.

Una característica importante de esta página es que los elementos se **ordenan automáticamente según su estado de oferta**, priorizando aquellos videojuegos que actualmente se encuentran en promoción. De esta forma, los juegos con descuento aparecen en las primeras posiciones, mientras que los que no están en oferta se muestran al final de la lista.

Además, la vista incorpora un **buscador interno**, que permite filtrar los elementos de la wishlist por título. Este buscador facilita la localización rápida de un juego concreto dentro de una lista potencialmente extensa, mejorando la usabilidad y la experiencia de navegación.

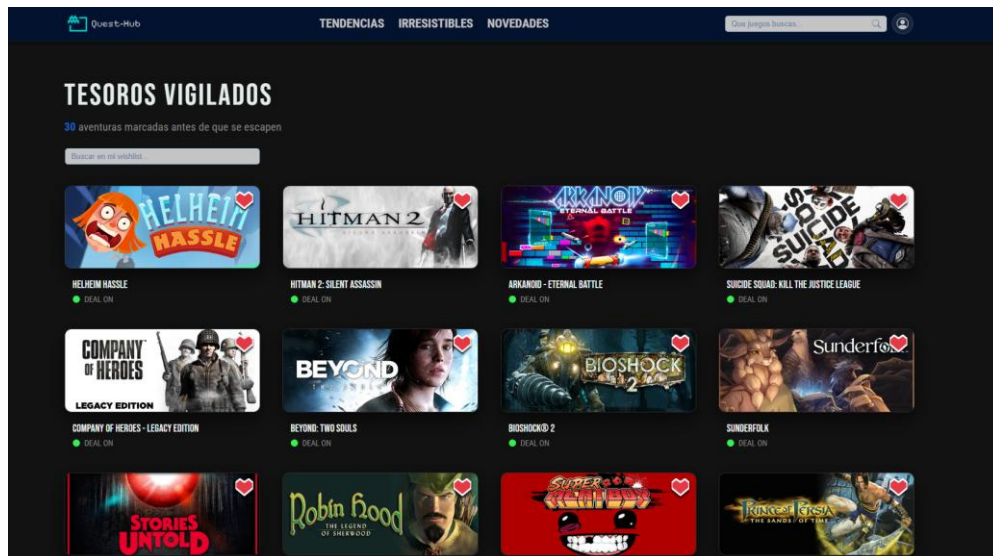


Figura 35. Página de Wishlist

8.4.6. Página de login y signup

La página de login y signup es la vista encargada de gestionar el acceso de los usuarios a la plataforma, permitiendo tanto el inicio de sesión como la creación de nuevas cuentas dentro del sistema. Su diseño integra ambas funcionalidades en una misma interfaz, alternando entre el modo de login y el de signup de forma dinámica.

Una de las características principales de esta página es la posibilidad de **mostrar u ocultar las contraseñas** en ambos formularios, tanto en el inicio de sesión como en el registro. Esto mejora la usabilidad, ya que permite al usuario verificar fácilmente los datos introducidos y evitar errores al escribir su contraseña.

En el proceso de **registro de nuevos usuarios**, se realiza una validación adicional en el frontend para comprobar que las dos contraseñas introducidas coincidan antes de enviar la información. El resto de validaciones, como la verificación de credenciales o la creación de la cuenta, son gestionadas directamente por el sistema de autenticación de Supabase, que se encarga de garantizar la seguridad y consistencia del proceso.

Además del formulario principal, la página cuenta con un **panel visual complementario**, situado en el lateral, que muestra imágenes de videojuegos acompañadas de frases temáticas relacionadas con la experiencia del usuario dentro de la plataforma. Estos elementos son dinámicos y van cambiando, aportando una experiencia más atractiva y reforzando la identidad visual del proyecto.

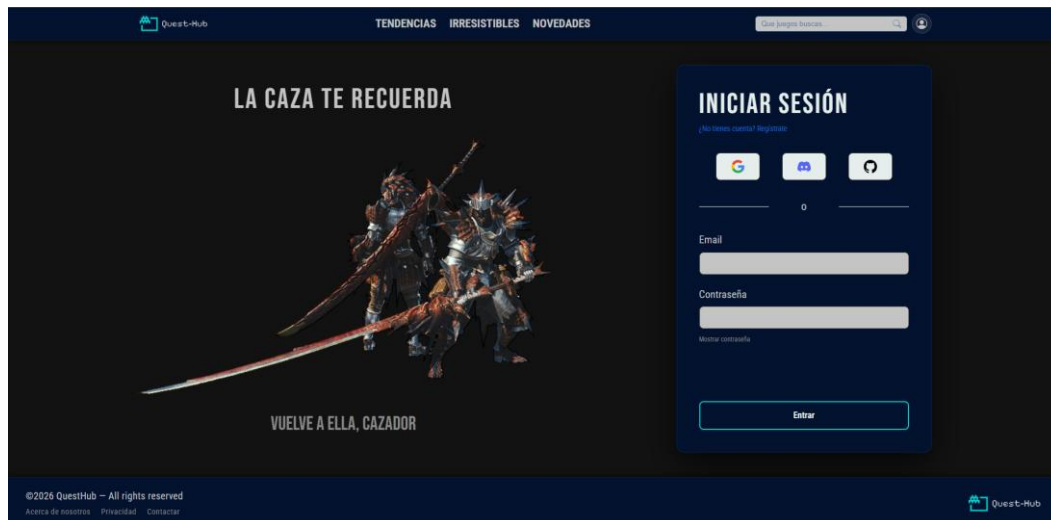


Figura 36. Página de login

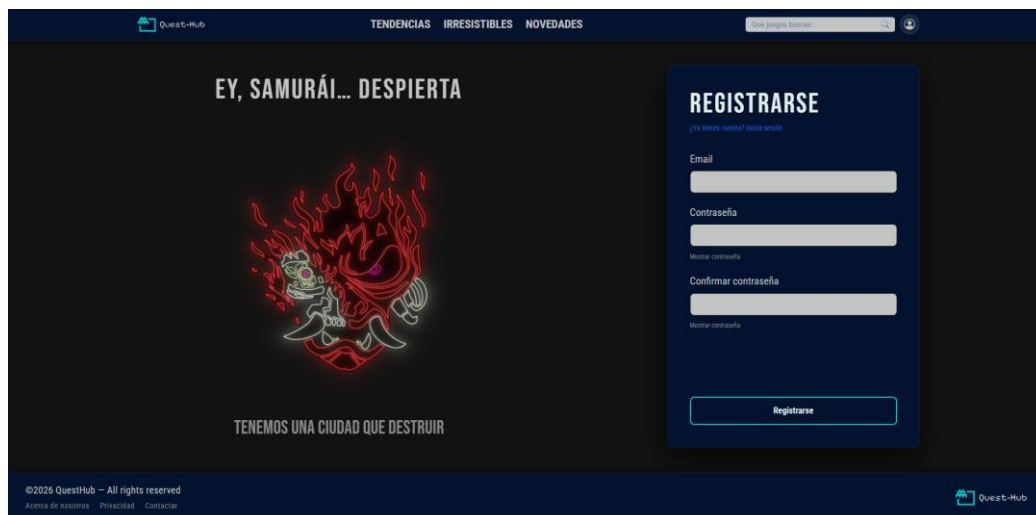


Figura 37. Página de signup

8.4.7. Página 404

La página de error 404 es la vista que se muestra cuando el usuario intenta acceder a una ruta que no existe dentro de la aplicación. Su objetivo principal es informar de forma clara que la página solicitada no ha podido ser encontrada y redirigir al usuario de manera amigable hacia contenido válido del sistema.

En lugar de mostrar un mensaje genérico, esta vista está diseñada con un enfoque más dinámico y temático, manteniendo la estética general de la aplicación. Para ello, incorpora elementos visuales llamativos, como imágenes animadas aleatorias (GIFs) relacionadas con videojuegos, que refuerzan el estilo del proyecto y hacen que la experiencia de error sea menos abrupta.

Además, la página incluye **mensajes personalizados aleatorios**, que cambian cada vez que se accede a esta vista. Estos mensajes aportan variedad y un tono más cercano al usuario, evitando una experiencia repetitiva en caso de errores de navegación.

También dispone de opciones de navegación que permiten al usuario **volver a la página anterior o regresar directamente a la página de inicio**, facilitando la recuperación de la navegación dentro de la aplicación sin necesidad de usar el historial del navegador manualmente.

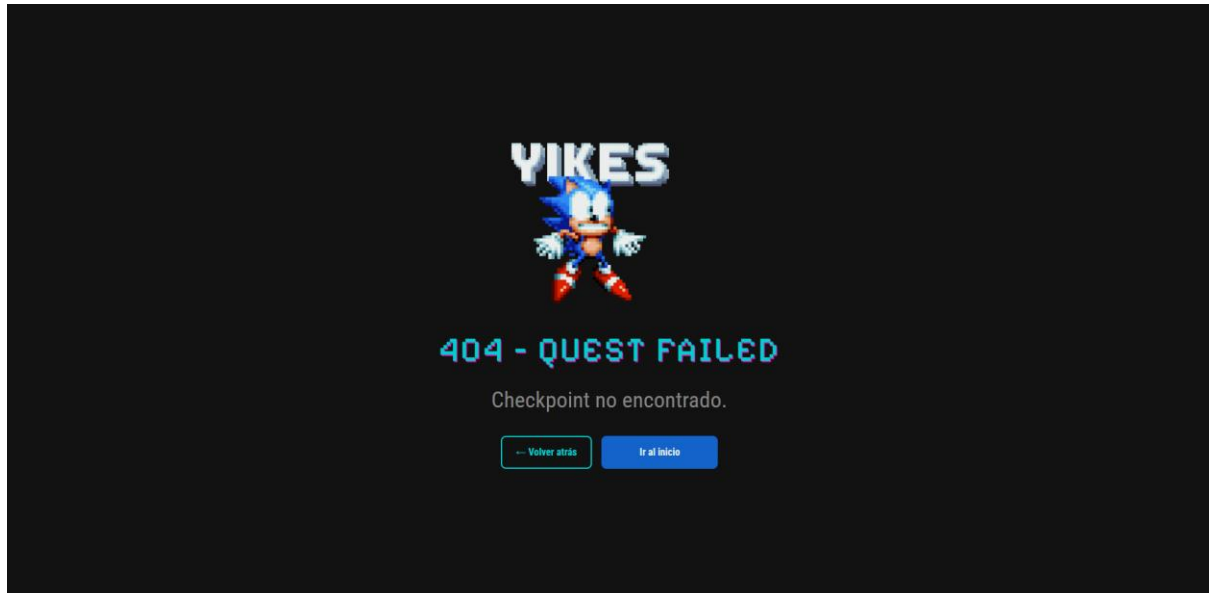


Figura 38. Página 404

8.4.8. Página de privacidad

La página de privacidad tiene como objetivo informar al usuario sobre el tratamiento de sus datos dentro de la plataforma, cumpliendo con los principios básicos de protección y transparencia de la información personal.

En esta vista se detalla qué tipo de datos se recopilan, cómo se utilizan y con qué finalidad son tratados. También se explican las medidas de seguridad aplicadas para proteger la información del usuario, así como las condiciones bajo las cuales dichos datos podrían ser compartidos, siempre de forma limitada y justificada.

Asimismo, se incluyen los derechos que tiene el usuario sobre su información personal, como el acceso, modificación o eliminación de sus datos, garantizando así el control sobre su propia información dentro del sistema.



Figura 39. Página privacidad

8.4.9. Página de acerca de

La página de contacto está orientada a facilitar la comunicación entre los usuarios y el equipo desarrollador del proyecto. En esta sección se ofrece información general sobre los creadores de la aplicación, explicando brevemente el contexto del proyecto y su finalidad dentro del entorno académico.

Además, la vista incluye distintos canales de contacto, como direcciones de correo electrónico y enlaces a redes sociales o perfiles profesionales, permitiendo a los usuarios comunicarse fácilmente con el equipo o conocer más sobre los desarrolladores.

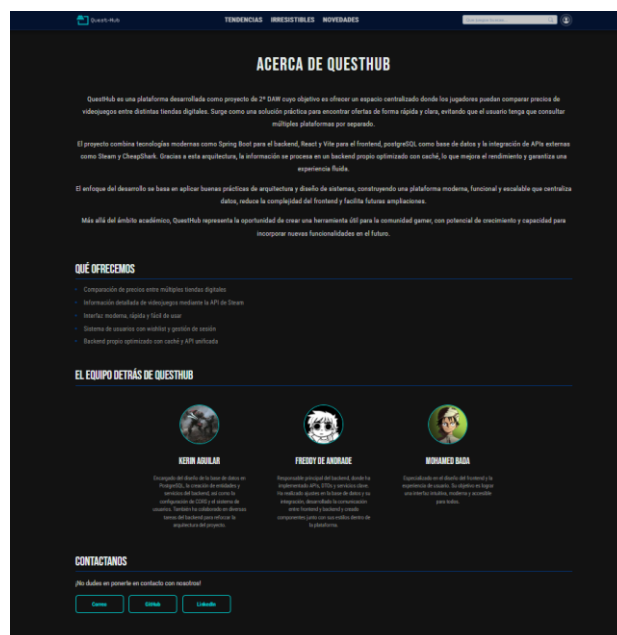


Figura 40. Página acerca de

8.5. Comunicación con backend

8.5.1. Axios e interceptor HTTP

La comunicación con el backend se basa en una instancia centralizada de Axios, configurada específicamente para la aplicación. Esta instancia define una *baseURL* dinámica que cambia automáticamente según el entorno de ejecución, utilizando la API local en desarrollo y la API en producción cuando la aplicación está desplegada. De esta forma se evita la duplicación de rutas y se simplifica el mantenimiento del código.

Además, se estandarizan las cabeceras de las peticiones HTTP, principalmente utilizando **Content-Type: application/json**, lo que garantiza un formato consistente en el intercambio de información entre frontend y backend.

Sobre esta instancia se implementa un sistema de interceptores, que permite gestionar de forma global tanto las peticiones como las respuestas:

-  Interceptor de peticiones (request): antes de enviar cualquier solicitud, se comprueba el estado de la variable global **backCaído**. Si el backend ha sido detectado como no disponible, la petición se rechaza automáticamente, evitando llamadas innecesarias al servidor.
-  Interceptor de respuestas (response): se encarga de gestionar los distintos tipos de errores de forma centralizada:
 -  Si no existe *error.response*, se interpreta como un fallo de conexión o backend caído, activando **backCaído** y mostrando una notificación al usuario.
 -  **Errores 400**: peticiones inválidas, mostrando el mensaje devuelto por el servidor.
 -  **Errores 404**: recurso no encontrado, redirigiendo automáticamente a la página de error 404.
 -  **Errores 500**: errores internos del servidor, mostrando un mensaje genérico de fallo.
 -  **Otros errores**: tratados como errores generales con notificación al usuario.

Este sistema permite controlar los errores de red de forma global, evitando repetir lógica en cada servicio y asegurando un comportamiento uniforme en toda la aplicación.

8.5.2. Servicios de comunicación con el backend

Sobre la instancia de Axios se construye una capa de servicios que encapsula todas las peticiones HTTP de la aplicación. Estos servicios actúan como intermediarios entre el frontend y la API, agrupando las llamadas por dominio funcional. Cada servicio se encarga de un conjunto específico de funcionalidades:

-  **Servicio de ofertas**: gestiona la obtención de ofertas, filtrado avanzado, ordenación, búsqueda por título, consulta de detalles de videojuegos o bundles y obtención de información adicional como precios máximos.

- 📁 **Servicio de tiendas:** obtiene el listado de tiendas disponibles, utilizado principalmente en los filtros de la página de ofertas.
- 📁 **Servicio de usuarios:** gestiona las preferencias del usuario, como las notificaciones por correo y la eliminación de la cuenta.
- 📁 **Servicio de wishlist:** permite añadir o eliminar juegos de la lista de deseos, obtener los favoritos del usuario y consultar sus identificadores.
- 📁 **authSync:** se encarga de sincronizar los datos que nos llega del usuario de supabase a los de nuestra base de datos.

8.6. Estados globales (Context)

Se utiliza **React Context** como sistema de estado global para gestionar información compartida entre múltiples componentes. Este enfoque permite centralizar estados críticos como la autenticación del usuario y la wishlist, manteniendo la coherencia de datos en toda la aplicación.

8.6.1. Contexto de autenticación

El contexto de autenticación es el encargado de gestionar todo el ciclo de vida del usuario dentro de la aplicación, incluyendo inicio de sesión, registro, cierre de sesión y sincronización con el backend.

Este contexto se basa en la integración con **Supabase Auth**, y expone de forma global el estado del usuario (user) y la sesión activa (session), además de un estado de carga (loading) y un indicador de sincronización (isSynced) con el backend propio.

Una de las funciones clave de este contexto es la **escucha de cambios de autenticación**, que permite reaccionar automáticamente a eventos como:

- 📁 Inicio de sesión (SIGNED_IN)
- 📁 Cierre de sesión (SIGNED_OUT)
- 📁 Registro o cambios de sesión

Cuando el usuario inicia sesión, se realiza una **sincronización automática con el backend propio (Spring Boot)**, enviando los datos necesarios para registrar o actualizar el usuario en el sistema. Este proceso asegura que ambos sistemas (Supabase y backend propio) estén alineados.

Además, este contexto centraliza toda la lógica de autenticación, incluyendo:

- 📁 Inicio de sesión con email y contraseña
- 📁 Registro de nuevos usuarios
- 📁 Login mediante proveedores externos (Google, Discord y GitHub)
- 📁 Cierre de sesión
- 📁 Notificaciones automáticas asociadas a cada acción

De esta forma, cualquier componente de la aplicación puede acceder al estado del usuario y a las funciones de autenticación sin necesidad de gestionar lógica adicional.

8.6.2. Contexto de wishlist

El contexto de wishlist se encarga de gestionar de forma global la lista de deseos del usuario, permitiendo acceder y modificar esta información desde cualquier parte de la aplicación.

Este contexto mantiene un array de identificadores de videojuegos (wishlist) que representa los elementos guardados por el usuario. A partir de este estado se genera una estructura optimizada para mejorar la eficiencia en la comprobación de existencia de elementos.

Una de sus funciones principales es la **carga inicial de la wishlist**, que se realiza cuando el usuario ha iniciado sesión y la sincronización con el backend ha finalizado. Esto asegura que los datos estén actualizados antes de interactuar con la interfaz.

También se encarga de la función de **toggle de wishlist**, que permite añadir o eliminar juegos de la lista de deseos. Este proceso incluye una actualización optimista del estado (cambio inmediato en la UI) y posteriormente la sincronización con el backend. En caso de error, se revierte el estado anterior para mantener la consistencia de los datos. Además, expone utilidades globales como:

-  Comprobar si un juego está en la wishlist
-  Recargar manualmente los datos de la wishlist

8.7. Rutas de navegación

La navegación de la aplicación se gestiona mediante **React Router DOM**, organizando todas las vistas en un sistema de rutas centralizado dentro del componente principal `App`. Esta estructura permite definir de forma clara y mantenible qué páginas forman parte de la aplicación y cómo se accede a cada una de ellas.

Para mantener una estructura consistente en toda la interfaz, se utiliza un **Layout principal**, que envuelve la mayoría de las rutas. Este *layout* incluye elementos comunes como el **Header** y el **Footer**, mientras que el contenido específico de cada página se renderiza dinámicamente a través de **Outlet**. Esto evita la repetición de código y asegura una experiencia de navegación uniforme.

```

16 function App() {
17   const Layout = () => {
18     return (
19       <>
20         <Header />
21         <Outlet />
22         <Footer />
23       </>
24     );
25   };
26 };
27
28 return (
29   <>
30     <div className="fondo">
31       <Routes>
32         <Route element={<Layout />} />
33         <Route path="/ofertas" element={<Ofertas />} />
34         <Route path="/login" element={<Login />} />
35         <Route path="/" element={<Inicio />} />
36         <Route path="/acerca" element={<Acerca />} />
37         <Route path="/privacidad" element={<Privacidad />} />
38         <Route
39           path="/cuenta"
40           element={
41             <ProtectedRoute>
42               <Cuenta />
43             </ProtectedRoute>
44           }
45         />
46         <Route
47           path="/wishlist"
48           element={
49             <ProtectedRoute>
50               <WishList />
51             </ProtectedRoute>
52           }
53         />
54         <Route path="/juego/:id" element={<GameDetalles />} />
55       </Routes>
56       <Route path="*" element={<Error404 />} />
57     </div>
58   </>
59 );
60 };
61 }
62

```

Figura 41. Código de las rutas

8.7.1. Rutas

Rutas públicas:

-  **/:** Página de inicio.
-  **/ofertas:** Pagina de ofertas.
-  **/juego/:id:** Página de detalles.
-  **/login:** Pagina de login/signup.
-  **/acerca:** Pagina acerca de.
-  **/privacidad:** Pagina de privacidad.
-  *****: Página de error 404 para rutas inexistentes.

Rutas protegidas:

-  **/cuenta:** Pagina de cuenta del usuario.
-  **/wishlist:** Pagina wishlist del usuario.

8.7.2. Protección de rutas (ProtectedRoute)

Para controlar el acceso a las páginas privadas se implementa un componente llamado **ProtectedRoute**, que actúa como una capa de seguridad a nivel de frontend.

Este componente utiliza el contexto global de autenticación (**useAuth**) para determinar si el usuario tiene una sesión activa. Su funcionamiento es el siguiente:

- 📁 Si el usuario **no está autenticado** y la comprobación de estado ya ha finalizado (*loading = false*), se redirige automáticamente al usuario a la página de **/login** mediante *Navigate*.
- 📁 Si el usuario **sí está autenticado**: Se permite el acceso y se renderiza el contenido protegido (**children**).

Este sistema evita que usuarios no autenticados puedan acceder directamente a rutas privadas incluso si intentan escribir la URL manualmente en el navegador.

8.8. Modelos y constantes

8.8.1. Modelos (estructuras de datos)

Los **modelos** definen la forma y estructura de los datos que se manejan dentro de la aplicación, especialmente aquellos que provienen del backend o que se comparten entre distintos servicios y componentes.

Entre los principales modelos utilizados se encuentran:

- 📁 Usuarios (UserResponse, AuthContextType)
- 📁 Ofertas (PageOfertas, DetalleResponse, SearchOfertas)
- 📁 Wishlist (PageWishlist, ToggleWishlistResponse)
- 📁 Tiendas (Tienda)
- 📁 Estructuras de paginación y filtros (FilterPageable, PageableMod)

El uso de estos modelos permite:

- 📁 Definir estructuras de datos claras y consistentes.
- 📁 Evitar errores derivados de datos mal formateados o inesperados.
- 📁 Mejorar la legibilidad del código.
- 📁 Facilitar la comunicación con el backend mediante contratos bien definidos.

```
export interface Videojuego {
  id: number;
  imagen: string;
  imagenCapsule: string;
  nombre: string;
  ratingText: string;
  rating: number;
  lanzamiento: string;
  descripcion: string;
  descripcionCorta: string;
  acercaDe: string;
  desarrolladores: string;
  distribuidores: string;

  generos: string[];
  movies: Movie[];
  capturas: Captura[];
  ofertas: Oferta[];
  bundles: VideojuegoBundle[];
}
```

Figura 42. Modelo para los datos de videojuego

8.8.2. Constantes (configuración global)

Las **constantes** agrupan valores fijos que se utilizan en distintas partes de la aplicación, con el objetivo de centralizar configuraciones y evitar su repetición.

En este proyecto se emplean principalmente para:

- Configuración de ordenación: `DEFAULT_SORT_BY`, `DEFAULT_DIRECTION`
- Tipos de notificaciones (`typeToast`)
- Iconos utilizados en la interfaz (`toastIcons`)
- Parámetros reutilizables en filtros o peticiones

Este enfoque permite:

- Evitar valores hardcodeados dispersos por el código.
- Facilitar cambios globales desde un único punto.
- Mantener consistencia en el comportamiento de la aplicación.
- Reutilizar configuraciones en múltiples servicios y componentes.

```
export const sortLabels: Record<string, {order:SortBy,dir:Direction}> = {  
  "Tendencias":{order:SortBy.RATING,dir:Direction.DESC},  
  "Top descuentos":{order:SortBy.AHORRO,dir:Direction.DESC},  
  "Título: A-Z":{order:SortBy.TITULO,dir:Direction.ASC},  
  "Título: Z-A":{order:SortBy.TITULO,dir:Direction.DESC},  
  "Precio: Menor a mayor":{order:SortBy.PRECIO,dir:Direction.ASC},  
  "Precio: Mayor a menor":{order:SortBy.PRECIO,dir:Direction.DESC},  
  "Reseñas: Mejor":{order:SortBy.REVIEWS,dir:Direction.DESC},  
  "Reseñas: Peor":{order:SortBy.REVIEWS,dir:Direction.ASC},  
  "Más Recientes":{order:SortBy.RECIENTE,dir:Direction.DESC},  
  "Más Antiguas":{order:SortBy.RECIENTE,dir:Direction.ASC},  
};
```

Figura 43. Constante para la ordenación de ofertas y labels

8.9. Utilidades (utils)

8.9.1. Sistema de notificaciones (Toast)

La aplicación implementa un sistema global de notificaciones basado en **React Toastify**, centralizado en la función `enviarNoti`. Este sistema se utiliza en prácticamente toda la app (servicios, contextos y acciones del usuario) para mostrar feedback inmediato sobre lo que está ocurriendo.

Se definen tipos estándar de notificación como *success*, *error*, *info* y *warn*, lo que permite unificar el estilo y el comportamiento de los mensajes en toda la aplicación.

Además, se centralizan colores e iconos personalizados, lo que da coherencia visual al sistema de alertas. Cada tipo de notificación tiene un icono por defecto, aunque también se puede sobrescribir con uno personalizado si es necesario.

La función `enviarNoti` recibe el tipo, título, mensaje y opcionalmente un icono, y se encarga de construir el contenido del toast con un formato común. Esto permite que todas las notificaciones mantengan la misma estructura visual, mejorando la experiencia del usuario y evitando duplicar lógica en cada parte del código donde se necesite mostrar un aviso.

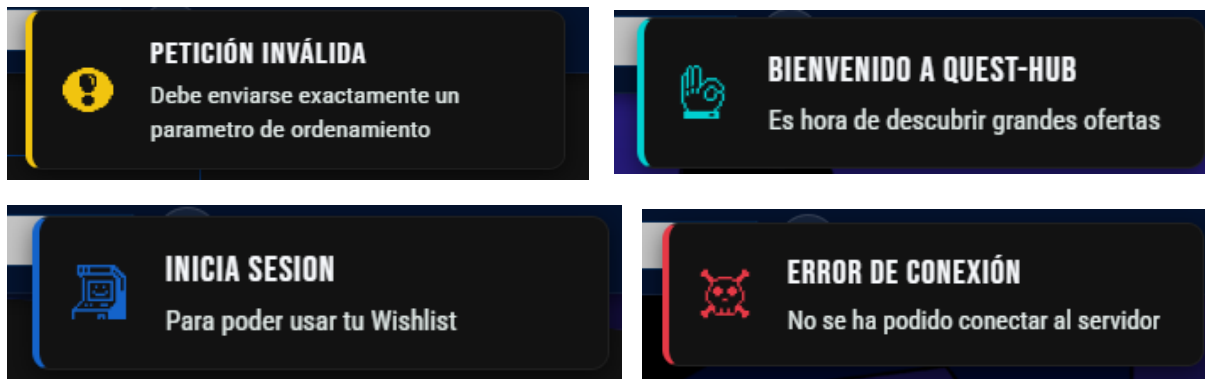


Figura 44. Los distinto tipos de notificaciones

8.9.2. Scroll automático y navegación visual

Para mejorar la experiencia de navegación, se implementa un sistema de control de scroll que se encarga de reposicionar la vista al inicio de la página cuando el usuario cambia de ruta.

Esto se realiza mediante el componente *ScrollToTop*, que escucha los cambios en la ruta usando **useLocation** de React Router. Cada vez que el **pathname** cambia, se ejecuta un efecto que fuerza el scroll a la posición inicial de la página, evitando que el usuario permanezca en la posición anterior al navegar entre vistas.

Además, se incluye una función utilitaria llamada *smoothScrollToTop*, que realiza el mismo comportamiento, pero con una animación suave. Esta función se utiliza en casos concretos como botones de “volver arriba” o interacciones dentro de páginas largas, mejorando la fluidez visual de la aplicación.

9. Testing

9.1. Frontend

El testing del frontend fue una de las áreas donde se alcanzó un mayor nivel de madurez dentro del proyecto. Desde el principio se estableció como objetivo lograr una cobertura mínima del **80%** en general, con la intención de asegurar que la interfaz funcionara de forma estable y que las rutas, componentes, vistas/páginas y demás elementos respondieran correctamente ante diferentes escenarios. Este enfoque buscaba no solo detectar errores, sino también garantizar que el comportamiento de la aplicación fuera consistente a medida que se añadían nuevas funcionalidades.

Para llevar a cabo este proceso se utilizaron **Vitest** e **Istanbul**, dos herramientas que se integraron de manera natural con el entorno de desarrollo basado en Vite. Vitest permitió ejecutar pruebas de forma rápida y flexible, facilitando el mockeo de dependencias y la validación de componentes de manera aislada. Por su parte, Istanbul proporcionó informes detallados de cobertura, permitiendo identificar qué partes del código estaban siendo ejecutadas durante las pruebas y cuáles requerían una mayor atención.

El trabajo de testing se centró en validar la renderización de los componentes principales, la navegación entre rutas, la gestión de estados internos y la respuesta a eventos del usuario. Se prestó especial atención a los elementos que forman la estructura base de la aplicación, como el enrutado, las vistas/páginas, los componentes y demás elementos, ya que cualquier fallo en estas áreas tendría un impacto directo en la experiencia del usuario.

Los resultados obtenidos fueron positivos. El informe de cobertura muestra que la mayoría de los componentes superaron ampliamente el umbral marcado, con archivos como *OfertaTarjeta.tsx* alcanzando un **100%** de cobertura. Otros componentes relevantes también presentan niveles elevados de cobertura, lo que refleja un esfuerzo consistente por validar las partes más críticas del frontend y asegurar su correcto funcionamiento.

En conjunto, el testing del frontend cumplió con los objetivos planteados y dejó una base sólida para futuras ampliaciones o refactorizaciones, proporcionando una mayor confianza en la estabilidad del sistema y reduciendo la probabilidad de errores en fases posteriores del desarrollo.

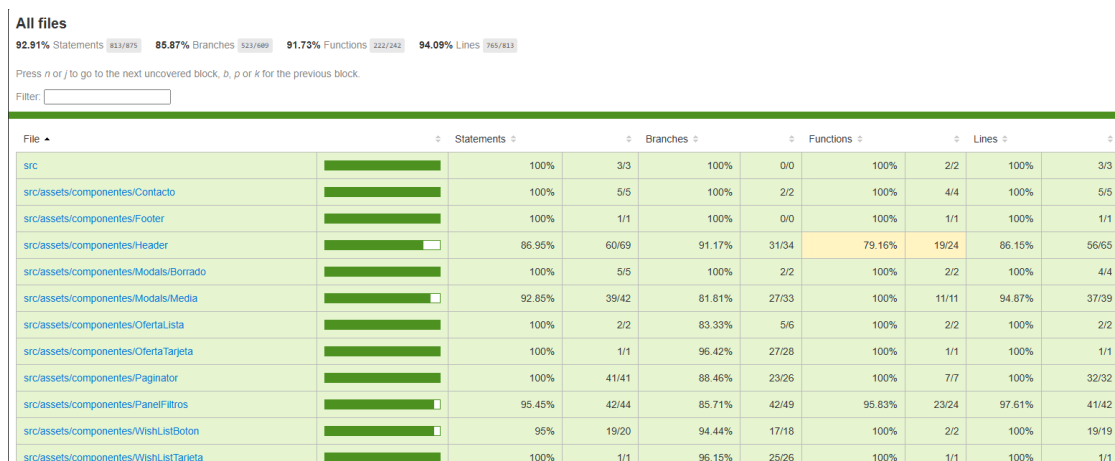


Figura 45. Cobertura Front con Istanbul

9.2. Backend

El testing del backend se utilizaron las herramientas incluidas en Spring Boot, en la dependencia de `starter-test`. El objetivo era validar la lógica interna de la aplicación y asegurar un comportamiento consistente en los componentes principales.

Sin embargo, debido al tiempo disponible y a la carga técnica del proyecto, no fue posible desarrollar un conjunto completo de pruebas automatizadas sino solamente algunas. La complejidad de elementos como la sincronización de ofertas, el uso de tareas programadas, la *caché*, la integración con APIs externas y la gestión de JWT requería un esfuerzo adicional que excedía el margen temporal del proyecto, luego de los distintos problemas que han surgido a lo largo del desarrollo. Por ello, no fue posible cubrir todo este de las pruebas necesarias para tener un coverage de al menos el 80% en todos los parámetros.

Aun así, se establecieron las bases necesarias para ampliar el testing en futuras versiones, ya que la arquitectura del backend está preparada para incorporar pruebas más avanzadas, incluyendo tests de integración, simulación de APIs externas y validación de procesos asincrónicos.

10. Despliegue

Durante el desarrollo se dedicó una parte importante al despliegue, con el objetivo de acercar la aplicación a una arquitectura real basada en servicios desacoplados. El sistema final se compone de diferentes plataformas, cada una responsable de una parte concreta de la aplicación: base de datos, backend, frontend, autenticación y servicios auxiliares.

Se trabajó también en la automatización del despliegue mediante integración con GitHub, así como en la correcta gestión de variables de entorno y configuración de servicios en producción.

El flujo de trabajo utilizado para el despliegue consistía en mantener una rama específica destinada únicamente a despliegue. Los cambios únicamente eran mergeados en esta rama cuando previamente se comprobaba que funcionaban correctamente y no generaban errores. De esta forma se evitaba desplegar versiones inestables y se mantenía un entorno más controlado.

10.1. Integración con GitHub (CI/CD)

Tanto el backend como el frontend se integraron con GitHub para permitir un flujo de despliegue continuo (CI/CD).

Cada vez que se realiza un push a la rama designada al despliegue (**Despliegue**), los servicios conectados detectan automáticamente los cambios y generan una nueva versión desplegada sin necesidad de intervención manual, en este caso **Railway** y **Vercel**. Esto permitió:

-  automatizar el despliegue completo de la aplicación
-  mantener el entorno siempre actualizado
-  reducir errores humanos en el proceso de publicación

10.2. Despliegue en Neon (Base de datos)

Se utilizó **Neon** como proveedor de base de datos **PostgreSQL** en la nube para el desarrollo de la aplicación. Inicialmente se evaluaron alternativas como **Supabase** y **Railway**, pero finalmente se eligió Neon por su simplicidad, compatibilidad con PostgreSQL estándar y facilidad de integración con **Spring Boot**.

Uno de los principales retos fue la configuración de la conexión en el backend, especialmente en el archivo *application.properties*, donde fue necesario ajustar correctamente la URL de conexión, el uso de **SSL** y las credenciales, ya que hubo algunos problemas al inicio con la conexión.

Durante el desarrollo surgieron problemas derivados de la evolución constante del modelo de datos, lo que provocaba cambios frecuentes en tablas y relaciones. Esto generaba incompatibilidades temporales entre versiones del backend y la base de datos.

Por este motivo, la base de datos fue el primer componente en ser desplegado, ya que durante las primeras fases del proyecto se trabajaba con instancias locales de PostgreSQL compartidas entre

los miembros del equipo. Esto provocaba problemas frecuentes de sincronización cuando algún integrante realizaba cambios en el esquema sin una comunicación inmediata, obligando a realizar ajustes constantes en el entorno local.

El despliegue de la base de datos en **Neon** permitió centralizar el acceso a un único entorno común, evitando inconsistencias entre versiones locales y facilitando notablemente el trabajo en equipo. Gracias a esto, todos los integrantes pudieron trabajar sobre la misma estructura de datos en tiempo real, reduciendo errores y mejorando la coordinación del desarrollo.

Finalmente, el despliegue definitivo se consolidó una vez estabilizada la estructura del sistema, lo que permitió trabajar con un entorno más consistente y controlado.

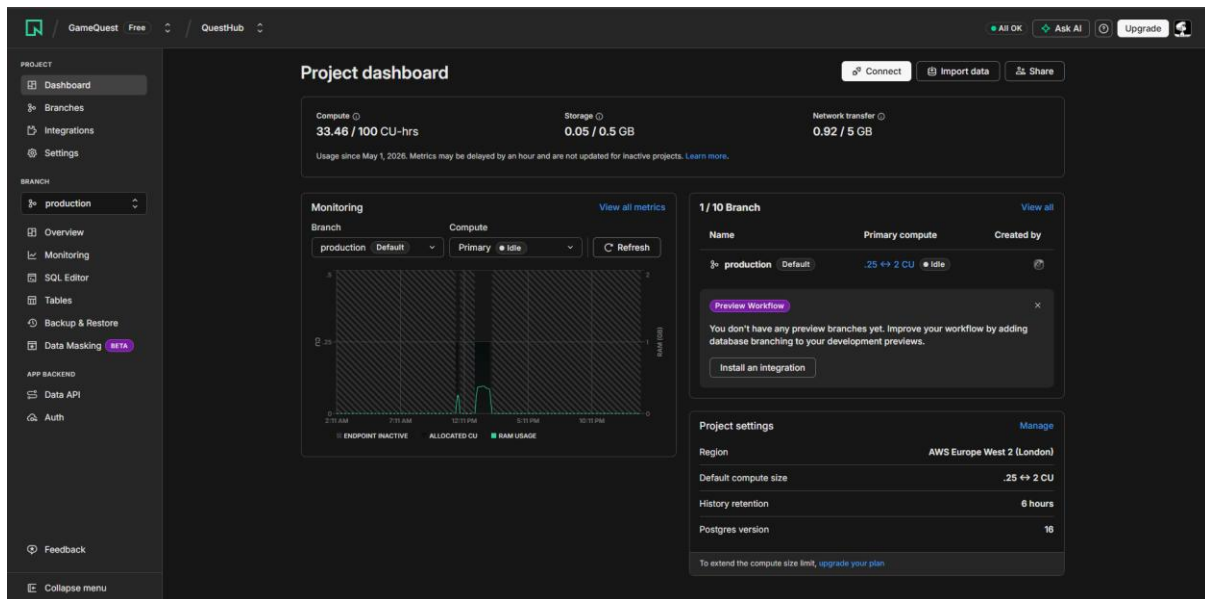


Figura 46. Dashboard del proyecto en Neon

10.3. Despliegue en Railway (Backend)

El backend fue desplegado en **Railway**, seleccionando esta plataforma por su compatibilidad directa con aplicaciones **Java** y su facilidad de configuración, así como por la integración sencilla con repositorios de **GitHub** y su capacidad de despliegue automático.

Durante el despliegue se configuraron todas las variables de entorno necesarias directamente en Railway, incluyendo credenciales de base de datos, claves de **Supabase**, tokens de servicios externos y parámetros de configuración sensibles. Esto permitió evitar la exposición de información crítica a la hora de realizar el despliegue.

En el backend se preparó previamente el archivo *application.properties* para soportar la carga de estas variables mediante entorno, utilizando referencias dinámicas en lugar de valores *hardcodeados*. Esto permitió mantener una configuración flexible y reutilizable, reduciendo posibles errores derivados de cambios de entorno y facilitando el despliegue en diferentes escenarios.

Además, Railway ofrece un sistema de *logs* en tiempo real, lo que facilitó la monitorización del comportamiento del backend, la detección de errores y la validación de procesos internos como sincronizaciones, llamadas a APIs externas y ejecución de tareas programadas. Este sistema de logs resultó especialmente útil durante las fases iniciales del despliegue, ya que permitió identificar rápidamente problemas como el bloqueo de la API de **CheapShark**, que en local se había resuelto pero desplegado volvió a aparecer.

Finalmente, la plataforma proporcionó un entorno estable de ejecución continua, permitiendo mantener el backend siempre disponible y reduciendo la necesidad de mantenimiento manual una vez configurado correctamente el despliegue. No obstante, al utilizar la versión gratuita de Railway, actualmente el despliegue del backend se mantiene apagado cuando no se está utilizando para ahorrar recursos hasta el momento de la presentación final del proyecto.

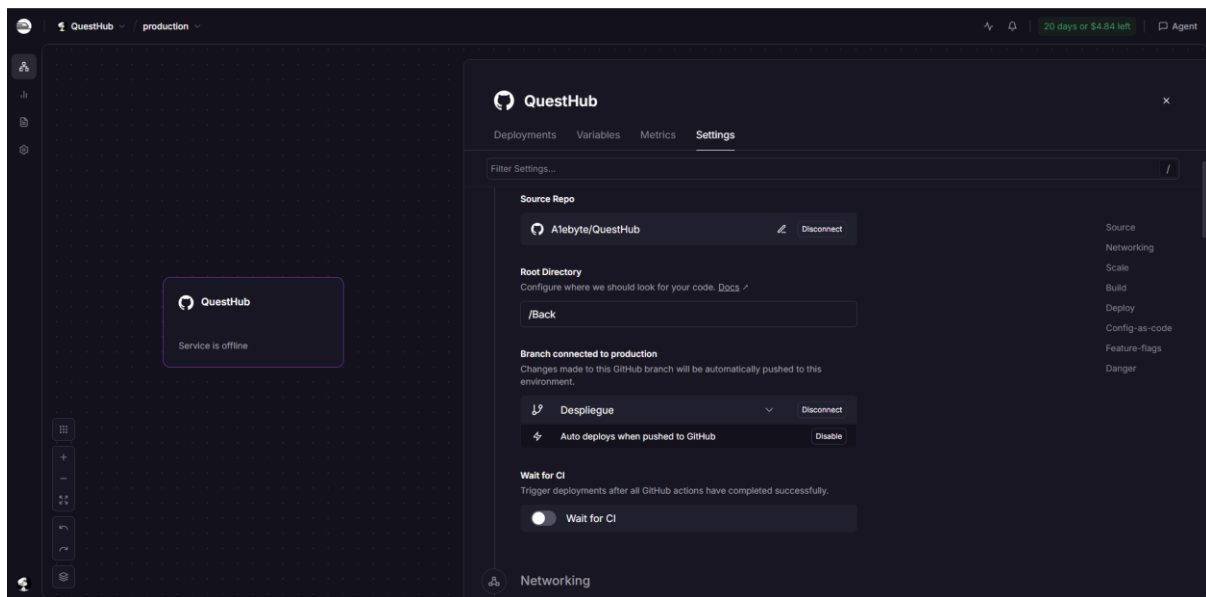


Figura 47. Dashboard del proyecto en Railway

10.4. Despliegue en Vercel (Frontend)

El frontend fue desplegado utilizando **Vercel** como plataforma debido a su compatibilidad con proyectos desarrollados en **React** y **Vite**, así como por la simplicidad y automatización que ofrece para aplicaciones frontend.

La configuración del despliegue resultó relativamente sencilla, ya que Vercel detecta automáticamente tecnologías como React y Vite y genera gran parte de la configuración necesaria de forma automática. Además, al utilizar una estructura *monorepo* compartiendo frontend y backend dentro del mismo repositorio, fue necesario configurar correctamente el directorio raíz (*root folder*) correspondiente al frontend para indicar qué parte del proyecto debía desplegarse.

Uno de los aspectos más importantes fue la integración con **GitHub**, que permitió conectar directamente el repositorio con la plataforma de despliegue. De esta forma, cada vez que se realizaban

cambios en la rama destinada al despliegue, Vercel detectaba automáticamente las modificaciones y generaba una nueva versión de la aplicación.

Además, Vercel proporciona herramientas de monitorización y *logs* de compilación, lo que facilitó la detección de errores relacionados con dependencias, configuraciones o fallos durante el proceso de *build*, como también métricas de tráfico, velocidad de la web y más.

Finalmente, la rapidez de despliegue de la plataforma permitió disponer de nuevas versiones del frontend en pocos minutos, ofreciendo un entorno estable y eficiente para la publicación de la interfaz web.

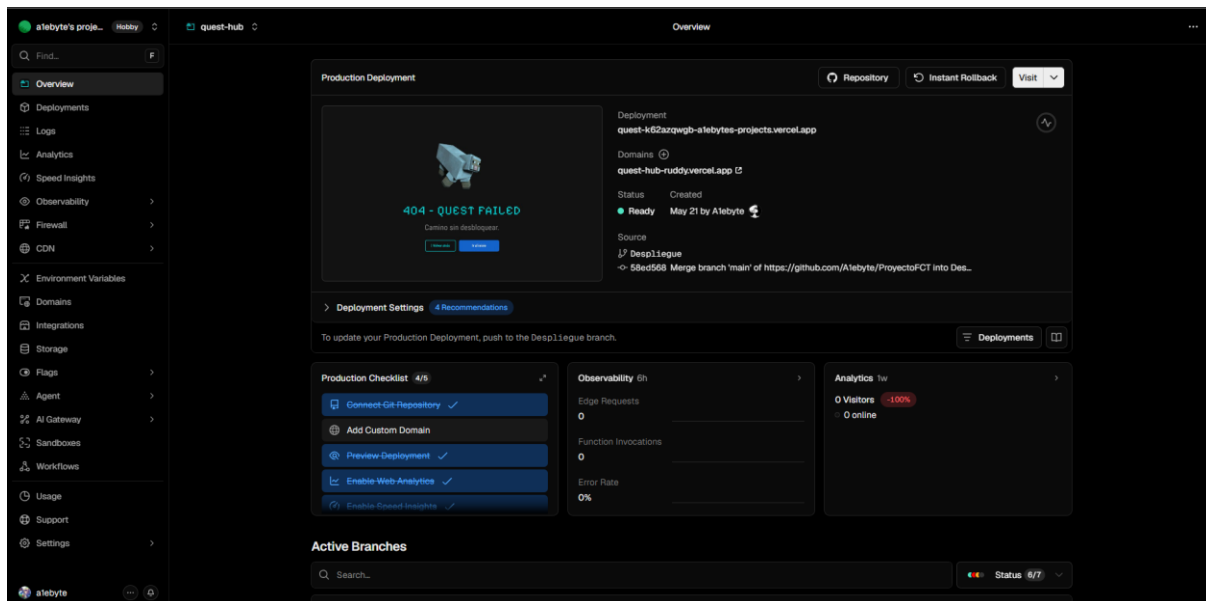


Figura 48. Dashboard del proyecto en Vercel

10.5. Despliegue de Supabase (Authentication)

Una de las partes más importantes fue el despliegue e integración del sistema de autenticación mediante **Supabase Auth**, lo cual nos permitió evitar desarrollar manualmente funcionalidades complejas relacionadas con usuarios como:

-  registro e inicio de sesión.
-  almacenamiento seguro de contraseñas.
-  gestión de sesiones.
-  validación de credenciales.
-  autenticación mediante proveedores externos.

Además de la autenticación tradicional mediante correo y contraseña, se integraron distintos proveedores **OAuth** para permitir el acceso utilizando cuentas de:

-  **Google**
-  **Discord**
-  **GitHub**

La integración básica con Supabase resultó relativamente sencilla, aunque la configuración de los proveedores externos presentó algunas dificultades iniciales. El caso más complejo fue **Google OAuth**, ya que fue necesario utilizar **Google Cloud Console** para:

- ❏ crear un proyecto,
- ❏ configurar la pantalla de consentimiento,
- ❏ registrar dominios autorizados,
- ❏ definir correctamente las URLs de redirección,
- ❏ generar las credenciales necesarias para Supabase.

Pequeños errores en estas configuraciones provocaban fallos frecuentes durante las primeras pruebas.

La integración con **Discord** fue más simple, siendo suficiente con crear una aplicación en el portal de desarrolladores, habilitar OAuth2 y configurar las URLs correspondientes.

En el caso de **GitHub**, los problemas encontrados estuvieron relacionados principalmente con permisos y visibilidad de la aplicación OAuth. Finalmente, se descubrió que la aplicación no había sido configurada correctamente como pública.

Además, Supabase permitió centralizar la gestión de usuarios y simplificar la validación de tokens JWT utilizados posteriormente por el backend para identificar usuarios autenticados.

En conjunto, **Supabase Auth** proporcionó una solución moderna y funcional para la autenticación del sistema, reduciendo considerablemente la complejidad del desarrollo y acercando el proyecto a una arquitectura similar a la utilizada en aplicaciones reales.

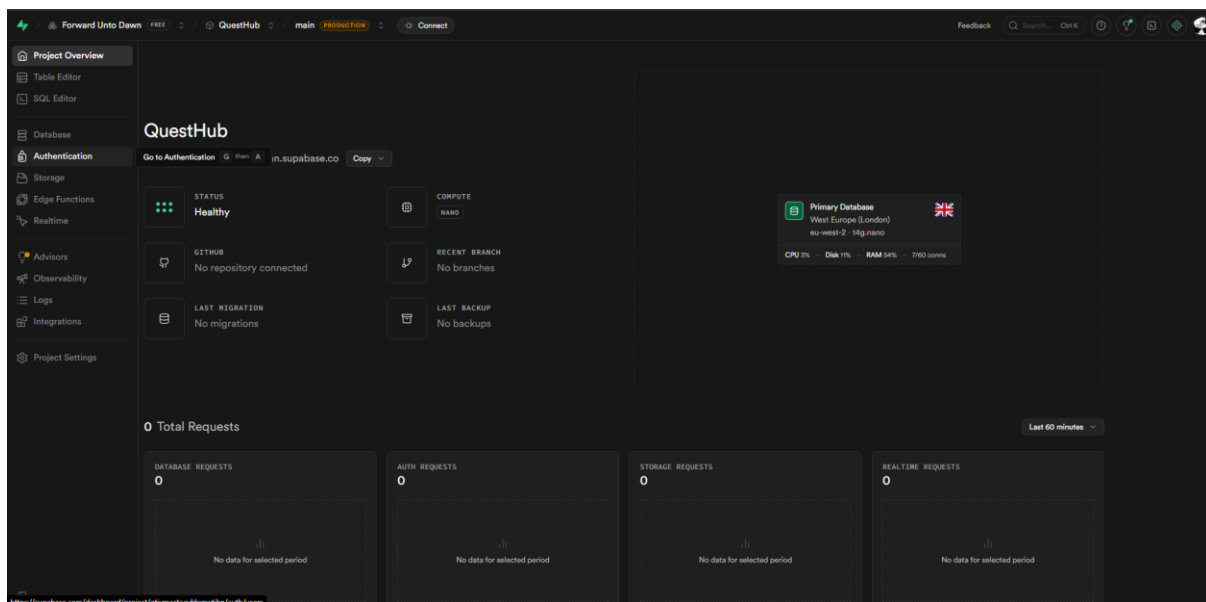


Figura 49. Dashboard del proyecto en Vercel

10.6. Despliegue de Resend (Correos)

Otra de las integraciones realizadas durante el desarrollo fue la implementación de un sistema de envío de correos electrónicos utilizando **Resend** como proveedor SMTP y servicio de mailing.

El objetivo principal de esta integración era permitir el envío automático de notificaciones relacionadas con ofertas y cambios en videojuegos o bundles guardados en wishlist. De esta forma, los usuarios pueden recibir avisos cuando alguno de sus elementos favoritos presenta nuevos descuentos o actualizaciones relevantes.

La integración con **Spring Boot** se realizó mediante configuración en el archivo *application.properties* y el uso de claves proporcionadas por Resend. Esto permitió establecer correctamente la conexión entre el backend y el servicio externo de correo.

Uno de los principales retos fue el diseño y generación de correos HTML dinámicos, ya que fue necesario trabajar con plantillas visuales compatibles con distintos clientes de email y generar contenido personalizado automáticamente.

Además, se implementó un sistema de envío automático mediante tareas programadas utilizando **@Scheduled**. El backend comprueba periódicamente si:

-  el usuario tiene activadas las notificaciones,
-  existen elementos en wishlist,
-  si alguna oferta ha cambiado recientemente.

En caso afirmativo, el sistema genera automáticamente un correo notificando los cambios detectados.

Actualmente, debido a las limitaciones del entorno gratuito de Resend y la ausencia de un dominio verificado, el servicio funciona en modo **sandbox/testing**, permitiendo únicamente el envío de correos a la dirección asociada a la cuenta principal utilizada durante el desarrollo.

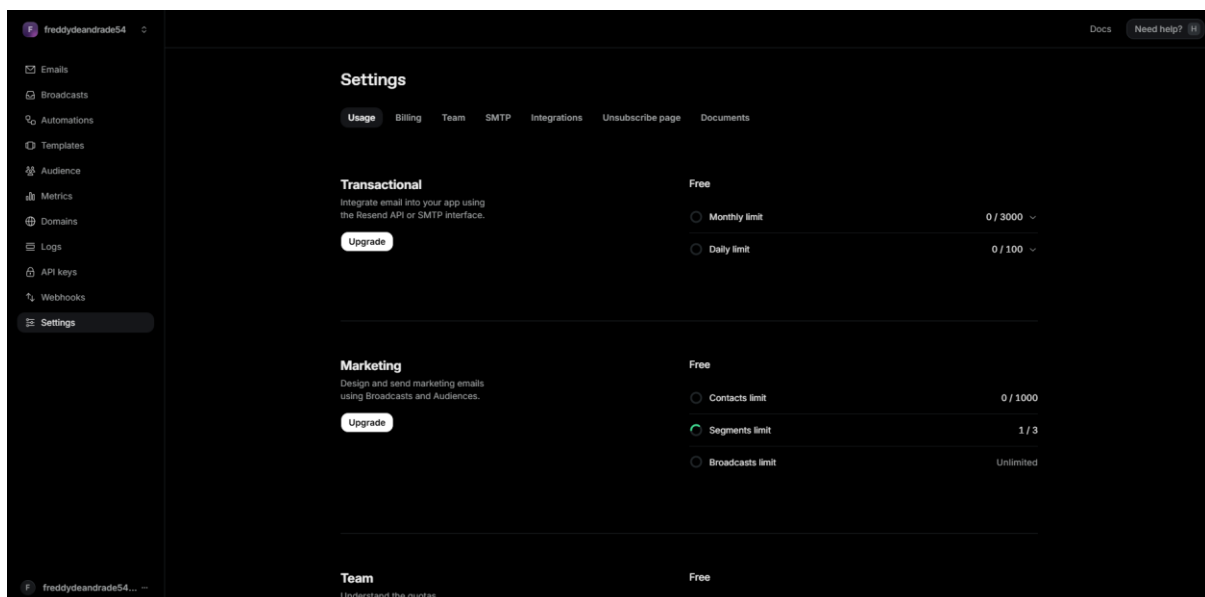


Figura 50. Dashboard del proyecto en resend

11. Problemas y errores

El desarrollo del proyecto no fue un camino libre de dificultades. A lo largo del proyecto surgieron diversos problemas tanto técnicos como no técnicos que afectaron al ritmo de trabajo, a la arquitectura del sistema y a la organización del equipo. Estas incidencias aparecieron por múltiples razones: integración con servicios externos, decisiones de diseño que evolucionaron con el tiempo, limitaciones de los entornos de despliegue y ajustes necesarios en la coordinación del equipo. En este apartado se recogen algunos de los problemas más relevantes encontrados, junto con su causa y la solución aplicada.

11.1. Errores técnicos

11.1.1. Bloqueos y límites de la API de CheapShark

Error: La API bloqueaba peticiones durante la sincronización, provocando fallos, evitando la sincronización de manera correcta y la entrega de datos incompletos.

Causa: CheapShark aplica límites estrictos de peticiones por minuto. Durante el desarrollo en local, la sincronización generaba un volumen elevado de solicitudes en poco tiempo, lo que provocaba bloqueos temporales de la API. En Railway el problema se intensificó, ya que las IPs son compartidas, y puede que otros proyectos usasen la API también y se hayan excedido esos límites previamente.

Solución:

-  Implementación de un **delay dinámico** entre peticiones.
-  Reducción del número de hilos async.
-  Inclusión de un **header personalizado** para evitar bloqueos.

11.1.2. Acceso concurrente durante la sincronización

Error: Los usuarios podían acceder a datos inconsistentes mientras se actualizaban las ofertas.

Causa: La sincronización se realizaba directamente sobre las tablas principales, lo que provocaba que durante el proceso los datos quedaran temporalmente en estados intermedios o eliminados. Esto significaba que algunas ofertas estaban actualizadas, otras no o hayan sido eliminadas, dejando el sistema con información incompleta o inexistente si un usuario accedía en ese momento.

Solución: Implementación de una **tabla staging** para aislar el proceso y garantizar consistencia.

11.1.3. Incompatibilidad de versiones de Spring Boot en Railway

Error: El backend fallaba al desplegarse en Railway.

Causa: Railway no soportaba correctamente la versión más reciente de Spring Boot utilizada en desarrollo, generando errores en el arranque.

Solución: Ajuste de la versión a Spring Boot 3.5.0, compatible con el entorno en la rama que esta asignada para los despliegues.

11.1.4. Errores de persistencia de datos

Error: Se produjeron fallos en la persistencia de distintas entidades, especialmente en las relaciones entre videojuegos e imágenes/vídeos y en la gestión de la wishlist. Estos problemas generaban registros duplicados, vacíos o referencias inválidas.

Causa: Los errores se debían principalmente a una gestión incorrecta del orden y la lógica de persistencia. En algunos casos, se intentaban crear relaciones antes de que la entidad principal estuviera guardada, lo que provocaba referencias no válidas. En otros, como en la wishlist, la lógica inicial no comprobaba si los datos ya existían, lo que ocasionaba duplicados o listas huérfanas. Además, el modelo relacional no estaba completamente alineado con el comportamiento esperado, lo que amplificaba estos problemas.

Solución:

-  Reordenar el flujo de guardado para asegurar que las entidades principales estuvieran persistentes antes de crear relaciones dependientes.
-  Rediseñar la relación entre usuario y wishlist para garantizar que cada usuario tuviera **una única lista**, reutilizándola en todas las operaciones.
-  Ajustar la lógica de persistencia para evitar duplicados y eliminar registros huérfanos.

11.1.5. Ofertas sin steamAppId en CheapShark

Error: Algunas ofertas no podían procesarse correctamente.

Causa: CheapShark no garantiza que todas las ofertas incluyan **steamAppId**, lo que rompía la lógica inicial del sistema que relacionaba la información de los juegos/bundles y las ofertas usando ese campo.

Solución: Implementación de un filtro para descartar los datos de ofertas que no llegasen con un **steamAppId**.

11.1.6. Errores en la tarea de sincronización

Error: Durante la sincronización se produjeron bloqueos, tiempos excesivos y errores silenciosos que afectaban tanto al rendimiento como a la estabilidad del backend.

Causa: El proceso de sincronización pasó por varias fases que generaron distintos problemas. Inicialmente, se ejecutaba de forma secuencial mediante un bucle sin ningún tipo de delay, lo que saturaba la API externa y bloqueaba el backend durante largos periodos. Para evitarlo, se añadió un delay fijo entre peticiones, pero esto provocó tiempos de sincronización demasiado largos. Posterior-

mente, se migró el proceso a un sistema asíncrono, pero la configuración inicial permitía lanzar demasiados hilos simultáneos y luego en el despliegue el error de bloqueo que se pensaban solucionados volvió a aparecer.

Solución:

-  Ajustar el proceso para combinar asincronía controlada con un **delay dinámico**, evitando saturar la API.
-  Reducir el número de hilos y optimizar la configuración async para equilibrar rendimiento y estabilidad.
-  Estabilizar el flujo de sincronización hasta lograr un proceso eficiente y seguro

11.1.7. Rediseño del modelo por bundles

Error: El modelo inicial no soportaba la gestión de bundles ni contemplaba su estructura particular, lo que impedía integrarlos correctamente en el sistema.

Causa: La funcionalidad de bundles no estaba prevista en el diseño original y sus datos diferían significativamente de los videojuegos. Mientras que un videojuego es una entidad única con atributos propios, un bundle no comparte suficientes campos con videojuegos para simplemente usar esta tabla para bundles y requiere relaciones adicionales para representar su contenido. Esta diferencia estructural obligó a replantear parte del modelo y backend, ya que las tablas existentes y métodos no podían adaptarse de forma sencilla a esta nueva entidad.

Solución:

-  Creación de una tabla específica para bundles, con su propia estructura y relaciones.
-  Ajuste del modelo de datos, las entidades asociadas y el backend para permitir la integración correcta de bundles dentro del sistema.

11.1.8. Sistema de filtros inconsistente

Error: Los filtros no se aplicaban correctamente o no se actualizaban.

Causa: El sistema de filtros combinaba estados internos del componente con parámetros en la URL, lo que hacía que cada parte gestionara su propia versión del filtro. Esto provocaba desincronización entre lo que mostraba la interfaz, lo que se enviaba al backend y lo que mostraba la URL. Como resultado, los filtros podían no aplicarse, aplicarse parcialmente o quedarse con valores antiguos, generando un comportamiento inconsistente.

Solución: Reescritura completa del sistema para depender únicamente de la URL.

11.1.9. Condiciones de carrera en peticiones

Error: Las peticiones realizadas desde el frontend provocaban errores cuando se ejecutaban de forma simultánea.

Causa: El frontend lanzaba varias solicitudes en paralelo sin ningún tipo de control ni coordinación entre ellas. Esto hacía que algunas peticiones terminaran antes que otras, sobrescribiendo datos o dejando el estado en un punto intermedio. En operaciones sensibles —como añadir o quitar elementos de la wishlist o actualizar información dependiente del backend— esta falta de orden provocaba resultados inconsistentes, duplicados temporales o datos que desaparecían y reaparecían según qué petición finalizara primero.

Solución:

-  Implementación de mecanismos de bloqueo temporal para evitar que el usuario enviara múltiples peticiones simultáneas sobre la misma acción.
-  Reestructuración del flujo para garantizar un **guardado asíncrono seguro**, asegurando que cada operación se completara antes de iniciar la siguiente cuando fuera necesario.

11.2. Errores no técnicos

11.2.1. Distribución desigual de la carga de trabajo

Error: La carga de trabajo no siempre estuvo equilibrada entre los miembros del equipo.

Causa: Existieron diferencias en disponibilidad y ritmo de avance, lo que provocó que ciertas tareas recayeran de forma desproporcionada en algunos integrantes.

11.2.2. Falta de homogeneidad

Error: Se encontraron funciones repetidas, estilos inconsistentes y fragmentos de código que no seguían un criterio común.

Causa: Aunque se definió varios detalles de estilos, en varias ocasiones no se siguieron, y además se trabajó sobre componentes de backend y frontend sin revisar previamente el código existente. Esto provocó que se implementaran soluciones nuevas donde ya existían otras similares, generando duplicados y diferencias notables en la estructura del código.

11.2.3. Falta de tiempo para completar el testing del backend

Error: No fue posible cubrir todo el backend con pruebas automatizadas.

Causa: El tiempo disponible se vio reducido por la acumulación de tareas críticas que fueron apareciendo durante el desarrollo. Además, la distribución desigual del trabajo hizo que ciertas partes avanzaran más rápido que otras, dejando el testing en un segundo plano. Como resultado, cuando se estabilizaron los módulos principales, el margen para implementar pruebas completas ya era limitado.

11.2.4. Uso de IA sin comprender el código generado

Error: Se integraron fragmentos de código generados por IA tanto en el frontend como en el backend sin una validación adecuada.

Causa: Algunos miembros copiaban y pegaban directamente las respuestas de la IA sin analizar su funcionamiento ni verificar si encajaba con la arquitectura del proyecto. Esto introdujo código que, aunque funcionaba en apariencia, era difícil de mantener, poco coherente con el resto del sistema y en ocasiones innecesariamente complejo. Además, la falta de comprensión sobre cómo operaban esos fragmentos dificultaba su depuración y evolución posterior, lo que trajo tener que realizar varias refactorizaciones del código.

12. Conclusiones

12.1. Conclusiones generales

El desarrollo de **QuestHub** ha permitido construir una aplicación web funcional, escalable y orientada a resolver un problema real: la fragmentación de información en el mercado digital de videojuegos. La plataforma centraliza ofertas procedentes de múltiples tiendas y permite al usuario encontrar de forma rápida la opción más económica, cumpliendo así el objetivo principal del proyecto.

A nivel técnico, el sistema destaca por su arquitectura desacoplada, el uso de tecnologías modernas y la integración con APIs externas. Uno de los logros más relevantes ha sido el sistema de sincronización automática, que permitió *“actualizar grandes volúmenes de información sin afectar a la disponibilidad de la aplicación”*. Este mecanismo, basado en tablas staging, tareas programadas y *cache*, ha sido clave para garantizar el rendimiento y la disponibilidad del sistema.

El proyecto también ha demostrado la importancia de una interfaz clara y minimalista, alineada con la necesidad detectada en el análisis previo. La experiencia de usuario ha sido un pilar fundamental del diseño enfocándonos.

En conjunto, QuestHub constituye un proyecto completo, técnicamente sólido y alineado con las prácticas actuales del desarrollo web profesional que, aunque posea sus fallos tiene bastante potencial a crecer en una aplicación web completa.

12.2. Aprendizaje y conocimientos adquiridos

El desarrollo de este proyecto ha supuesto un proceso de aprendizaje integral que abarca todas las fases del desarrollo web moderno trabajadas durante el ciclo formativo, las prácticas y el estudio autónomo. Cada tecnología, patrón y decisión arquitectónica ha aportado conocimientos que difícilmente podrían haberse adquirido únicamente mediante ejercicios teóricos, permitiendo enfrentarse a problemas reales y comprender cómo se construyen aplicaciones completas en un entorno profesional.

A nivel backend, trabajar con **Spring Boot** ha permitido entender cómo se estructura un sistema basado en controladores, servicios, repositorios, DTOs, mappers y validación. Se ha aprendido a diseñar endpoints REST robustos, manejar excepciones personalizadas, aplicar principios SOLID y organizar el código en capas bien definidas. La integración con APIs externas mediante RestClient ha sido especialmente relevante, ya que ha permitido comprender cómo gestionar errores, límites de peticiones y transformaciones de datos, reforzando la importancia de desacoplar la lógica interna de las fuentes externas.

En el frontend, el uso de **React** y TypeScript ha facilitado la creación de una interfaz modular, reutilizable y mantenible. El tipado estático ha permitido detectar errores de forma temprana y mejorar la coordinación dentro del equipo, mientras que herramientas como React Router, Framer Motion o Toastify han permitido implementar navegación dinámica, animaciones fluidas y notificaciones vi-

suales. También se ha trabajado en diseño responsive y en la creación de una experiencia visual minimalista sin depender de librerías externas de UI, lo que ha permitido un mayor control sobre la estética y la usabilidad.

La integración de **APIs externas** como Steam y CheapShark ha sido uno de los retos más significativos del proyecto. Se ha aprendido a consumir datos heterogéneos, gestionar paginación, normalizar información y diseñar procesos de sincronización que respeten las limitaciones de cada servicio. Este trabajo ha reforzado la importancia de disponer de una API propia que actúe como capa intermedia entre el frontend y las fuentes externas.

Otro aspecto clave ha sido el uso de asincronía mediante Spring Async y tareas programadas con Spring Scheduling. Gracias a ello, ha sido posible actualizar miles de registros sin afectar al rendimiento del sistema, garantizando la disponibilidad incluso durante operaciones pesadas. Junto a esto, la implementación de *caché* con Spring Cache y Caffeine ha permitido reducir tiempos de respuesta y disminuir la carga sobre APIs externas y la base de datos, comprendiendo cuándo es útil invalidar *caché* y cómo afecta al rendimiento global.

En el ámbito de la persistencia, trabajar con **PostgreSQL** ha permitido diseñar un modelo relacional complejo con múltiples relaciones, vistas materializadas, tablas intermedias y una tabla *staging* para sincronización eficiente. Se ha aprendido a optimizar consultas, gestionar índices, trabajar con triggers y diseñar estructuras escalables y mantenibles, aplicando conceptos avanzados de bases de datos relacionales.

El despliegue en entornos cloud como Vercel, Railway, Neon y Supabase ha aportado experiencia práctica en la configuración de servicios distribuidos, el uso de variables de entorno, la gestión de logs en producción, la separación de servicios y la integración continua. Finalmente, la incorporación de **testing** tanto en frontend como en backend ha reforzado la importancia de validar el comportamiento del sistema, detectar regresiones y garantizar la estabilidad del proyecto.

12.3. Objetivos alcanzados

Todos los objetivos principales del proyecto han sido cumplidos, tanto en el plano funcional como en el técnico, consolidando a **QuestHub** como una plataforma web alineada con los requisitos definidos al inicio del desarrollo. El sistema ha logrado implementar una **arquitectura desacoplada** basada en frontend, backend y base de datos independientes, lo que ha permitido trabajar cada componente de forma modular y garantizar la escalabilidad futura del proyecto.

En el ámbito del backend, se ha desarrollado una **API REST robusta** utilizando Spring Boot, capaz de gestionar videojuegos, ofertas, bundles, usuarios, wishlist y notificaciones. Esta API actúa como intermediaria entre el frontend y las APIs externas, cumpliendo el objetivo de desacoplar completamente la interfaz de usuario de las fuentes externas de datos. Asimismo, se ha implementado un sistema de autenticación seguro mediante Supabase y JWT, cumpliendo el objetivo de ofrecer un acceso fiable y protegido a las funcionalidades del sistema.

En cuanto al frontend, se ha logrado construir una **interfaz moderna y responsive** utilizando React y TypeScript, priorizando la claridad visual, la rapidez de navegación y la experiencia de usuario.

Funcionalidades como la búsqueda dinámica, la wishlist personalizada y la visualización de contenido multimedia han sido implementadas con éxito, cumpliendo los objetivos funcionales establecidos.

A nivel de persistencia, se ha diseñado un modelo de datos completo en PostgreSQL, incorporando relaciones complejas, vistas materializadas y una tabla *staging* para la sincronización de ofertas. Esto ha permitido cumplir el objetivo de gestionar información persistente de forma eficiente y escalable.

Además, se han alcanzado objetivos avanzados como la sincronización automática de datos mediante tareas programadas, el uso de *caché* para optimizar el rendimiento y el despliegue completo en entornos cloud independientes (Vercel, Railway, Neon y Supabase), acercando el proyecto a un escenario de producción real.

Finalmente, se han abordado retos técnicos reales que han enriquecido el desarrollo del proyecto. Entre ellos destacan la necesidad de mantener la disponibilidad del sistema durante procesos de actualización masiva, la gestión eficiente de grandes volúmenes de datos procedentes de APIs externas y la integración coordinada de múltiples servicios distribuidos. Estos desafíos han permitido aplicar soluciones prácticas propias del desarrollo profesional, consolidando la madurez técnica del proyecto y demostrando la capacidad de diseñar y mantener un sistema web complejo y funcional.

12.4. Mejoras a futuro

Aunque **QuestHub** alcanza una fase beta plenamente funcional y cumple con los objetivos planteados, existen diversas mejoras que permitirían ampliar sus capacidades, enriquecer la experiencia del usuario y aumentar el valor del sistema. Varias de estas funcionalidades no se implementaron en esta versión debido a las limitaciones de tiempo propias del proyecto académico, pero sí fueron consideradas desde el diseño inicial para facilitar su incorporación en futuras iteraciones. Entre las mejoras más relevantes se encuentran:

-  Historial de precios
-  Recomendaciones personalizadas
-  Integración con nuevas APIs
-  Gráficos estadísticos
-  Recuperación y cambio de contraseña
-  Soporte para mercados de claves
-  Mejoras de interfaz y accesibilidad
-  Mayor cobertura de testing
-  Optimización del sistema de sincronización

Estas mejoras son importantes porque permitirían evolucionar QuestHub hacia una plataforma más completa, competitiva y alineada con las expectativas actuales de los usuarios. Funcionalidades como el historial de precios, las recomendaciones personalizadas o la integración con nuevas APIs aportarían un valor añadido significativo, mientras que otras, como la recuperación de contraseña, la mejora de accesibilidad o el aumento de la cobertura de testing, reforzarían la calidad técnica y la usabilidad del sistema.

Asimismo, la incorporación de estas mejoras permitiría continuar desarrollando el proyecto con una base sólida, ya que la arquitectura actual ha sido diseñada pensando en la escalabilidad y en la posibilidad de integrar nuevas funcionalidades sin necesidad de modificar la estructura principal del sistema.

12.5. Últimos pensamientos

El desarrollo de este TFG ha representado una oportunidad única para aplicar de forma práctica los conocimientos adquiridos durante el ciclo formativo y enfrentarse a retos propios del desarrollo web profesional. A lo largo del proyecto se ha construido una plataforma completa, funcional y escalable, capaz de integrar datos procedentes de múltiples fuentes externas, gestionar grandes volúmenes de información y ofrecer una experiencia de usuario clara y moderna. La arquitectura implementada, basada en la separación entre frontend, backend y base de datos, ha demostrado ser sólida y adecuada para futuras ampliaciones, permitiendo incorporar nuevas funcionalidades sin comprometer la estructura principal del sistema.

El proyecto no solo ha permitido consolidar habilidades técnicas en áreas como **desarrollo backend**, **interfaces modernas**, **bases de datos relacionales** o **despliegue en la nube**, sino también adquirir experiencia en la resolución de problemas reales, la toma de decisiones arquitectónicas y la gestión de procesos complejos como la sincronización automática de datos. Asimismo, ha puesto de manifiesto la importancia de la planificación, el trabajo en equipo y la capacidad de adaptación ante imprevistos técnicos.

Aunque algunas funcionalidades han quedado fuera por limitaciones de tiempo, la base construida permite continuar evolucionando QuestHub hacia una plataforma más completa y competitiva. El proyecto finaliza en una fase beta plenamente operativa, con un amplio margen de crecimiento y con una arquitectura preparada para afrontar futuras mejoras.

En definitiva, QuestHub constituye un reflejo del aprendizaje, el esfuerzo y la evolución técnica alcanzada durante todo el proceso formativo, demostrando la capacidad de diseñar, desarrollar y desplegar una aplicación web moderna, funcional y orientada a resolver una necesidad real del usuario.



13. Bibliografía

-  **Spring Boot Documentation** — Documentación oficial de Spring Boot.
-  **React Documentation** — Documentación oficial de React.
-  **TypeScript Handbook** — Guía oficial de TypeScript.
-  **PostgreSQL Documentation** — Documentación oficial de PostgreSQL.
-  **Supabase Documentation** — Documentación oficial de Supabase.
-  **CheapShark API** — Documentación de la API pública de CheapShark.
-  **Steam Web API** — Documentación de la API de Steamworks.
-  **Railway Documentation** — Documentación de Railway para despliegue de servicios.
-  **Vercel Documentation** — Documentación de la plataforma Vercel.
-  **Neon Database Documentation** — Documentación de Neon Serverless Postgres.
-  **Caffeine Cache** — Documentación de la librería Caffeine para *caché*.
-  **Framer Motion Documentation** — Documentación oficial de Framer Motion.
-  **Axios Documentation** — Documentación del cliente HTTP Axios.
-  **React Toastify Documentation** — Documentación de React Toastify.
-  **JWT Documentation** — Información oficial sobre JSON Web Tokens.
-  **GitHub Documentation** — Documentación de GitHub y GitHub Actions.
-  **REST Architectural Style** — Recursos generales sobre el estilo arquitectónico REST.
-  **Apuntes de Desarrollo Cliente Servidos** — Materiales proporcionados en clase sobre arquitectura web, APIs y patrones de diseño.
-  **Apuntes de Desarrollo React** — Materiales proporcionados en clase sobre arquitectura web, APIs y patrones de diseño.
-  **Apuntes de Bases de Datos** — Recursos docentes sobre SQL, normalización y diseño relacional.
-  **Material de prácticas de empresa** — Recursos y guías utilizados durante el periodo de prácticas.
-  **Stack Overflow** — Comunidad de desarrolladores para resolución de dudas técnicas.
-  **Stack Overflow en español** — Versión en español de la comunidad.
-  **Dev.to** — Comunidad de artículos técnicos y guías prácticas.
-  **Reddit r/programming** — Comunidad de discusión sobre desarrollo y tecnologías.
-  **GitHub Issues** — Seguimiento de problemas y soluciones en repositorios oficiales.
-  **Para desarrollo** — Asistencia en generación de código, localización de errores y resolución de dudas técnicas.
-  **Para documentación** — Uso de herramientas de IA para aclarar conceptos y mejorar redacción técnica.